# 目录

**量子物理计算方法选讲 · 第二讲逐页中文讲义** **2**
这份文档是什么 . . . 2
怎么用这份讲义 . . . 3
全局约定（先读一遍） . . . 3
逐页索引 . . . 4

**第一部分：精确对角化总览（第 1–7 页）** **5**
第 1 页 · 封面 . . . 5
第 2 页 · 精确对角化（ED）：本讲路线图 . . . 6
第 3 页 · 为什么要做精确对角化 . . . 8
第 4 页 · 希尔伯特空间的大小 . . . 9
第 5 页 · 精确对角化的应用 . . . 11
第 6 页 · 今天能做到多大 . . . 12
第 7 页 · 精确对角化程序的结构 . . . 14

**第二部分：比特、基矢与自旋哈密顿量（第 8–15 页）** **16**
第 8 页 · 基矢：自旋态 ↔ 二进制 ↔ 十进制 . . . 16
第 9 页 · 比特运算（一）：按位运算与移位 . . . 18
第 10 页 · 比特运算（二）：置位、清零、翻转、读位 . . . 20
第 11 页 · 比特运算（三）：数位、取位、循环移位 . . . 22
第 12 页 · 自旋系统：哈密顿量矩阵的对角元 . . . 24
第 13 页 · 自旋系统：非对角元 . . . 26
第 14 页 · 非对角元全表：稀疏矩阵的三元组 . . . 28
第 15 页 · 示例代码：一维反铁磁海森堡链 . . . 30

**第三部分：对称性与基矢查找（第 16–20 页）** **34**
第 16 页 · 对称性：块对角化 . . . 34
第 17 页 · 实现 U(1) 对称性 . . . 36
第 18 页 · 基矢查找：两次直接内存读取 . . . 38
第 19 页 · Lin 表（一）：A / B 分块 . . . 40
第 20 页 · Lin 表（二）：论文 Table II 与推广问题 . . . 42

**第四部分：费米子、有限温度与磁化（第 21–24 页）** **44**
第 21 页 · 费米子系统（一）：Hubbard 模型与符号约定 . . . 44
第 22 页 · 费米子系统（二）：带符号的基矢操作 . . . 46
第 23 页 · 有限温度：期望值、电荷密度与 $\mu$–$n$ 曲线 . . . 48
第 24 页 · 磁化：均匀/交错磁化与磁化率 . . . 51

**第五部分：工具链：Python 包（第 25–27 页）** **53**
第 25 页 · Python 包（一）：完全对角化 . . . 53
第 26 页 · Python 包（二）：迭代对角化 . . . 55
第 27 页 · Python 包（三）：matrix-free 与 `LinearOperator` . . . 57

**第六部分：实时演化与非厄米（第 28–32 页）** **60**

第 28 页 · 实时演化：形式解与 Lanczos 加速 . . . . . 60
第 29 页 · 实时演化实例（一）：高斯波包与色散 . . . . . 62
第 30 页 · 实时演化实例（二）：环上磁通与非扩散波包 . . . . . 64
第 31 页 · 实时演化实例（三）：开边界、自干涉与量子复兴 . . . . . 66
第 32 页 · 非厄米哈密顿量：双正交基与时间演化 . . . . . 68

**第七部分：作业（第 33–34 页）** **70**
第 33 页 · 作业规则 . . . . . 70
第 34 页 · 本次作业评分标准 . . . . . 72

**附录** **73**
附录 A · 中英术语对照 . . . . . 73
附录 B · 常见错误清单（写 ED 代码前先读一遍） . . . . . 75
附录 C · 延伸阅读 . . . . . 76
结语 . . . . . 77

# 量子物理计算方法选讲 · 第二讲逐页中文讲义

**（配套第二讲幻灯片，共 34 页，逐页对照）**

---

## 这份文档是什么

这是一份**逐页对照**的中文讲义：第二讲幻灯片的每一页，都按原来的顺序在下面出现一次，先给出这一页的**原图**，再给出这一页的**中文对译、背景补充**和**要点展开**。目标是：你拿着这一份文档（配合幻灯片本身），就能把第二讲的全部内容完整读懂、跟上。

第一讲把整门课的地图铺开了——**一个方程**（定态薛定谔方程 $H|\Psi\rangle = E|\Psi\rangle$）、**四类方法**（精确对角化、张量网络、量子蒙特卡罗、其它）。第二讲开始兑现地图上的第一块：**精确对角化（Exact Diagonalization, ED）**。这一讲是“第一章的上半段”，主线是**如何把一个量子多体问题变成一个可以交给计算机的稀疏矩阵**。

**关于插图**：本文档中的 34 张插图，是把幻灯片原页逐页渲染出来的（200 dpi），放在与本文档同目录的 `slides/` 文件夹里，用相对路径 `slides/slide-NN.png` 引用，在支持 Markdown 的编辑器（VS Code、Typora、Obsidian 等）里可以直接显示。

**页码约定**：本文中“第 $N$ 页”一律指**第二讲**幻灯片第 $N$ 页。

**与第一讲的关系**：本讲大量使用第一讲第 16–19 页的二次量子化与张量积语言（产生/湮灭算符、占据数表象、$2^N$ 指数墙）。读本讲之前，建议把第一讲第 16–19 页再翻一遍；本讲第 21–22 页的费米子符号，就是第一讲第 17 页反对易关系的“代码版”。

---

## 怎么用这份讲义

每一页都按下面几块组织，你可以按需跳读：

- **这一页写了什么** ——把幻灯片上的英文条目（含公式）翻成中文，尽量逐条对应。
- **图里画了什么** ——幻灯片上的示意图、曲线、表格、截图里都画了些什么。这一块**只在有图的页面上单独给出**；其余页面的图意就写在“这一页写了什么”里。
- **要点展开** ——这一页真正想讲的物理与数学；公式的来历、数字的量级、为什么重要。
- **提示 / 易错** ——实践中的经验，以及这一页最容易误解、或幻灯片排版容易看错的地方。

读法建议：先自己看一眼原图和“这一页写了什么”，再读“要点展开”。本讲有一半是**代码页**，读的时候请**同时打开编辑器**把代码敲一遍——第二讲的幻灯片本质上是一份“可以照着写出来的程序说明书”。

---

## 全局约定（先读一遍）

幻灯片没有集中列出约定，但它的公式与示例代码默认了下面这套写法。**大多数“莫名其妙”的符号错误和下标错误，都是约定不一致造成的**，不是物理错误。

| 量 | 本讲（第二讲）约定 |
|---|---|
| 单位 | $\hbar = 1$，$k_B = 1$，晶格常数 $a = 1$；能量以耦合常数（$J$ 或 $t$）为单位 |
| 自旋算符 | $S^\alpha = \frac{1}{2}\sigma^\alpha$，$\sigma^z = \mathrm{diag}(1, -1)$ |
| 自旋基 | $\lvert\uparrow\rangle = (1,0)^\mathsf{T}$，$\lvert\downarrow\rangle = (0,1)^\mathsf{T}$；$\uparrow \leftrightarrow$ **比特 1**，$\downarrow \leftrightarrow$ **比特 0** |
| 基矢整数标号 | $n = \sum_i n_i 2^i$，**格点 $i$ 对应第 $i$ 位**；格点 0 是**最低位（最右边）**，格点 $N-1$ 是最左边 |
| 比特位序 | 写成二进制串时**从左到右是第 $N-1, N-2, \ldots, 0$ 位**；`ReadBit(n,i) = (n >> i) & 1` |
| 自旋哈密顿量 | $H = \sum_i \mathbf{S}_i \cdot \mathbf{S}_{i+1}$，环上取 $S_N \equiv S_0$；开链时去掉最后一条键 |
| 费米子基 | $\lvert 0\rangle =$ 空，$\lvert 1\rangle =$ 占据；**算符串中把所有 $\uparrow$ 放在 $\downarrow$ 的左边** |
| 费米子比特布局 | 低 $N$ 位放 $\uparrow$，高 $N$ 位放 $\downarrow$：$n = n_\uparrow + 2^N n_\downarrow$ |
| 温度 | $\beta = 1/(k_B T)$，$k_B = 1$；实时间 $t$，$\hbar = 1$ |
| 边界条件 | 除特别说明外，自旋模型画在**环**上（周期性边界）；代码示例是**开链**，请注意区分 |

> **注意 · 与第一讲“全局约定表”的差异（务必注意）**
>
> 第一讲全局约定表写的是“格点 0 在**最左边**、$n = \sum_i s_i 2^{N-1-i}$（格点 0 是**最高位**）”。第二讲的幻灯片与示例代码用的是**相反**的标签：**格点 $i$ 就是第 $i$ 位，格点 0 在最右边**。证据有两个：第 8 页把 $\lvert 0000\rangle$ 的最左边标为“第 3 位”、最右边标为“第 0 位”；第 21 页的费米子例子中，$\uparrow$ 的比特串 `1101` 恰好对应格点 $\{0, 2, 3\}$，$\downarrow$ 的 `1010` 对应 $\{1, 3\}$，只有“格点 $i$ = 第 $i$ 位”才成立。

**这只是标签约定，不改变任何物理**（相当于把格点编号整体反了一个方向）。但**在同一份代码里必须二选一**：如果你把第一讲第 19 页用 `np.kron` 写的矩阵与第二讲用比特运算写的矩阵混在一起，格点顺序就会反，所有局域量（关联函数、结构因子、边界项）都会错。**本讲义全部采用第二讲的约定。**

---

## 逐页索引


| 页码 | 主题 |
|---|---|
| 1 | 封面与课程信息 |
| 2 | 精确对角化（ED）：本讲路线图 |
| 3 | 为什么要做精确对角化 |
| 4 | 希尔伯特空间的大小 |
| 5 | 精确对角化的应用 |
| 6 | 今天能做到多大（present day limits） |
| 7 | 精确对角化程序的结构 |
| 8 | 基矢：自旋态 ↔ 二进制 ↔ 十进制 |
| 9 | 比特运算（一）：按位运算与移位 |
| 10 | 比特运算（二）：置位、清零、翻转、读位 |
| 11 | 比特运算（三）：数位、取位、循环移位 |
| 12 | 自旋系统：哈密顿量矩阵的对角元 |
| 13 | 自旋系统：非对角元 |
| 14 | 非对角元全表：稀疏矩阵的三元组 |
| 15 | 示例代码：一维反铁磁海森堡链 |
| 16 | 对称性：块对角化 |
| 17 | 实现 U(1) 对称性 |
| 18 | 基矢查找：两次直接内存读取 |
| 19 | Lin 表（一）：A / B 分块 |
| 20 | Lin 表（二）：论文 Table II 与推广问题 |
| 21 | 费米子系统（一）：Hubbard 模型与符号约定 |
| 22 | 费米子系统（二）：带符号的基矢操作 |
| 23 | 有限温度：期望值、电荷密度与 $\mu$–$n$ 曲线 |
| 24 | 磁化：均匀/交错磁化与磁化率 |
| 25 | Python 包（一）：完全对角化 |
| 26 | Python 包（二）：迭代对角化 |
| 27 | Python 包（三）：matrix-free 与 `LinearOperator` |
| 28 | 实时演化：形式解与 Lanczos 加速 |
| 29 | 实时演化实例（一）：高斯波包与色散 |
| 30 | 实时演化实例（二）：环上磁通与非扩散波包 |
| 31 | 实时演化实例（三）：开边界、自干涉与量子复兴 |
| 32 | 非厄米哈密顿量：双正交基与时间演化 |
| 33 | 作业规则 |

| 页码 | 主题 |
|---|---|
| 34 | 本次作业评分标准 |


## 第一部分：精确对角化总览（第 1–7 页）

### 第 1 页 · 封面

Sep 21, 2026

# Selected Topics in Computational Quantum Physics
# 量子物理计算方法选讲


Shuo Yang（杨硕）

*Department of Physics, Tsinghua University*

Email: shuoyang@tsinghua.edu.cn

WeChat: condmat-ys


图 1: 幻灯片第 1 页

#### 这一页写了什么

课程名称（中英文）、授课教师杨硕（清华大学物理系）、邮箱 `shuoyang@tsinghua.edu.cn`、微信 `condmat-ys`，以及讲课日期 **2026 年 9 月 21 日**。

#### 要点展开

- 这是本学期的第二次课，也是**第 1 章“精确对角化”的第一次课**（第一讲第 8 页的课程大纲里，第 1 章就是“精确对角化：哈密顿量的矩阵表示、Lanczos 方法、时间演化、动量空间精确对角化、各种应用”；本讲是这个大章的前半场 **ED I**）。从今天起，课程从“地图”进入“第一块工地”。

- 与第一讲的关系可以概括成一句话：**第一讲告诉你 $H|\Psi\rangle = E|\Psi\rangle$ 有多难；第二讲告诉你，怎么在小格子上真的把它解出来。**
- 精确对角化是四类方法里**唯一不做任何物理近似**的方法。它只有两种误差：**有限尺寸**（格子太小）和**浮点舍入**（$10^{-15}$ 量级）。正因为它"诚实"，它才成为其它所有方法的**裁判**（第 3 页的第 3 条理由）。

**提示 / 易错**

- 本讲幻灯片标题写的是 *Exact Diagonalization*，缩写 **ED**。后面讲义里 ED 一律指精确对角化，不要与"电动力学"（electrodynamics）混。

---

### 第 2 页 · 精确对角化（ED）：本讲路线图

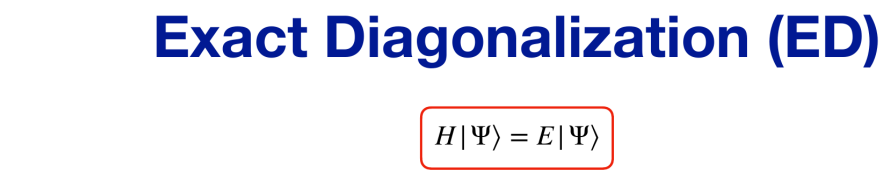



图 2: 幻灯片第 2 页

**这一页写了什么**

标题 *Exact Diagonalization (ED)*，中间用一个方框突出核心方程

$$H\,|\Psi\rangle = E\,|\Psi\rangle,$$

下面列出七个主题：

- main idea（基本思想）
- matrix representation of Hamiltonian（哈密顿量的矩阵表示）
- implement symmetries（对称性的实现）
- Lanczos method（Lanczos 方法）
- time evolution（时间演化）
- finite temperature（有限温度）
- various applications（各种应用）

**要点展开**

这一页是全讲的**目录页**，也是整章的骨架。它对应第一讲第 15 页那个方程 $H|\Psi\rangle = E|\Psi\rangle$——ED 的全部工作就是把这个方程“翻译”成矩阵问题：

$$\underbrace{H}_{\text{一个算符}} \longrightarrow \underbrace{\mathsf{H}_{mn} = \langle m|H|n\rangle}_{\text{一个矩阵}} \longrightarrow \underbrace{\mathsf{H}\,\mathbf{c} = E\,\mathbf{c}}_{\text{一个线性代数问题}} .$$

七个主题可以归成**四个模块**（正是第 7 页的四块）：

1. **基矢**（第 8–11 页）：希尔伯特空间的每个基矢用一个整数表示，用比特运算做“字典”。
2. **哈密顿量矩阵**（第 12–15、21–22 页）：把 $H$ 的每一项翻译成“行、列、值”三元组；自旋模型用 $S^{\pm}$，费米子模型还要多记一个符号。
3. **对称性与查找**（第 16–20 页）：用守恒量把大矩阵切成小块，并高效地“由整数找到下标”。
4. **线性代数与应用**（第 23–32 页）：完全/迭代对角化、有限温度、实时演化、非厄米。

**提示 / 易错**

- 幻灯片把“本讲路线图”和“第 1 章目录”放在一起，**顺序就是讲课顺序**，请按这个顺序读。
- 注意这里说的是 *matrix representation*，不是“矩阵求逆”：ED 的核心动作是 **构造矩阵 + 求本征值**，而不是解线性方程组。

---

### 第 3 页 · 为什么要做精确对角化

# Why exact diagonalization?

- complete knowledge of a quantum system can be obtained
  with the eigenstates available, any static or dynamic quantity can be computed

- insights gained from exact diagonalization studies are very useful
  in there own right and as a complement to other calculations

- exact results for small lattices are indispensable for testing the correctness of other methods
  e.g., DMRG, QMC, TN, etc

- exact diagonalization methods provide a concrete path for learning many important aspects of quantum mechanics
  in particular the symmetry properties of many-body states

图 3: 幻灯片第 3 页

#### 这一页写了什么

- *complete knowledge of a quantum system can be obtained* ——有了本征态，任何静态或动态的量都能算出来。
- *insights gained from exact diagonalization studies are very useful in their own right and as a complement to other calculations* ——从 ED 研究中得到的洞察本身就很有用，也能补充其它计算。
- *exact results for small lattices are indispensable for testing the correctness of other methods*，例如 DMRG、QMC、TN 等。
- *exact diagonalization methods provide a concrete path for learning many important aspects of quantum mechanics*，特别是多体态的对称性质。

#### 要点展开

四条理由，一条比一条“功利”：

1. **完备性**。一旦拿到本征对 $\{E_n, |\psi_n\rangle\}$，你就拥有了一切：基态能量、关联函数 $\langle\psi_0|S^z_i S^z_j|\psi_0\rangle$、动力学谱 $A(k,\omega)=\sum_n |\langle\psi_n|S^z_k|\psi_0\rangle|^2\delta(\omega-(E_n-E_0))$、有限温度的 $\langle O\rangle=\mathrm{Tr}(e^{-\beta H}O)/\mathrm{Tr}(e^{-\beta H})$、实时间演化 $e^{-iHt}|\psi_0\rangle$。**这就是 ED 与 DMRG/QMC 最大的差别**：后两者通常只给你基态或少量低能态，而 ED 给你**整个谱**（在能算得动的维数内）。

2. **洞察**。小格子上的精确解经常直接给出物理图像：比如自旋液体、拓扑序、分数化的激发、简并的基态多重态——这些概念很多是先在小簇的 ED 里看清楚，再推广的。
3. **裁判**。这是 ED 在当代最不可替代的角色。DMRG、QMC、张量网络、神经网络量子态都有各自的近似/系统误差（键维数、符号问题、抽样噪声、表达力），而小格子（$N = 16 \sim 24$）的 ED 结果是**精确的**，可以直接拿来对答案。做新方法的人第一件事几乎都是“先和 ED 对上”。
4. **学习量子力学**。ED 强迫你把波函数、算符、对称性、守恒量全部**具体化**：基矢是整数、算符是比特翻转、对称性是块对角。上完这一章，第一讲第 11–19 页那些抽象概念都会“落地”。

**提示 / 易错**

- 幻灯片把 *in their own right* 误写成 *in there own right*，是笔误。
- **不要把 ED 想成“只能算玩具”**：第 6 页会看到，今天自旋模型的 ED 已经能做到 40–48 个格点、十亿量级的基矢——这足以回答很多二维阻挫磁体的真问题。

---

## 第 4 页 · 希尔伯特空间的大小

# Hilbert space sizes

- the Hilbert space of a quantum many body system grows exponentially in general
- for $N$ spin 1/2 particles, the complete Hilbert space has dim = $2^N$ states

  10 spins dim = 1'024

  30 spins dim = 1'073'741'824

  50 spins dim = 1'125'899'906'842'624 ...
- the quantum mechanical wave function is a vector in this Hilbert (vector) space and we would like to know the ground state and a few other low lying eigenstates
- exact diagonalization studies are limited to rather small lattices

  great care therefore has to be taken in drawing conclusions about thermodynamic limit, which may not even be possible

图 4: 幻灯片第 4 页

**这一页写了什么**

- 量子多体系统的希尔伯特空间一般**随粒子数指数增长**。
- 对 $N$ 个自旋-1/2，完整希尔伯特空间维数 $\dim = 2^N$：

  - 10 spins: dim = 1 024
  - 30 spins: dim = 1 073 741 824
  - 50 spins: dim = 1 125 899 906 842 624 ⋯
- 量子力学波函数是这个（矢量）空间里的一个矢量；我们想知道**基态和少数几个低激发态**。
- ED 研究局限于**相当小的格子**；因此从 ED 结果推断热力学极限时必须**非常小心**，有时甚至根本不可能。

**要点展开**

这是第一讲第 19 页“指数墙”的**量化版**。把这几个数字和“内存”联系起来看：

| $N$ | $2^N$ | 一个复态矢量（16 byte/分量） |
|---|---|---|
| 10 | 1 024 | 16 KB |
| 20 | 1 048 576 | 17 MB |
| 30 | $1.07 \times 10^9$ | 17 GB |
| 40 | $1.10 \times 10^{12}$ | **17.6 TB** |
| 50 | $1.13 \times 10^{15}$ | 18 PB |

也就是说，**仅仅存放 $N = 40$ 的一个波函数就要 17.6 TB 内存**——比一台机器的内存大三个数量级。而哈密顿量如果是稠密的，还需要 $(2^N)^2$ 个元素，那是完全不可能的。所以 ED 的三条活路是：

1. **只存基矢中真正用到的部分**（对称性扇区，第 17 页）；
2. **只存矩阵的非零元**（稀疏，第 14 页）；
3. **不显式存矩阵**（matrix-free，第 27 页）。

第二点是“稀疏”的用武之地：Heisenberg 模型每个基矢只和**每条键**的一个伙伴相连，非零元个数 $\sim N \cdot 2^N$，而不是 $2^{2N}$。

**提示 / 易错**

- 幻灯片上“dim = $2N$”是上标排版丢失，应为 $2^N$。
- 数字里用的是**瑞士式千位分隔符**（`1'024`），等价于 `1,024`。
- **“波函数是指数维空间里的一个矢量”** 这句话要背下来：后面所有方法的差别，都在于“怎么表示这个矢量”（ED 用完整矢量、MPS 用张量、QMC 用抽样）。

---

### 第 5 页 · 精确对角化的应用

# Exact diagonalization: applications

- quantum magnets: nature of novel phases, critical points in 1D, dynamical correlation functions in 1D & 2D
- fermionic models (Hubbard / *t-J*): gaps, pairing properties, correlation exponents, etc
- fractional quantum Hall states: energy gaps, overlap with model states, entanglement spectra
- quantum dimer models or other constrained models (anyon chains, ...)
- full Configuration Interaction (CI) in quantum chemistry, nuclear structure
- quantum field theory

图 5: 幻灯片第 5 页

#### 这一页写了什么

- **quantum magnets**：新奇的相、一维临界点、一维与二维的动力学关联函数。
- **fermionic models（Hubbard / t-J）**：能隙、配对性质、关联指数等。
- **fractional quantum Hall states**：能隙、与模型波函数的重叠、纠缠谱。
- **quantum dimer models 或其它受约束模型**（任意子链等）。
- **full Configuration Interaction (CI)**：量子化学与核结构中的全组态相互作用。
- **quantum field theory**：量子场论。

#### 要点展开

按“算的是什么量”来分：

- **量子磁体**：这是 ED 的主战场。典型问题有——阻挫磁体的基态是“有序”还是“自旋液体”？一维链的临界指数、二维方格子/三角格子/笼目格子的低能谱与低能隙；动力学结构因子 $S(k,\omega)$（可以直接和中子散射实验比）。
- **费米子模型**：Hubbard 与 $t$–$J$ 模型的能隙（charge gap / spin gap）、超导配对关联 $\langle\Delta_i^\dagger\Delta_j\rangle$、关联函数幂律指数。这类模型需要处理**费米子符号**（第 21–22 页）。
- **分数量子霍尔**：在球面/环面几何上对角化（最多 16–20 个电子），算能隙、与 Laughlin/Moore–Read 等模型波函数的重叠、以及**纠缠谱**（第 1 章后半段会讲）。

- **受约束模型**：量子二聚体模型（每个键上恰好一个二聚体）需要**自定义基矢生成**（不是简单的 $2^N$），这类“非张量积的希尔伯特空间”是 ED 灵活性的体现。
- **量子化学 / 核结构**：full CI 就是化学家对“ED”的叫法——在给定的单粒子轨道基里做完全对角化；区别只是轨道数少（十几个）、但每个轨道的相互作用更复杂。
- **量子场论**：把连续场论离散到格点上（例如 $1+1$ 维的 $\phi^4$、Ising 场论），用 ED 做小格子的精确基准。

**提示 / 易错**

- 幻灯片说 *quantum magnets* 里 *dynamical correlation functions in 1D & 2D*，这里 **1D 能用 DMRG 做到上千格点**，而 ED 的价值在于**二维**（DMRG 在二维很吃力）。
- 注意“full CI”与“ED”是**同一件事的两种叫法**，只是社区不同。

---

## 第 6 页 · 今天能做到多大

# Exact diagonalization: present day limits

- spin $S = 1/2$ models

  square lattice: $N = 40$, triangular lattice: $N = 39$, Honeycomb lattice: $N = 42$, kagome lattice: $N = 48$, maximum dimension of basis: 1.5 billion

- fractional quantum hall effect

  different filling fractions ν, up to 16-20 electrons

  maximum dimension of basis: 3.5 billion

- Hubbard models

  square lattice at half filling: $N = 20$, triangular lattice: $N = 21$, honeycomb lattice: $N = 24$, quantum dot structure: $N = 20$

  maximum dimension of basis: 3 billion

- Holstein models

  chain with $N = 14$ + phonon pseudo-sites

  maximum dimension of basis: 30 billion

图 6: 幻灯片第 6 页

**这一页写了什么**

标题 *Exact diagonalization: present day limits*。

- **spin $S = 1/2$ models**

  - 方格子：$N = 40$；三角格子：$N = 39$；蜂窝格子：$N = 42$；笼目格子：$N = 48$
  - maximum dimension of basis（最大基矢维数）：**1.5 billion**（15 亿）
- **fractional quantum Hall effect**
  - 不同填充因子 $\nu$，最多 16–20 个电子
  - 最大基矢维数：**3.5 billion**
- **Hubbard models**
  - 半满方格子：$N = 20$；三角格子：$N = 21$；蜂窝格子：$N = 24$；量子点结构：$N = 20$
  - 最大基矢维数：**3 billion**
- **Holstein models**
  - 链：$N = 14$ + 声子赝格点（phonon pseudo-sites）
  - 最大基矢维数：**30 billion**

**要点展开**

这页给出的是**“今天”（2026 年）的实际能力边界**，值得逐条理解：

- **自旋模型为什么能到 40–48 个格点？**因为它每个格点只有 2 个态，而且**对称性特别多**（第 16 页）。以 40 格点方格子为例：仅用 $S^z = 0$ 扇区，维数是 $C_{40}^{20} \approx 1.38 \times 10^{11}$；再除以平移群（40 个元素）和点群（4 个元素），就降到 $8.6 \times 10^8$ 量级——与幻灯片说的“最大基矢维数 15 亿”**在同一量级**（具体数字取决于用了哪几种对称性、以及是哪个格子）。**对称性是这里最大的加速来源。**
- **Hubbard 模型为什么只能到 20–24 个格点？**因为每个格点有 4 个态（空、↑、↓、双占），半满时 $S^z = 0$ 扇区的维数是 $C_N^{N/2} \times C_N^{N/2}$ 量级，比同尺寸自旋模型大得多；而且还要处理费米子符号。
- **Holstein 模型为什么基矢维数反而最大（300 亿）？**因为声子是玻色子，每个格点有多个声子态，需要用**截断的声子赝格点**（pseudo-site）；基矢维数大但**每个基矢的非零元很少**，且可以用 Lanczos 只求少数低能态。
- **FQH 的 16–20 个电子**：电子在最低朗道能级里的态数是 $N_\phi + 1$，在固定总角动量/动量扇区里做 ED；维数可达 35 亿，是 ED 的另一个极限场景。

**提示 / 易错**

- 这页的数字都是“**最大基矢维数**”不是“总希尔伯特空间维数”——前者是用了对称性之后**实际对角化的那个矩阵的大小**。两者可能差好几个数量级（第 17 页会讲怎么用 U(1) 把 $2^N$ 切小）。
- “$N = 48$ **的笼目格子**”是 ED 的标志性成果（笼目格子的自旋液体问题），值得记住这个数字。
- 表格里的 $N$ 是**格点数**，不是电子数。Hubbard 半满时电子数 $= N$。

---

### 第 7 页 · 精确对角化程序的结构

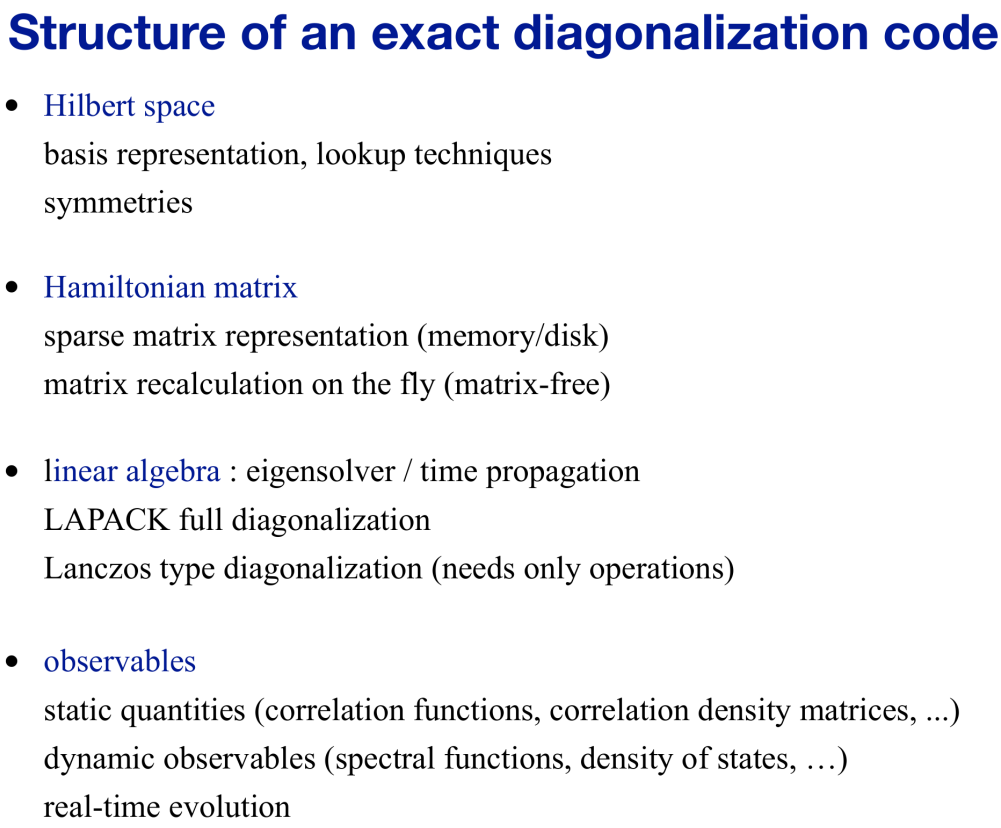



图 7: 幻灯片第 7 页

#### 这一页写了什么

- **Hilbert space**（希尔伯特空间）
  - basis representation, lookup techniques（基矢表示、查找技术）
  - symmetries（对称性）
- **Hamiltonian matrix**（哈密顿量矩阵）
  - sparse matrix representation (memory/disk)（稀疏矩阵表示）
  - matrix recalculation on the fly (matrix-free)（实时重算 / 无矩阵）
- **linear algebra: eigensolver / time propagation**（线性代数：本征求解 / 时间传播）
  - LAPACK full diagonalization（LAPACK 完全对角化）
  - Lanczos type diagonalization (needs only operations)（Lanczos 型对角化，只需要“算符作用”）
- **observables**（可观测量）
  - static quantities（静态量：关联函数、关联密度矩阵等）
  - dynamic observables（动力学量：谱函数、态密度等）
  - real-time evolution（实时演化）

#### 要点展开

这是**本讲的“程序流程图”**也是你自己写 ED 代码时应该采用的**模块划分**。强烈建议把文件也按这四块来组织：

```
ed.py
├── basis    :  生成基矢、比特运算、扇区划分、查找表
├── hamiltonian :  由基矢生成 (row, col, value) 三元组 → scipy.sparse
├── solver   :  eigh（完全）/ eigsh（迭代）/ expm_multiply（时间演化）
└── measure  :  关联函数、磁化、谱函数、热平均
```

两个关键词需要重点理解：

- **sparse（稀疏）**：哈密顿量在**每个基矢上只有 $O(N)$ 个非零元**。以 Heisenberg 环为例：每个基矢有 1 个对角元，加上“相邻自旋相反”的键贡献的非对角元（平均 $N/2$ 条），所以 $\mathrm{nnz} \approx (1+N/2)\,2^N$。例如 $N=4$ 环：$(1+2)\times 16 = 48$，与第 14 页的数出来的 48 个非零元完全一致。当 $N=40$、$2^N = 10^{12}$ 时这不现实，但在 $N \le 24$、维数 $10^6 \sim 10^7$ 时完全可行。
- **matrix-free（无矩阵）**：连稀疏矩阵都不存，只写一个函数 `apply_H(v) -> H @ v`。这样内存只要 $O(2^N)$（几个矢量），代价是每次“作用”都要现场算一遍。第 27 页的 `LinearOperator` 就是干这个的。

**提示 / 易错**

- “Lanczos type diagonalization (**needs only operations**)”这句是关键：Lanczos 不需要矩阵元，只需要 $H$ 对矢量的作用。**这正是 matrix-free 的动机。**
- **不要把“完全对角化”和“迭代对角化”混用**：完全对角化（`eigh`）给全部本征值，代价 $O(D^3)$、内存 $O(D^2)$；迭代对角化（`eigsh`）只给少数极值本征值，代价 $\sim O(D^2)$ 或更低、内存 $O(D)$。这是第 25–26 页的主题。

---

# 第二部分：比特、基矢与自旋哈密顿量（第 8–15 页）

## 第 8 页 · 基矢：自旋态 ↔ 二进制 ↔ 十进制

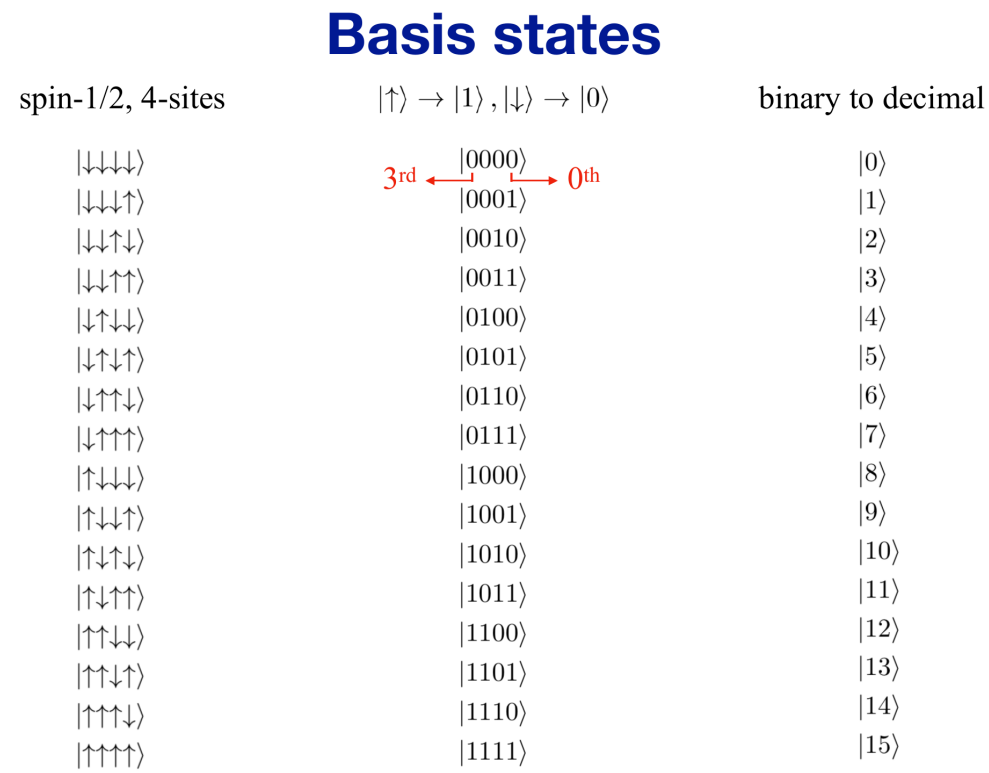



图 8: 幻灯片第 8 页

### 这一页写了什么

标题 *Basis states*，副标题 *spin-1/2, 4-sites* 与 *binary to decimal*。核心是一张 16 行的对照表，把**自旋态、比特串、整数**三者一一对应：

| spin-1/2, 4-sites | $\|\uparrow\rangle \to \|1\rangle,\ \|\downarrow\rangle \to \|0\rangle$ | binary to decimal |
|---|---|---|
| $\|\downarrow\downarrow\downarrow\downarrow\rangle$ | $\|0000\rangle$ | $\|0\rangle$ |
| $\|\downarrow\downarrow\downarrow\uparrow\rangle$ | $\|0001\rangle$ | $\|1\rangle$ |
| $\|\downarrow\downarrow\uparrow\downarrow\rangle$ | $\|0010\rangle$ | $\|2\rangle$ |
| $\|\downarrow\downarrow\uparrow\uparrow\rangle$ | $\|0011\rangle$ | $\|3\rangle$ |
| $\|\downarrow\uparrow\downarrow\downarrow\rangle$ | $\|0100\rangle$ | $\|4\rangle$ |
| $\|\downarrow\uparrow\downarrow\uparrow\rangle$ | $\|0101\rangle$ | $\|5\rangle$ |
| $\|\downarrow\uparrow\uparrow\downarrow\rangle$ | $\|0110\rangle$ | $\|6\rangle$ |
| $\|\downarrow\uparrow\uparrow\uparrow\rangle$ | $\|0111\rangle$ | $\|7\rangle$ |
| $\|\uparrow\downarrow\downarrow\downarrow\rangle$ | $\|1000\rangle$ | $\|8\rangle$ |
| $\|\uparrow\downarrow\downarrow\uparrow\rangle$ | $\|1001\rangle$ | $\|9\rangle$ |

| spin-1/2, 4-sites | $\|\uparrow\rangle \to \|1\rangle,\ \|\downarrow\rangle \to \|0\rangle$ | binary to decimal |
|---|---|---|
| $\|\uparrow\downarrow\uparrow\downarrow\rangle$ | $\|1010\rangle$ | $\|10\rangle$ |
| $\|\uparrow\downarrow\uparrow\uparrow\rangle$ | $\|1011\rangle$ | $\|11\rangle$ |
| $\|\uparrow\uparrow\downarrow\downarrow\rangle$ | $\|1100\rangle$ | $\|12\rangle$ |
| $\|\uparrow\uparrow\downarrow\uparrow\rangle$ | $\|1101\rangle$ | $\|13\rangle$ |
| $\|\uparrow\uparrow\uparrow\downarrow\rangle$ | $\|1110\rangle$ | $\|14\rangle$ |
| $\|\uparrow\uparrow\uparrow\uparrow\rangle$ | $\|1111\rangle$ | $\|15\rangle$ |

**图里画了什么**

第一行 $|0000\rangle$ 的上方有红色标注：左边写 **“3rd ←,”** 右边写 **“→ 0th.”** 意思是：比特串**最左边是第 3 位**（最高位，权重 $2^3 = 8$），**最右边是第 0 位**（最低位，权重 $2^0 = 1$）。这一条约定在第 9–11 页的代码里反复用到。

**要点展开**

- **为什么可以用一个整数代表一个多体基矢?** 因为自旋-1/2 每个格点只有两个状态，正好对应一个二进制位（bit）。$N$ 个格点 $\leftrightarrow$ $N$ 位二进制数 $\leftrightarrow$ $[0, 2^N - 1]$ 的整数。这个对应关系是整章的地基：**“基矢”就是一个 `int`，“波函数”就是一个长度为 $2^N$ 的复数数组**。
- **比特位与格点的对应**：本讲约定**格点 $i$ = 第 $i$ 位**，即

$$n = \sum_{i=0}^{N-1} n_i\, 2^i, \qquad n_i = \text{格点} i \text{ 的占据数 } (\uparrow\colon 1,\ \downarrow\colon 0).$$

  所以表格中 $|0001\rangle = 1$ 表示**格点 0 向上、格点 1–3 向下**（因为第 0 位是最右边那一位）。
- **“从左到右是第 $N-1$ 位到第 0 位”** 这是写给人看的排版约定，与“格点 $i$ = 第 $i$ 位”是同一件事的两种说法。**看表时先看红色标注，不要想当然。**
- 表中 16 个态按**整数递增**排列，这正好就是“字典序”。后面第 17 页要做的，就是把这张表按“↑ 的个数”重新分桶。

**提示 / 易错**

- **最容易错的地方**：$|\downarrow\downarrow\downarrow\uparrow\rangle = 1$ 还是 8? 按本讲约定是 1（最右边是格点 0，权重 1）。如果你从第一讲的 `np.kron` 路线过来，直觉会给出 8。**两者都对，但必须全篇统一**（见前面“全局约定”的差异提醒）。
- **调试建议**：永远用 `np.binary_repr(n, width=N)` 打印，不要靠脑子做二进制心算。
- 注意“↑→ 1”这个映射是**人为选择**；选 ↑→ 0 也完全可以，只是所有公式里的 `-0.5` 要改成 `+0.5`（第 12 页）。

### 第 9 页 · 比特运算（一）：按位运算与移位

# Bit operations

- bitwise operations

x & y　　x | y　　x ^ y　　~ x

```
AND | 0 1     OR | 0 1     XOR | 0 1     NOT | 0 1
----+-----    ---+----    ----+----    ----+----
 0  | 0 0      0 | 0 1      0 | 0 1        | 1 0
 1  | 0 1      1 | 1 1      1 | 1 0
```

```
    201: 1100 1001
AND  15: 0000 1111
------------------
 IS   9  0000 1001
```

- bit shift

x << y
x >> y

```
x = 1        # 0001
x << 2       # Shift left 2 bits: 0100
# Result: 4
```

- show bit configuration

```
>>> np.binary_repr(3)
'11'
>>> np.binary_repr(-3)
'-11'
>>> np.binary_repr(3, width=4)
'0011'
```

```
>>> np.base_repr(5)
'101'
>>> np.base_repr(6, 5)
'11'
>>> np.base_repr(7, base=5, padding=3)
'00012'
```

图 9: 幻灯片第 9 页

#### 这一页写了什么

标题 *Bit operations*。

- **bitwise operations**（按位运算）：`x & y`、`x | y`、`x ^ y`、`~x`
- **bit shift**（移位）：`x << y`、`x >> y`
- **show bit configuration**（显示比特配置）

幻灯片给出的是四张真值表（AND / OR / XOR / NOT）、一个例子和两张移位示意图。

#### 图里画了什么

- 三张 $2\times 2$ 真值表：`x&y`（只有 $1\&1=1$，其余为 0）、`x|y`（只有 $0|0=0$，其余为 1）、`x^y`（相异为 1，相同为 0）。
- 一张一维的 NOT 表：`~0 = 1`、`~1 = 0`（**按位**取反）。
- 例子：$201\ \&\ 15 = 9$。写成二进制就一目了然：

$$201 = 1100\,1001_2, \qquad 15 = 0000\,1111_2,$$

$$201\,\&\,15 = 0000\,1001_2 = 9.$$

这个例子其实就是“**取低 4 位**”——正是第 11 页 `PickBit` 的雏形。

- 移位示意：`x << 2` 把比特整体左移两位（低位补 0），$x = 1 = 0001_2 \to 0100_2 = 4$；`x >> 2` 右移两位，$1 \to 0$。

**要点展开**

四种按位运算在 ED 里各有分工：

| 运算 | 记法 | 在 ED 里的用途 |
|---|---|---|
| AND | `x & y` | **读位**：`x & (1<<n)`；取掩码：`x & mask` |
| OR | `x \| y` | **置位**：`x \| (1<<n)` |
| XOR | `x ^ y` | **翻位**：`x ^ (1<<n)`（自旋翻转 $= S^{\pm}$） |
| NOT | `~x` | 配合 AND 做**清零**：`x & ~(1<<n)` |

移位则负责“**把位搬到正确的位置**”：

- `x << n`：把第 $i$ 位搬到第 $i+n$ 位（左移 $n$）；
- `x >> n`：把第 $i$ 位搬到第 $i-n$ 位（右移 $n$）。

**关于 Python 的 `~`**：Python 的整数是任意精度、且用补码语义，所以 `~x = -x-1`（例如 `~5 = -6`），看起来“很怪”。但 `x & ~(1<<n)` 是安全的：`~(1<<n)` 的二进制是“除了第 $n$ 位以外全是 1”，AND 上去正好把第 $n$ 位清零，高位不受影响。**只要不单独使用 `~x` 就没问题。**

**提示 / 易错**

- **位运算的优先级**：在 Python 里 `&` 的优先级**低于** `==` 和 `+`，所以 `x & 1 == 1` 会被解析成 `x & (1 == 1)`！写代码时**永远加括号**：`(x & 1) == 1`。
- `np.binary_repr(n, width=4)` 给固定宽度（前面补 0），`np.base_repr(n, base=2)` 不给前导 0。**调试时用前者**，因为位对齐很重要。
- 幻灯片上的真值表是**按位**的，不是逻辑运算（`and`/`or`/`not`）；在 numpy 数组上要区分 `&`（按位）和 `np.logical_and`（逻辑）。

---

## 第 10 页 · 比特运算（二）：置位、清零、翻转、读位

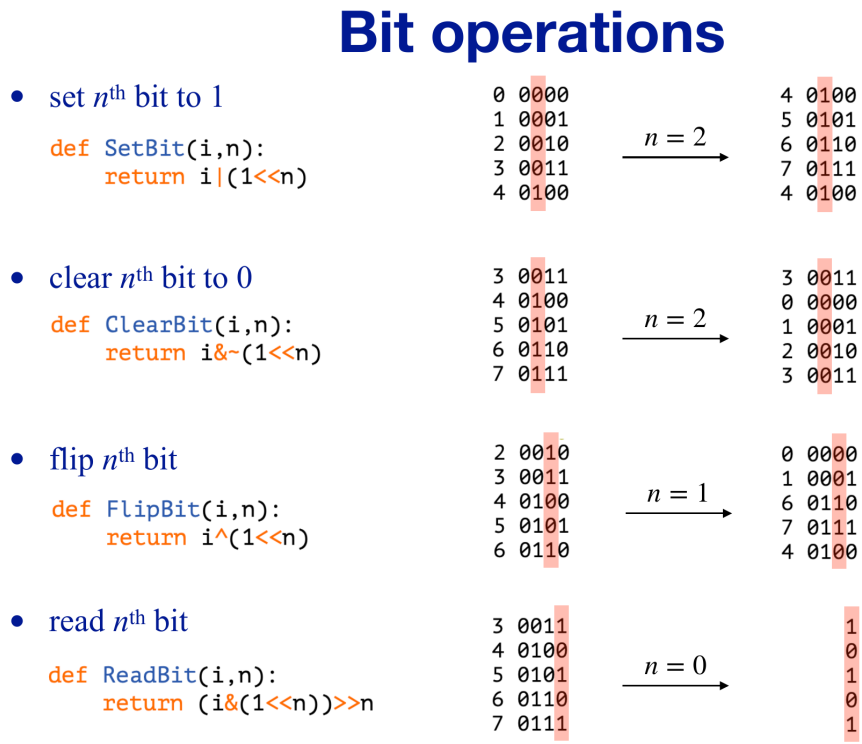



图 10: 幻灯片第 10 页

### 这一页写了什么

标题 *Bit operations*，四个基本操作，每个配一张“操作前 / 操作后”的比特图：

- **set nth bit to 1**（把第 $n$ 位置 1）：$\mathrm{SetBit}(i,n) = i \mid (1 \ll n)$
- **clear nth bit to 0**（把第 $n$ 位清 0）：$\mathrm{ClearBit}(i,n) = i \;\&\; \sim(1 \ll n)$
- **flip nth bit**（翻转第 $n$ 位）：$\mathrm{FlipBit}(i,n) = i \;\mathrm{XOR}\; (1 \ll n)$
- **read nth bit**（读取第 $n$ 位）：$\mathrm{ReadBit}(i,n) = (i \;\&\; (1 \ll n)) \gg n$

幻灯片上给出的参数是 $n = 2$（置位、清零）、$n = 1$（翻转）、$n = 0$（读位）。

### 要点展开

这四个函数就是**整个 ED 的“指令集”**——第 12–15 页的哈密顿量，全部由它们拼出来。逐个看一遍：

- **`SetBit(i,n)`**：`i | (1<<n)`。因为 `1<<n` 只有第 $n$ 位是 1，OR 上去就把它强制置 1，其余位不变。
- **`ClearBit(i,n)`**：`i & ~(1<<n)`。`~(1<<n)` 是“除了第 $n$ 位全是 1”的掩码，AND 上去就把第 $n$ 位强制清 0。
- **`FlipBit(i,n)`**：`i ^ (1<<n)`。XOR 与 1 相遇就翻转，与 0 相遇不变。**这是自旋翻转 $S^{\pm}$ 的全部内容**——第 13 页的非对角元就是“翻转两个比特”。

- **`ReadBit(i,n)`**：先 AND 取出第 $n$ 位（结果要么是 0 要么是 $2^n$），再右移 $n$ 位归一化成 0 或 1。**返回的是整数 0/1，不是布尔值。**

一个具体的例子（本讲义自加，方便对照幻灯片上的比特图）：取 $i = 5 = 0101_2$。

| 操作 | 计算 | 结果 | 二进制 |
|---|---|---|---|
| `SetBit(5,1)` | $5 \mid 2 = 7$ | 7 | `0111` |
| `ClearBit(5,2)` | $5 \;\&\; \sim 4 = 1$ | 1 | `0001` |
| `FlipBit(5,1)` | $5 \;\mathrm{XOR}\; 2 = 7$ | 7 | `0111` |
| `ReadBit(5,2)` | $(5 \;\&\; 4) \gg 2 = 1$ | 1 | `1` |

**提示 / 易错**

- **`ReadBit` 的返回值必须归一化到 0/1**。如果你忘了 `>> n`，得到的是 0 或 $2^n$，在 `(ReadBit(i,Pos0)-0.5)` 里就会变成 $-0.5$ 或 $2^n - 0.5$——这是初学者最常踩的坑（第 15 页的示例代码是对的，但你自己写的时候务必检查一遍）。
- **自旋与比特的换算**：$S_i^z = \mathrm{ReadBit}(i, \mathrm{site}) - \frac{1}{2}$，即 $\uparrow \to +1/2$，$\downarrow \to -1/2$。
- 在 C/C++ 里要用 `1u << n` 或 `1LL << n` 防止溢出；Python 没有这个问题（任意精度整数），但也因此**更慢**，大规模 ED 里常用 `numpy` 的 `uint64` 数组做批量位运算。

---

## 第 11 页 · 比特运算（三）：数位、取位、循环移位

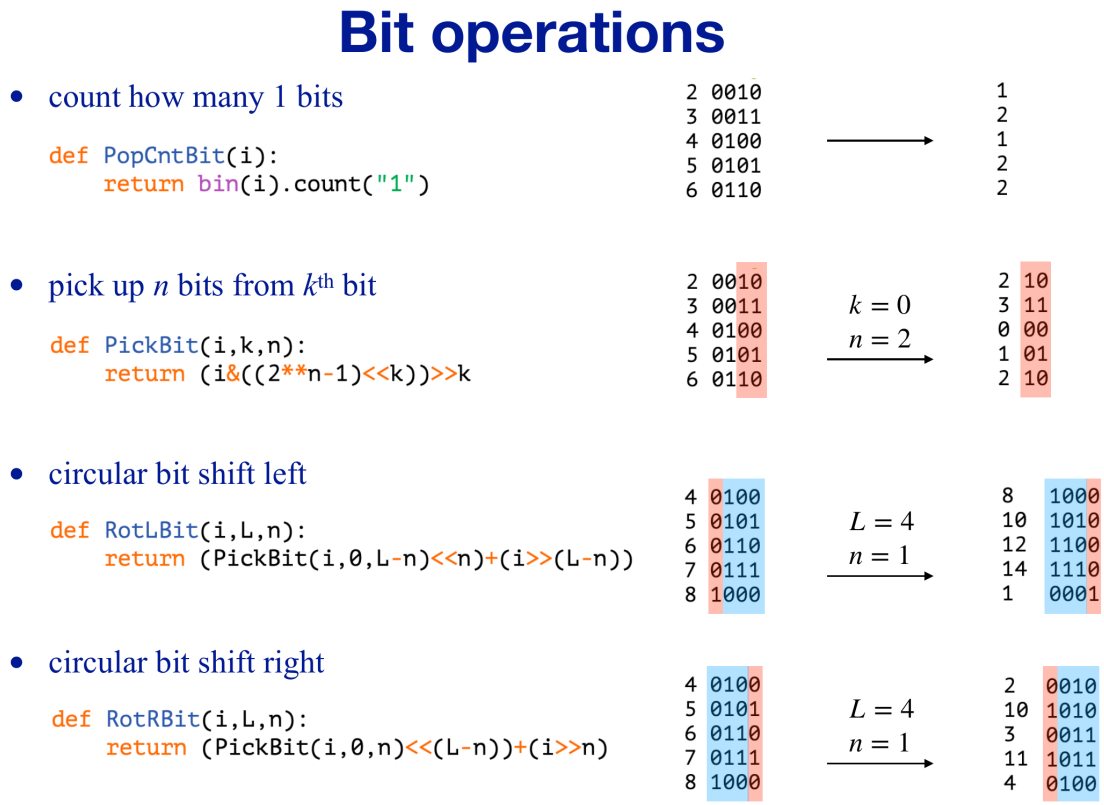



图 11: 幻灯片第 11 页

### 这一页写了什么

标题 *Bit operations*，四个进阶操作：

- **count how many 1 bits**（数 1 的个数，popcount）：

$$\mathrm{PopCntBit}(i) = \texttt{bin}(i).\texttt{count}(\texttt{"1"})$$

- **pick up n bits from kth bit**（从第 $k$ 位起取 $n$ 位），参数 $k=0,\ n=2$：

$$\mathrm{PickBit}(i,k,n) = (i \ \&\ ((2^n-1) \ll k)) \gg k$$

- **circular bit shift left**（循环左移），参数 $L=4,\ n=1$：

$$\mathrm{RotLBit}(i,L,n) = (\mathrm{PickBit}(i,0,L-n) \ll n) + (i \gg (L-n))$$

- **circular bit shift right**（循环右移），参数 $L=4,\ n=1$：

$$\mathrm{RotRBit}(i, L, n) = \big(\mathrm{PickBit}(i, 0, n) \ll (L - n)\big) + \big(i \gg n\big)$$

**要点展开**

这四个操作各自对应一个**物理用途**：

1. **`PopCntBit` = 数自旋向上的格点数**。因为 $\uparrow \to 1$，所以 popcount 就是 $N_\uparrow$，而

$$S^z_{\mathrm{tot}} = \sum_i S^z_i = N_\uparrow - \frac{N}{2}.$$

   **这是 U(1) 扇区的划分依据**（第 17 页）。

2. **`PickBit(i,k,n)` = 取出比特串的一段**。掩码 $(2^n - 1) \ll k$ 是“从第 $k$ 位起连续 $n$ 个 1”，AND 之后右移 $k$ 位就把它对齐到低位。用途：把整数拆成“前段 / 后段”，这正是 Lin 表（第 19 页）的关键。

3. **`RotLBit` / `RotRBit` = 环上的平移**。在**周期性边界**的环上，把整个自旋构型沿环平移一格，就是**循环移位**。用途：实现平移对称性（第 16 页），或者生成一个动量扇区里的全部基矢。注意这里 $L$ 是**总比特数**（这里是 4），$n$ 是平移的格点数。

用幻灯片给的参数（$L = 4$，$n = 1$）验算一遍，取 $i = 13 = 1101_2$：

| 操作 | 计算 | 结果 | 二进制 |
|---|---|---|---|
| `PopCntBit(13)` | `bin(13).count("1")` | 3 | `11` |
| `PickBit(13,1,2)` | $(13 \,\&\, 6) \gg 1 = 4 \gg 1$ | 2 | `10` |
| `RotLBit(13,4,1)` | $(5 \ll 1) + (13 \gg 3) = 10 + 1$ | 11 | `1011` |
| `RotRBit(13,4,1)` | $(1 \ll 3) + (13 \gg 1) = 8 + 6$ | 14 | `1110` |

对照：`1101` 循环左移 1 位得 `1011` ✓，循环右移 1 位得 `1110` ✓。

**提示 / 易错**

- **循环移位 ≠ 普通移位**：普通移位会“丢掉”移出去的位并补 0；循环移位把丢掉的位补到另一端。**只有循环移位才是环上的平移。**如果你用 `<<`/`>>` 做平移，边界上的基矢会变成“自旋跑出去了”的非法态，而且会丢掉真正的边界项。
- **`PopCntBit` 用 `bin(i).count("1")` 在 Python 里是“优雅但慢”的写法**。如果要在 $10^7$ 个基矢上反复调用，用查表（把 16 位一块的 popcount 预先存好）或者 `numpy` 的 `np.bitwise_count`（NumPy $\geq 2.0$）会快几十倍。也可以用 `int.bit_count()`（Python $\geq 3.10$）。
- **`RotLBit` 里的 `L` 必须等于比特串的总长度**。如果你用 `L=4` 却处理 6 格点的态，平移就会把第 4、5 位搅乱。

---

### 第 12 页 · 自旋系统：哈密顿量矩阵的对角元

# Spin system - Hamiltonian matrix

- spin-1/2 AF Heisenberg model on a ring

$$H = \sum_{i=0}^{N-1} \mathbf{S}_i \cdot \mathbf{S}_{i+1} = \sum_{i=0}^{N-1} \left( S_i^x S_{i+1}^x + S_i^y S_{i+1}^y + S_i^z S_{i+1}^z \right)$$
$$= \sum_{i=0}^{N-1} \left[ \left( S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+ \right)/2 + S_i^z S_{i+1}^z \right]$$

- periodic boundary condition $S_N = S_0$

- diagonal terms $\langle \mathrm{col} | S_i^z S_{i+1}^z | \mathrm{col} \rangle$

$${}_i\langle \uparrow | S_i^z | \uparrow \rangle_i = +0.5, \quad {}_i\langle \downarrow | S_i^z | \uparrow \rangle_i = 0, \quad \Rightarrow \quad {}_i\langle 1 | S_i^z | 1 \rangle_i = +0.5, \quad {}_i\langle 0 | S_i^z | 1 \rangle_i = 0,$$

$${}_i\langle \downarrow | S_i^z | \downarrow \rangle_i = -0.5, \quad {}_i\langle \uparrow | S_i^z | \downarrow \rangle_i = 0, \quad \Rightarrow \quad {}_i\langle 0 | S_i^z | 0 \rangle_i = -0.5, \quad {}_i\langle 1 | S_i^z | 0 \rangle_i = 0,$$

$$\langle \mathrm{col} | S_i^z S_{i+1}^z | \mathrm{col} \rangle = \langle S_i^z \rangle \langle S_{i+1}^z \rangle$$

$$\langle S_i^z \rangle = \mathrm{ReadBit(col, i)} - 0.5$$

图 12: 幻灯片第 12 页

#### 这一页写了什么

标题 *Spin system - Hamiltonian matrix*。

- **spin-1/2 AF Heisenberg model on a ring**（环上的自旋-1/2 反铁磁海森堡模型）
- **periodic boundary condition**（周期性边界条件）
- **diagonal terms**（对角元）

幻灯片先给出单格点矩阵元的表：

$$\langle \uparrow | S_i^z | \uparrow \rangle = +0.5, \qquad \langle \downarrow | S_i^z | \uparrow \rangle = 0, \qquad \langle \uparrow | S_i^z | \downarrow \rangle = 0, \qquad \langle \downarrow | S_i^z | \downarrow \rangle = -0.5,$$

用比特写就是

$$\langle 1 | S_i^z | 1 \rangle = +0.5, \qquad \langle 0 | S_i^z | 0 \rangle = -0.5, \qquad \langle 1 | S_i^z | 0 \rangle = 0.$$

然后是两条核心公式：

$$\langle \mathrm{col} \,|\, S_i^z S_{i+1}^z \,|\, \mathrm{col} \rangle = \langle S_i^z \rangle \, \langle S_{i+1}^z \rangle,$$

$$\langle S_i^z \rangle = \mathrm{ReadBit}(\mathrm{col}, i) - 0.5.$$

**要点展开**

海森堡模型的哈密顿量（第一讲第 23 页）是

$$H = \sum_i \mathbf{S}_i \cdot \mathbf{S}_{i+1} = \sum_i \Big[ \underbrace{S_i^z S_{i+1}^z}_{\text{对角}} + \underbrace{\tfrac{1}{2}(S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+)}_{\text{非对角}} \Big].$$

**对角部分**之所以"对角"，是因为 $S_i^z$ 是**对角算符**：它在每个基矢上的作用就是乘一个数 $\pm\frac{1}{2}$。因此

$$\langle \mathrm{col} | S_i^z S_{i+1}^z | \mathrm{col} \rangle = \big(\mathrm{ReadBit}(\mathrm{col}, i) - \tfrac{1}{2}\big)\big(\mathrm{ReadBit}(\mathrm{col}, i+1) - \tfrac{1}{2}\big) \in \Big\{ +\tfrac{1}{4},\ -\tfrac{1}{4} \Big\}.$$

- 两个自旋**平行**（↑↑ 或 ↓↓）：$+\frac{1}{4}$；
- 两个自旋**反平行**：$-\frac{1}{4}$。

对反铁磁（$J > 0$），这正好意味着**反平行的键能量更低**——这就是反铁磁性的微观起源。把 $N$ 条键加起来，对角元是

$$H_{\mathrm{col,col}} = \sum_{i=0}^{N-1} \big(\mathrm{ReadBit}(\mathrm{col}, i) - \tfrac{1}{2}\big)\big(\mathrm{ReadBit}(\mathrm{col}, i+1) - \tfrac{1}{2}\big),$$

环上取 $i+1 \to (i+1) \bmod N$（幻灯片里写的 $S_N = S_0$ 就是周期性边界）。

**为什么这个式子这么值钱？**因为它说明**对角元不需要任何"矩阵"**：对每个基矢（一个整数），做 $N$ 次 `ReadBit` 就得到对角元。内存是 $O(1)$（只存一个数），时间 $O(N)$。这是 ED 高效的核心原因之一。

**提示 / 易错**

- 幻灯片第 15 页的代码里，对角元是**在遍历键的循环里累加**的：每个键给 `H[i0,i0]` 加一项。因为 `coo_matrix` 会把重复的 `(row,col)` **自动求和**，这样写完全正确，而且很优雅。
- **`ReadBit` 忘了归一化**是最典型的 bug：$\langle S_i^z \rangle$ 会变成 $-0.5$ 或 $2^i - 0.5$，能量错得离谱但代码不报错。**务必用"基态能量是否等于已知值"来验证。**
- 环上最后一条键是 $(N-1, 0)$，**不要漏**（第 15 页的示例代码用的是开链，正好少了这一条）。
- 注意幻灯片上写 $\langle \downarrow | S_i^z | \uparrow \rangle = 0$：这是对的，$S^z$ 不改变自旋方向。

---

## 第 13 页 · 自旋系统：非对角元

# Off-diagonal terms

- matrix elements $\left\langle \text{row} \left| \left(S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+\right)/2 \right| \text{col} \right\rangle$

$$ {}_{i+1}\langle\downarrow|_i\langle\uparrow|S_i^+S_{i+1}^-|\downarrow\rangle_i|\uparrow\rangle_{i+1} = 1, \qquad {}_{i+1}\langle\downarrow|_i\langle\uparrow|S_i^-S_{i+1}^+|\downarrow\rangle_i|\uparrow\rangle_{i+1} = 0, $$

$$ {}_{i+1}\langle\uparrow|_i\langle\downarrow|S_i^+S_{i+1}^-|\uparrow\rangle_i|\downarrow\rangle_{i+1} = 0, \qquad {}_{i+1}\langle\uparrow|_i\langle\downarrow|S_i^-S_{i+1}^+|\uparrow\rangle_i|\downarrow\rangle_{i+1} = 1, $$

- to have non-zero contributions
  the $i$-th spin and the $(i{+}1)$-th spin must be different
  one need to flip the $i$-th spin and the $(i{+}1)$-th spin simultaneously

$$ {}_{i+1}\langle 0|_i\langle 1|\left(S_i^+S_{i+1}^- + S_i^-S_{i+1}^+\right)/2|0\rangle_i|1\rangle_{i+1} = 0.5 $$

$$ {}_{i+1}\langle 1|_i\langle 0|\left(S_i^+S_{i+1}^- + S_i^-S_{i+1}^+\right)/2|1\rangle_i|0\rangle_{i+1} = 0.5 $$

图 13: 幻灯片第 13 页

### 这一页写了什么

标题 *Off-diagonal terms*。

- **matrix elements**：

$$ \left\langle \text{row} \left| \tfrac{1}{2}(S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+) \right| \text{col} \right\rangle $$

- 单格点矩阵元表：

$$ {}_{i+1}\langle\downarrow|_i\langle\uparrow|S_i^+S_{i+1}^-|\uparrow\rangle_i|\downarrow\rangle_{i+1} = 1, \qquad {}_{i+1}\langle\downarrow|_i\langle\uparrow|S_i^-S_{i+1}^+|\uparrow\rangle_i|\downarrow\rangle_{i+1} = 0, $$

$$ {}_{i+1}\langle\uparrow|_i\langle\downarrow|S_i^+S_{i+1}^-|\downarrow\rangle_i|\uparrow\rangle_{i+1} = 0, \qquad {}_{i+1}\langle\uparrow|_i\langle\downarrow|S_i^-S_{i+1}^+|\downarrow\rangle_i|\uparrow\rangle_{i+1} = 1. $$

- **to have non-zero contributions**：
  - 第 $i$ 个自旋与第 $(i+1)$ 个自旋必须**不同**；
  - 需要**同时翻转**第 $i$ 与第 $(i+1)$ 个自旋。

- 用比特写：

$$ {}_{i+1}\langle 0|\,{}_i\langle 1|\,\tfrac{1}{2}(S_i^+S_{i+1}^- + S_i^-S_{i+1}^+)|0\rangle_i|1\rangle_{i+1} = 0.5, $$

$$ {}_{i+1}\langle 1|\,{}_i\langle 0|\,\tfrac{1}{2}(S_i^+S_{i+1}^- + S_i^-S_{i+1}^+)|1\rangle_i|0\rangle_{i+1} = 0.5. $$

**要点展开**

先回顾升降算符的定义（第一讲第 23 页，海森堡模型一节）：

$$ S^+ = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad S^- = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \qquad S^+|\downarrow\rangle = |\uparrow\rangle, \quad S^-|\uparrow\rangle = |\downarrow\rangle. $$

于是 $\frac{1}{2}(S_i^+S_{i+1}^- + S_i^-S_{i+1}^+)$ 的作用是“**交换一对反平行的自旋**”：

$$ |\uparrow\downarrow\rangle \;\longleftrightarrow\; |\downarrow\uparrow\rangle. $$

矩阵元为什么是 1？因为 $S^+$ 把 $\downarrow$ 变成 $\uparrow$ 时系数是 1（自旋-1/2 的特殊性），再乘上公式里的因子 $\frac{1}{2}$，所以**每次自旋交换的矩阵元恰好是** $0.5$。

用比特语言说，这个过程是：

$$ \text{col} = \cdots 1_i\, 0_{i+1} \cdots \xrightarrow{\text{FlipBit}(i)\ \text{和}\ \text{FlipBit}(i+1)} \cdots 0_i\, 1_{i+1} \cdots = \text{row}. $$

**两个判据**（幻灯片上的两条）：

1. **两个自旋必须不同**：如果 $n_i = n_{i+1}$，那么 $S^+S^-$ 和 $S^-S^+$ 都作用为零（因为 $S^+|\uparrow\rangle = 0$、$S^-|\downarrow\rangle = 0$）。
2. **必须同时翻转两个比特**：这就是代码里的

   ```
   i1 = FlipBit(i0, Pos0)
   i1 = FlipBit(i1, Pos1)
   ```

   两次 `FlipBit` **不能交换顺序**吗？其实可以——`FlipBit` 作用在不同的位上，互不影响。但如果两个格点编号相同（不可能），就有问题了。

**提示 / 易错**

- **矩阵元是** $0.5$ **不是** $1$。这是 $\frac{1}{2}$ 因子 + 自旋-1/2 的“巧合”共同造成的；对自旋-1 就不成立了（那时 $S^+$ 的矩阵元是 $\sqrt{2}$，见第 20 页的“如何推广”问题）。
- **非对角元只出现在“相邻自旋相反”的基矢之间**，所以矩阵非常稀疏：每个基矢最多有 $N$ 个非对角伙伴（环上）。
- **别把** $S^+S^-$ **写成** $S^+S^+$：前者守恒 $S^z$，后者不守恒。如果你写错了，$S^z_{\text{tot}}$ 会不守恒，U(1) 扇区就不成立了（第 17 页会立刻发现）。
- 幻灯片上的 $\langle\cdots|S_i^+S_{i+1}^-|\cdots\rangle$ 写法容易看漏下标顺序：是“**第** $i$ **个的** $S^+$，**第** $i+1$ **个的** $S^-$”。

## 第 14 页 · 非对角元全表：稀疏矩阵的三元组

# Off-diagonal terms

- matrix elements $\left\langle \text{row} \left| \left( S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+ \right)/2 \right| \text{col} \right\rangle$

| $(\text{col})_{10}$ | $(\text{col})_2$ | $i$ | $i+1$ | $(\text{row})_2$ | $(\text{row})_{10}$ | value |
|---|---|---|---|---|---|---|
| 1 | 0001 | 0 | 1 | 0010 | 2 | 0.5 |
| 1 | 0001 | 3 | 0 | 1000 | 8 | 0.5 |
| 2 | 0010 | 0 | 1 | 0001 | 1 | 0.5 |
| 2 | 0010 | 1 | 2 | 0100 | 4 | 0.5 |
| 3 | 0011 | 1 | 2 | 0101 | 5 | 0.5 |
| 3 | 0011 | 3 | 0 | 1010 | 10 | 0.5 |
| 4 | 0100 | 1 | 2 | 0010 | 2 | 0.5 |
| 4 | 0100 | 2 | 3 | 1000 | 8 | 0.5 |
| 5 | 0101 | 0 | 1 | 0110 | 6 | 0.5 |
| 5 | 0101 | 1 | 2 | 0011 | 3 | 0.5 |
| 5 | 0101 | 2 | 3 | 1001 | 9 | 0.5 |
| 5 | 0101 | 3 | 0 | 1100 | 12 | 0.5 |
| 6 | 0110 | 0 | 1 | 0101 | 5 | 0.5 |
| 6 | 0110 | 2 | 3 | 1010 | 10 | 0.5 |
| 7 | 0111 | 2 | 3 | 1011 | 11 | 0.5 |
| 7 | 0111 | 3 | 0 | 1110 | 14 | 0.5 |

| $(\text{col})_{10}$ | $(\text{col})_2$ | $i$ | $i+1$ | $(\text{row})_2$ | $(\text{row})_{10}$ | value |
|---|---|---|---|---|---|---|
| 8 | 1000 | 2 | 3 | 0100 | 4 | 0.5 |
| 8 | 1000 | 3 | 0 | 0001 | 1 | 0.5 |
| 9 | 1001 | 0 | 1 | 1010 | 10 | 0.5 |
| 9 | 1001 | 2 | 3 | 0101 | 5 | 0.5 |
| 10 | 1010 | 0 | 1 | 1001 | 9 | 0.5 |
| 10 | 1010 | 1 | 2 | 1100 | 12 | 0.5 |
| 10 | 1010 | 2 | 3 | 0110 | 6 | 0.5 |
| 10 | 1010 | 3 | 0 | 0011 | 3 | 0.5 |
| 11 | 1011 | 1 | 2 | 1101 | 13 | 0.5 |
| 11 | 1011 | 2 | 3 | 0111 | 7 | 0.5 |
| 12 | 1100 | 1 | 2 | 1010 | 10 | 0.5 |
| 12 | 1100 | 3 | 0 | 0101 | 5 | 0.5 |
| 13 | 1101 | 0 | 1 | 1110 | 14 | 0.5 |
| 13 | 1101 | 1 | 2 | 1011 | 11 | 0.5 |
| 14 | 1110 | 0 | 1 | 1101 | 13 | 0.5 |
| 14 | 1110 | 3 | 0 | 0111 | 7 | 0.5 |

- sparse matrix, only keep: row, column, value

图 14: 幻灯片第 14 页

### 这一页写了什么

标题 *Off-diagonal terms*，与上一页相同的矩阵元公式，主体是一张**大表**。表的列头是：

$$(\text{col})_{10} \quad | \quad (\text{col})_2 \quad | \quad i \quad | \quad i+1 \quad | \quad (\text{row})_2 \quad | \quad (\text{row})_{10} \quad | \quad \text{value}$$

最后一行写着：**sparse matrix, only keep: row, column, value**（稀疏矩阵，只保留：行、列、值）。

### 图里画了什么

两张并排的表，各有 16 行，共 **32 行**，`value` 列**全部是 0.5**。这两张表就是 **4 格点环上全部非零的非对角元**。左表覆盖 $(\text{col})_{10} = 1 \sim 7$，右表覆盖 $8 \sim 14$（`0` 与 `15` 没有非对角伙伴，因为全 ↓ 或全 ↑ 无法交换）。红色的 $i$ 与蓝色的 $i+1$ 用来强调“被交换的那一对”。

完整表格（本讲义按幻灯片重排，便于查阅）：

| $(\text{col})_{10}$ | $(\text{col})_2$ | $i$ | $i+1$ | $(\text{row})_2$ | $(\text{row})_{10}$ | value |
|---|---|---|---|---|---|---|
| 1 | 0001 | 0 | 1 | 0010 | 2 | 0.5 |
| 1 | 0001 | 3 | 0 | 1000 | 8 | 0.5 |
| 2 | 0010 | 0 | 1 | 0001 | 1 | 0.5 |

| $(\mathrm{col})_{10}$ | $(\mathrm{col})_2$ | $i$ | $i+1$ | $(\mathrm{row})_2$ | $(\mathrm{row})_{10}$ | value |
|---|---|---|---|---|---|---|
| 2 | 0010 | 1 | 2 | 0100 | 4 | 0.5 |
| 3 | 0011 | 1 | 2 | 0101 | 5 | 0.5 |
| 3 | 0011 | 3 | 0 | 1010 | 10 | 0.5 |
| 4 | 0100 | 1 | 2 | 0010 | 2 | 0.5 |
| 4 | 0100 | 2 | 3 | 1000 | 8 | 0.5 |
| 5 | 0101 | 0 | 1 | 0110 | 6 | 0.5 |
| 5 | 0101 | 1 | 2 | 0011 | 3 | 0.5 |
| 5 | 0101 | 2 | 3 | 1001 | 9 | 0.5 |
| 5 | 0101 | 3 | 0 | 1100 | 12 | 0.5 |
| 6 | 0110 | 0 | 1 | 0101 | 5 | 0.5 |
| 6 | 0110 | 2 | 3 | 1010 | 10 | 0.5 |
| 7 | 0111 | 2 | 3 | 1011 | 11 | 0.5 |
| 7 | 0111 | 3 | 0 | 1110 | 14 | 0.5 |
| 8 | 1000 | 2 | 3 | 0100 | 4 | 0.5 |
| 8 | 1000 | 3 | 0 | 0001 | 1 | 0.5 |
| 9 | 1001 | 0 | 1 | 1010 | 10 | 0.5 |
| 9 | 1001 | 2 | 3 | 0101 | 5 | 0.5 |
| 10 | 1010 | 0 | 1 | 1001 | 9 | 0.5 |
| 10 | 1010 | 1 | 2 | 1100 | 12 | 0.5 |
| 10 | 1010 | 2 | 3 | 0110 | 6 | 0.5 |
| 10 | 1010 | 3 | 0 | 0011 | 3 | 0.5 |
| 11 | 1011 | 1 | 2 | 1101 | 13 | 0.5 |
| 11 | 1011 | 2 | 3 | 0111 | 7 | 0.5 |
| 12 | 1100 | 1 | 2 | 1010 | 10 | 0.5 |
| 12 | 1100 | 3 | 0 | 0101 | 5 | 0.5 |
| 13 | 1101 | 0 | 1 | 1110 | 14 | 0.5 |
| 13 | 1101 | 1 | 2 | 1011 | 11 | 0.5 |
| 14 | 1110 | 0 | 1 | 1101 | 13 | 0.5 |
| 14 | 1110 | 3 | 0 | 0111 | 7 | 0.5 |

**要点展开**

- **这张表就是 `scipy.sparse` 的 COO 格式**：三个等长数组 `row`（$=(\mathrm{row})_{10}$）、`col`（$=(\mathrm{col})_{10}$）、`data`（=value）。ED 的哈密顿量构造本质上就是“**生成这三个数组**”。
- **非零元个数的规律**：环上有 $N$ 条键，每条键上有 $2^N/2$ 个“两个自旋相反”的基矢，所以非对角元总数 $= N \cdot 2^{N-1}$。$N=4$ 时是 $4\times 8 = 32$ ✓，与表格吻合。加上 16 个对角元，总非零元 48 个，而矩阵有 $16\times 16 = 256$ 个元素——**稀疏度约 19%**；$N$ 增大时稀疏度按 $1/2^N$ 指数下降。
- **对角线上的对称性**：如果 `(row, col)` 有一个非零元，那么 `(col, row)` 也一定有一个同样大小的元（$H$ 是厄米的，$S_i^+S_{i+1}^-$ 的厄米共轭是 $S_i^-S_{i+1}^+$）。在表里能直接看到这种“成对出现”：例如第 1 行 `(1→2, i=0)` 与第 3 行 `(2→1, i=0)`。**这是校验代码的一个免费测试**：构造完 $H$ 后检查 `(H - H.T).nnz == 0`（实矩阵）。

- **完整的哈密顿量** = 对角元（第 12 页）+ 这张表的非对角元。对 4 格点环，本讲义实际对角化得到的谱是

$$E \in \{-2\ (\times 1),\ -1\ (\times 3),\ 0\ (\times 7),\ +1\ (\times 5)\},$$

  基态能量 $E_0 = -2J$，即 $E_0/N = -0.5\,J$。**建议你照着这张表亲手构造一次 $16 \times 16$ 矩阵，看能不能复现这个谱。**

**提示 / 易错**

- 注意表中 $(\mathrm{col})_{10} = 5$（`0101`）有**四个**非对角伙伴：4 条键上自旋都相反，所以四对都翻转。而 $(\mathrm{col})_{10} = 1$（`0001`）只有两个。**每个基矢的非对角伙伴个数 = 相邻自旋相反的键数**，可以是 $0 \sim N$。
- 表格里 $i = 3,\ i+1 = 0$ 的行是**周期性边界**的标志（键 $(3, 0)$）。如果做开链，这些行必须删掉——矩阵会变成块结构（第 15 页代码）。
- **value 列全是 0.5 是自旋-1/2 的特例**。换成 XXZ 模型（$\Delta \neq 1$），非对角元仍然是 0.5，但对角元会乘上 $\Delta$；换成 XY 模型则对角元全为 0。

---

## 第 15 页 · 示例代码：一维反铁磁海森堡链

# Sample code

- AF Heisenberg model on a 1D chain with open boundary condition

```
def GetHopList(Ns):
    HopList = []
    for i in range(Ns-1):
        HopList.append([i,i+1])
    return HopList
```

```
HI = []
HJ = []
HV = []

for i0 in range(Nl):
    for ih in range(len(HopList)):
        Pos0 = HopList[ih][0]
        Pos1 = HopList[ih][1]
```

$i \to$ Pos0
$i+1 \to$ Pos1
col $\to$ i0
row $\to$ i1

```
        if ReadBit(i0,Pos0) != ReadBit(i0,Pos1):
            i1 = FlipBit(i0,Pos0)
            i1 = FlipBit(i1,Pos1)
            HI.append(i1)
            HJ.append(i0)
            HV.append(0.5)
```

off-diagonal terms

```
        HI.append(i0)
        HJ.append(i0)
        HV.append((ReadBit(i0,Pos0)-0.5)*(ReadBit(i0,Pos1)-0.5))
```

diagonal terms

```
Hamr = sparse.coo_matrix((HV,(HI,HJ)),shape=(Nl,Nl)).tocsc()
```

图 15: 幻灯片第 15 页

### 这一页写了什么

标题 *Sample code*，副标题：

- *AF Heisenberg model on a **1D chain with open boundary condition*** （一维**开边界**反铁磁海森堡链）

### 图里画了什么

左边一个虚线框标注变量对应关系：

```
i     → Pos0
i + 1 → Pos1
col   → i0
row   → i1
```

右上角一个点线框给出生成键表的函数：

```
def GetHopList(Ns):
    HopList = []
    for i in range(Ns-1):
        HopList.append([i, i+1])
    return HopList
```

中间是主代码：

```
HI = []
HJ = []
HV = []

for i0 in range(Nl):
    for ih in range(len(HopList)):
        Pos0 = HopList[ih][0]
        Pos1 = HopList[ih][1]
        if ReadBit(i0,Pos0) != ReadBit(i0,Pos1):
            i1 = FlipBit(i0,Pos0)
            i1 = FlipBit(i1,Pos1)
            HI.append(i1)
            HJ.append(i0)
            HV.append(0.5)
        HI.append(i0)
        HJ.append(i0)
        HV.append((ReadBit(i0,Pos0)-0.5)*(ReadBit(i0,Pos1)-0.5))

Hamr = sparse.coo_matrix((HV,(HI,HJ)),shape=(Nl,Nl)).tocsc()
```

蓝色标注：`if` 那一段是 **off-diagonal terms**，最后三行是 **diagonal terms**。

### 要点展开

**这段 20 行代码就是本讲的中心。**逐块拆开：

1. **基矢编号与总数**：`Nl = 2**Ns`，`i0` 遍历所有基矢（col），`i1` 是伙伴基矢（row）。
2. **键表**：`GetHopList(Ns)` 返回 `[[0,1],[1,2],...,[Ns-2,Ns-1]]`——**开链**，共 $Ns-1$ 条键。要做环，只需在列表末尾加 `[Ns-1, 0]`。
3. **非对角块**：若第 `Pos0`、`Pos1` 位不同，就同时翻转它们得到 `i1`，然后追加一条 `(row=i1, col=i0, value=0.5)`。
4. **对角块**：无论是否翻转，都对每个键累加 $(\mathrm{bit}_i-\frac{1}{2})(\mathrm{bit}_{i+1}-\frac{1}{2})$ 到 `(i0, i0)`。**因为循环遍历所有键，重复的 `(i0,i0)` 会被 COO 自动求和**——这正是第 12 页的求和 $\sum_i$ 的代码实现。
5. **组装**：`sparse.coo_matrix((HV,(HI,HJ)), shape=(Nl,Nl)).tocsc()`。为什么转成 **CSC**？COO 只是“三元组构造格式”，不能做矩阵-矢量乘法；转成压缩格式后才能参与运算。**COO 是“构造格式”，CSC/CSR 是“计算格式”。**（补充一点：对 $y=Ax$ 这种矩阵-矢量乘法，其实 **CSR 更自然**（按行取内积）；对 $A^{\mathsf{T}}y$ 或按列操作，CSC 更自然。两者都可以，`eigsh` 内部会自己选。）

**代码的三个小细节值得注意：**

- `ReadBit(i0,Pos0) != ReadBit(i0,Pos1)` 用了 Python 的 `!=` 比较两个整数，安全（不像 `&` 那样有优先级陷阱）。
- `HI.append(i1)` 在前、`HJ.append(i0)` 在后——**行是 `i1`、列是 `i0`**，因为矩阵元是 $\langle \mathrm{row}|H|\mathrm{col}\rangle$。对厄米 $H$ 且矩阵元为实数（本例即是）时，写反了矩阵完全不变（$H^{\mathsf{T}}=H$），所以谱不会错；但如果 $H$ 复厄米（$H^{\mathsf{T}}=H^{*}\neq H$），写反就等于用了 $H^{*}$，**关联函数、时间演化都会错**。养成“先想清楚行列”的习惯。
- `HV.append(0.5)` 硬编码了 $J=1$ 且自旋-1/2。要加 $J$，改成 `HV.append(0.5*J)` 即可。

**一段可以直接跑的完整版本**（本讲义补全，方便你立刻验证）：

```
import numpy as np
from scipy import sparse
from scipy.sparse.linalg import eigsh

def SetBit(i, n):    return i | (1 << n)
def ClearBit(i, n):  return i & ~(1 << n)
def FlipBit(i, n):   return i ^ (1 << n)
def ReadBit(i, n):   return (i & (1 << n)) >> n
def PopCntBit(i):    return bin(i).count("1")

def GetHopList(Ns, periodic=False):
    HopList = [[i, i+1] for i in range(Ns-1)]
    if periodic:
        HopList.append([Ns-1, 0])
    return HopList

def HeisenbergCOO(Ns, J=1.0, periodic=False):
    HopList = GetHopList(Ns, periodic)
    Nl = 2**Ns
    HI, HJ, HV = [], [], []
```

```
    for i0 in range(Nl):
        for (Pos0, Pos1) in HopList:
            if ReadBit(i0, Pos0) != ReadBit(i0, Pos1):
                i1 = FlipBit(FlipBit(i0, Pos0), Pos1)
                HI.append(i1); HJ.append(i0); HV.append(0.5 * J)
            HI.append(i0); HJ.append(i0)
            HV.append(J * (ReadBit(i0, Pos0) - 0.5) * (ReadBit(i0, Pos1) - 0.5))
    return sparse.coo_matrix((HV, (HI, HJ)), shape=(Nl, Nl)).tocsc()

for Ns, per in [(4, True), (4, False), (10, False)]:
    H = HeisenbergCOO(Ns, periodic=per)
    e0 = eigsh(H, k=1, which='SA', return_eigenvectors=False)[0]
    print(Ns, "periodic" if per else "open", "E0 =", round(float(e0), 6))
```

**验证清单**（这是本讲的“考试重点”）：

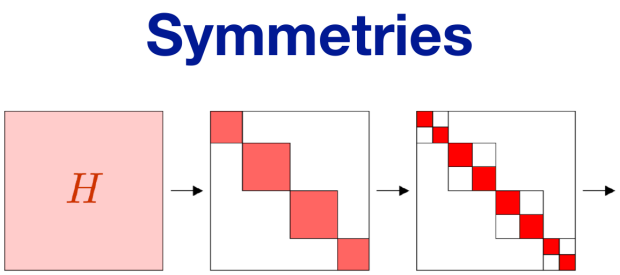
| 检验 | 期望值 |
|---|---|
| $N=4$ 开链 $E_0$ | $\approx -1.616025$ |
| $N=4$ 环 $E_0$ | $-2$（精确，见第 14 页） |
| $N=2$ 单键 $E_0$ | $-0.75$（第一讲第 23 页，海森堡二格点） |
| 迹 | $\mathrm{Tr}\,H=0$（每条键的 $S^zS^z$ 在 $\uparrow\downarrow$ 上正负抵消） |
| 厄米性 | `(H - H.T).nnz == 0` |
| 非零元个数（$N=4$ 环） | $32+16=48$ |

**提示 / 易错**

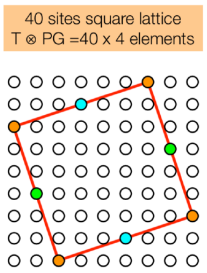
- **本页代码是开链，而第 12–14 页讲的是环**。两者只差一条键，但物理差别不小（开链有边界自旋，环是平移不变的）。**读幻灯片时不要以为前后矛盾**。
- **对角元写在 `if` 外面**：不管两个自旋是否相同，每条键都贡献对角项。如果误把对角项也放进 `if`，就只有“自旋相反”的基矢有对角元，而“自旋相同”的基矢对角元变成 0——这会漏掉 $+\frac{1}{4}$ 的贡献，能量完全错。
- **`Nl = 2**Ns` 不要写成 `2*Ns`**。这是最经典的笔误。
- 对 $N\geq 20$，这段“逐基矢逐键”的 Python 循环会变慢；下一步的优化是（a）只遍历 $S^z$ 扇区（第 17 页），（b）用 numpy 向量化位运算，（c）用 Lin 表做查找（第 19 页）。

# 第三部分：对称性与基矢查找（第 16–20 页）

## 第 16 页 · 对称性：块对角化




- $U(1)$ related symmetries: conservation of particle numbers or total $S_z$
- translation symmetry, momentum space
- parity symmetry, reflection symmetry
- spin-inversion symmetry
- $SU(2)$ symmetry
- various spatial symmetry



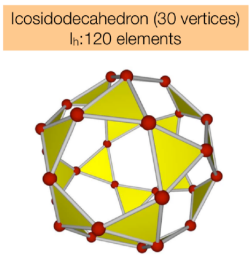




图 16: 幻灯片第 16 页

### 这一页写了什么

标题 *Symmetries*。六条：

- **U(1) related symmetries**：粒子数守恒或总 $S_z$ 守恒
- **translation symmetry, momentum space**（平移对称性，动量空间）
- **parity symmetry, reflection symmetry**（宇称对称性，反射对称性）
- **spin-inversion symmetry**（自旋反演对称性）
- **SU(2) symmetry**
- **various spatial symmetry**（各种空间对称性）

### 图里画了什么

- **顶部**：一张块对角化示意图。从左到右：一个标着 $H$ 的粉色方块 → 分成 4 个对角块 → 再细分成更小、更密的红色块 → 箭头指向更细的划分。意思是：**每用一个对称性，就把矩阵切得更小、更碎。**
- **右下两个例子**：

- 左：“**40 sites square lattice / T ⊗ PG = 40 x 4 elements**”画的是一个正方格子（小圆圈表示格点），上面叠了一个**倾斜的红色方框**，框角与框边上的格点用橙/青/绿点标出——这是一个“**团簇（cluster）**”的示意：在周期边界下，从 40 个格点里选出的、具有特定对称性的格点集合。
- 右：“**Icosidodecahedron (30 vertices) / $I_h$: 120 elements**”一个**二十-十二面体**的线框（红色顶点、灰色棱、黄色面），它的对称群 $I_h$ 有 120 个元素。

**要点展开**

**对称性是 ED 里“最便宜的加速”**——不需要任何近似，只需要一点群论。原理很简单：如果 $H$ 与某个算符 $G$ 对易（$[H,G]=0$），那么可以在 $G$ 的**本征基**里把 $H$ 写成**块对角**形式：

$$H = \bigoplus_{\text{irrep }\alpha} H^{(\alpha)}, \qquad \dim H = \sum_{\alpha} d_\alpha \dim H^{(\alpha)}.$$

**只对角化目标扇区**，就能把维数降低 $d_\alpha$ 倍、时间降低 $d_\alpha^3$ 倍。逐个看：

| 对称性 | 守恒量 / 量子数 | 扇区数 | 典型加速 |
|---|---|---|---|
| U(1)（粒子数 / $S^z_{\text{tot}}$） | $N_\uparrow - N_\downarrow$ | $N+1$ | 最大扇区维数 $\approx 2^N/\sqrt{\pi N/2}$，即加速 $\times\sqrt{\pi N/2}$ |
| 平移 $T$ | 动量 $k = 2\pi m/N$ | $N$ | $\times N$ |
| 点群 $PG$（反射、旋转） | 不可约表示 | $\lvert PG\rvert$ | $\times\lvert PG\rvert$ |
| 自旋反演（↑↔↓） | 宇称 $\pm$ | 2 | $\times 2$ |
| SU(2) | 总自旋 $S$ | $\sim N/2$ | 只对“多重态”加速 |

**幻灯片上两个例子的含义**：

- **40 格点方格子**：平移群 $T$ 有 40 个元素，点群 $PG$ 有 4 个元素，乘积群有 $40\times 4 = 160$ 个元素。于是 $C_{40}^{20}/(40\times 4) \approx 8.6\times 10^8$——**与第 6 页那个“15 亿”在同一量级**（幻灯片的具体数字还取决于用了哪几种对称性）。**但要注意**：只有当你选的**团簇本身**具有这些对称性时，才能用得上。画面上那个倾斜的红方框就是在展示“怎么在格子里挑一个对称性好的团簇”。
- **二十-十二面体（30 顶点）**：这是富勒烯/多面体分子式的磁性模型，它的对称群 $I_h$ 有 120 个元素，是“**分子对称性**”的代表。在这类问题上，对称性带来的加速比平移群更大。

**提示 / 易错**

- **对称性越多越好，但“能用的对称性”取决于几何**。三角格子/笼目格子的团簇往往只能保留部分平移对称性（例如 $N=39$ 的三角格子，不能整除成方格子形状）；**选团簇是 ED 研究的一门手艺**，选错了要么不能用对称性，要么引入严重的有限尺寸效应。
- **U(1) 与 SU(2) 的区别**：U(1) 只固定 $S^z_{\text{tot}}$；SU(2) 还固定 $S_{\text{tot}}$，能把同一多重态里的态合并。用 SU(2) 更省内存，但代码复杂得多（要处理 Clebsch–Gordan 系数），**本讲只要求用 U(1)**（第 34 页评分表里有“没用 U(1) 扣 20 分”）。
- **自旋反演对称性**（↑↔↓，即 $S^z \to -S^z$）只有在零磁场、且扇区 $S^z$ 与 $-S^z$ 对称时才能用。

## 第 17 页 · 实现 U(1) 对称性

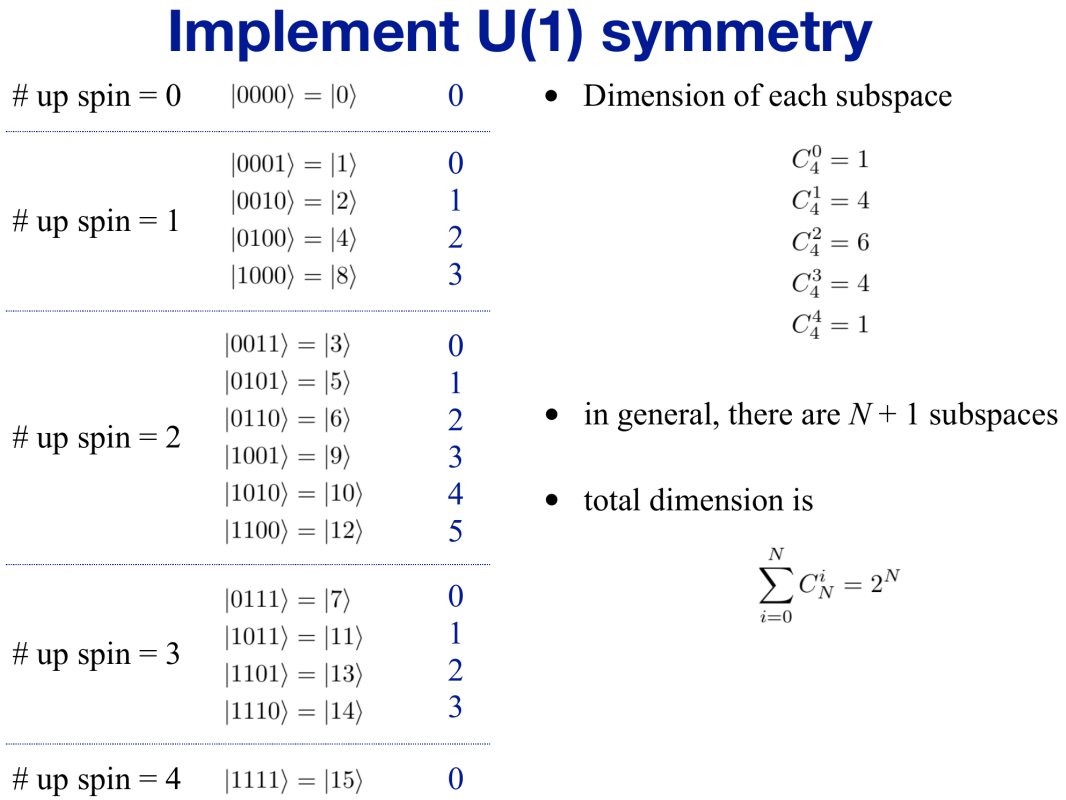



图 17: 幻灯片第 17 页

### 这一页写了什么

标题 *Implement U(1) symmetry*。左半边是一张 16 行的基矢表，按“↑ 自旋的个数”分成 5 组（`# up spin = 0,1,2,3,4`），每组里基矢的**局部下标从 0 重新开始编号**（幻灯片上用蓝色标出）。右半边是两条结论：

- **Dimension of each subspace**（每个子空间的维数）
- **in general, there are $N + 1$ subspaces**（一般有 $N + 1$ 个子空间）
- **total dimension is** $\sum_{i=0}^{N} C_N^i = 2^N$

### 要点展开

**U(1) 对称性 = $S^z_{\text{tot}}$ 守恒**。因为海森堡模型（和 Hubbard 模型）的每一项都不改变自旋向上的格点数，所以 $N_\uparrow$ 是好量子数。于是可以按 $N_\uparrow$ 把基矢分桶：

$$
\begin{aligned}
N_\uparrow = 0: \ |\downarrow\downarrow\downarrow\downarrow\rangle = \{0\}, &\qquad \dim = C_4^0 = 1,\\
N_\uparrow = 1: \ \{1, 2, 4, 8\}, &\qquad \dim = C_4^1 = 4,\\
N_\uparrow = 2: \ \{3, 5, 6, 9, 10, 12\}, &\qquad \dim = C_4^2 = 6,\\
N_\uparrow = 3: \ \{7, 11, 13, 14\}, &\qquad \dim = C_4^3 = 4,
\end{aligned}
$$

$$N_\uparrow = 4 : \{15\}, \qquad \dim = C_4^4 = 1.$$

维数就是**组合数** $C_N^{N_\uparrow}$（从 $N$ 个格点里选 $N_\uparrow$ 个放 ↑），而 $\sum_{i=0}^{N} C_N^i = 2^N$ 正是二项式定理——**这不是巧合，而是“分桶”的必然结果**：所有桶加起来就是原来的 $2^N$ 个基矢。

**“局部下标从 0 重新编号”是这一页的技术核心**：

扇区 `N_up = 2: {3, 5, 6, 9, 10, 12}`
（全局基矢标号）
局部下标：　　0　1　2　3　4　5　← 在扇区内的新编号

这样每个扇区都是一个**独立的、维数小得多的 ED 问题**。对 $N = 4$，最大扇区是 $C_4^2 = 6$，比 $2^4 = 16$ 小；对 $N = 40$，最大扇区是 $C_{40}^{20} \approx 1.38 \times 10^{11}$，比 $2^{40} \approx 1.1 \times 10^{12}$ 小约 8 倍（第 6 页的例子再叠上平移与点群，就到 $10^9$ 量级）。

**加速比**的估算：最大扇区维数

$$C_N^{N/2} \approx \frac{2^N}{\sqrt{\pi N/2}} \implies \text{加速比} \approx \sqrt{\pi N/2} \quad (N = 40 : \approx 7.9).$$

**实现要点**：

1. 生成全部 $2^N$ 个基矢，按 `PopCntBit(i)` 分桶；
2. 每个桶内部**按整数升序**排序（这样局部下标就是“桶内秩”）；
3. 建一个**查找表**：给定全局整数 $i$，返回它在所属扇区里的局部下标。第 18–20 页的 Lin 表就是为第 3 步设计的。

**提示 / 易错**

- **“↑ 的个数”与 $S^z_{\mathbf{tot}}$ 的关系**：$S^z_{\mathrm{tot}} = N_\uparrow - N/2$。所以 $N_\uparrow$ 扇区也常被叫做“$S^z$ 扇区”，编号方式有两种（$N_\uparrow$ 或 $S^z$），**看代码里怎么定义**。
- **每个扇区的维数 $C_N^{N_\uparrow}$ 是“基矢个数”，不是“矩阵大小”**：矩阵大小是它的平方（若非稀疏）。
- 幻灯片上每组的**蓝色数字是局部下标**，不是基矢的整数编号。**看这张表时别搞混。**
- 只有 $S^z$ 守恒的模型才能这么分桶。加**磁场** $h\sum_i S_i^z$ 仍然可以（磁场只改变对角元，不改变扇区）；加**横场**（$h\sum_i S_i^x$）就**不行**了，因为 $S^x$ 不守恒 $S^z$（第一讲第 25 页的横场 Ising 模型就是这种情况）。

### 第 18 页 · 基矢查找：两次直接内存读取

# Basis lookup procedures

- how to save time and memory?
- lookup can be done with two direct memory reads
  this is a time and memory efficient approach

PHYSICAL REVIEW B VOLUME 42, NUMBER 10 1 OCTOBER 1990

**Exact diagonalization of quantum-spin models**

H. Q. Lin
*Center for Nonlinear Studies, Los Alamos National Laboratory, Los Alamos, New Mexico 87545*
(Received 12 April 1990)

We have developed a technique to replace hashing in implementing the Lanczös method for exact diagonalization of quantum-spin models that enables us to carry out numerical studies on substantially larger lattices than previously studied. We describe the algorithm in detail and present results for the ground-state energy, the first-excited-state energy, and the spin-spin correlations on various finite lattices for spins $S=\frac{1}{2}$, 1, $\frac{3}{2}$, and 2. Results for an infinite system are obtained by extrapolation. We also discuss the generalization of our method to other models.

42 6561 ©1990 The American Physical Society

图 18: 幻灯片第 18 页

#### 这一页写了什么

标题 *Basis lookup procedures*。

- **how to save time and memory?**（怎么省时间和内存?）
- **lookup can be done with two direct memory reads** ——查找可以用**两次直接内存读取**完成。*this is a time and memory efficient approach*（这是一种既省时又省内存的做法）。

#### 图里画了什么

右侧贴了一张**论文截图**：H. Q. Lin, *Exact diagonalization of quantum-spin models*, Phys. Rev. B **42**, 6561 (1990) 的摘要页。这篇论文提出的“Lin 表”就是下面两页的主题。

#### 要点展开

**问题是什么?** 在 ED 里你会**反复**问同一个问题：“给定基矢的整数 $i$，它在当前扇区里排第几号（局部下标）?”例如非对角元 `(row, col)` 里的 `row` 是一个**整数**，但你构造稀疏矩阵时需要的是它在**扇区内的行号**。

**朴素做法**是开一个长度 $2^N$ 的查找数组：

```
index = -np.ones(2**N, dtype=int)   # -1 表示不在这个扇区
```

```
for k, i in enumerate(basis):
    index[i] = k
```

对 $N \le 24$ 这没问题（$2^{24} = 1.6 \times 10^7$ 个整数，约 128 MB）；但 $N = 40$ 时 $2^{40}$ 个整数要 8 TB，**内存就爆了**。

**Lin 表的想法**：既然基矢本身是 $N$ 个比特，把它**切成两半**（各 $N/2$ 位），分别记为配置 A 和配置 B。那么"查找"就变成

$$\text{局部下标} = J_a(I_a) + J_b(I_b),$$

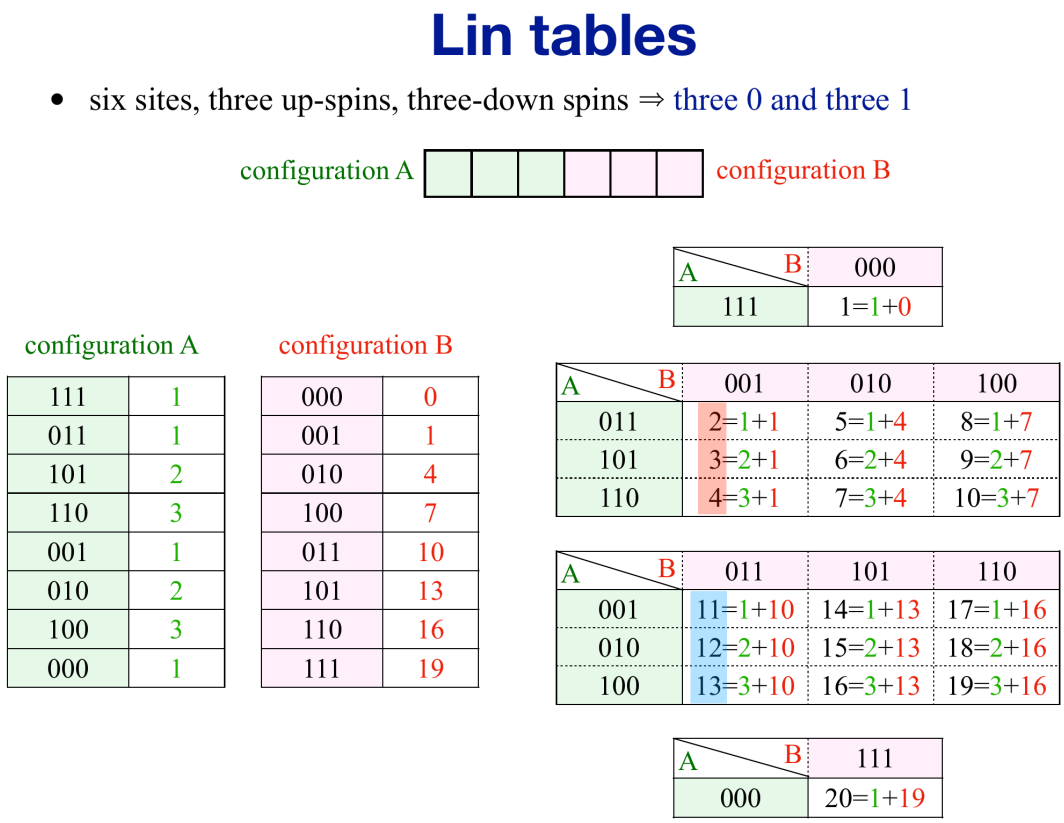
其中 $J_a$、$J_b$ 是两张**小表**（各约 $2^{N/2}$ 项）。**两次内存读取就得到答案**，而且表的大小是 $O(2^{N/2})$ 而不是 $O(2^N)$——对 $N = 40$，$2^{20}$ 项的表格只要几 MB。这就是"**time and memory efficient**"的含义。

**提示 / 易错**

- **Lin 表只适用于"基矢可以分解成前后两段、且两段的贡献可加"的情形**。对 $S^z$ 守恒的扇区，正是如此：$N_\uparrow = N_\uparrow^A + N_\uparrow^B$。所以"每个 B 配置能配哪些 A 配置"是确定的，这使 $J_b$ 可以预先算好。
- **这张表是"只读"的**，可以在对角化前算一次、全程复用。
- 另一种常见做法是**哈希表**（Python 的 `dict`）：实现简单，但每次查找要做哈希、比较，比"两次数组下标"慢很多。小系统用 `dict`，大系统用 Lin 表。

---

## 第 19 页 · Lin 表（一）：A / B 分块




| configuration A | |
|---|---|
| 111 | 1 |
| 011 | 1 |
| 101 | 2 |
| 110 | 3 |
| 001 | 1 |
| 010 | 2 |
| 100 | 3 |
| 000 | 1 |

| configuration B | |
|---|---|
| 000 | 0 |
| 001 | 1 |
| 010 | 4 |
| 100 | 7 |
| 011 | 10 |
| 101 | 13 |
| 110 | 16 |
| 111 | 19 |

图 19: 幻灯片第 19 页

### 这一页写了什么

标题 *Lin tables*，副标题：

- **six sites, three up-spins, three-down spins**（6 个格点，3 个 ↑，3 个 ↓）

左边是两张小表：

| configuration A | $I_a$ | $J_a(I_a)$ | configuration B | $I_b$ | $J_b(I_b)$ |
|---|---|---|---|---|---|
| 111 | 7 | 1 | 000 | 0 | 0 |
| 011 | 3 | 1 | 001 | 1 | 1 |
| 101 | 5 | 2 | 010 | 2 | 4 |
| 110 | 6 | 3 | 100 | 4 | 7 |
| 001 | 1 | 1 | 011 | 3 | 10 |
| 010 | 2 | 2 | 101 | 5 | 13 |
| 100 | 4 | 3 | 110 | 6 | 16 |
| 000 | 0 | 1 | 111 | 7 | 19 |

右边是**四张 Lin 表**（每一张对应一种“$N_\uparrow^A$ 与 $N_\uparrow^B$ 的组合”），格子里的数就是 $J = J_a + J_b$：

```
A     B    000                  A     B    001   010   100
111        1 = 1+0               011        2=1+1  5=1+4  8=1+7
                                 101        3=2+1  6=2+4  9=2+7
                                 110        4=3+1  7=3+4 10=3+7

A     B    011   101   110      A     B    111
001       11=1+10 14=1+13 17=1+16   000        20=1+19
010       12=2+10 15=2+13 18=2+16
100       13=3+10 16=3+13 19=3+16
```

**要点展开**

**设定**：$N = 6$ 个格点，$N_\uparrow = 3$。把格点切成 A（前 3 个）和 B（后 3 个），于是 $N_\uparrow = N_\uparrow^A + N_\uparrow^B$。扇区总维数是 $C_6^3 = 20$。

**两张表的分工**：

- $J_a(I_a)$：**A 段内部的名次**。把 A 的 8 种配置按“↑ 的个数”分组，组内按整数升序排名，名次从 **1** 开始：
  - 3 个 ↑：`111`$\to 1$
  - 2 个 ↑：`011`$\to 1$、`101`$\to 2$、`110`$\to 3$
  - 1 个 ↑：`001`$\to 1$、`010`$\to 2$、`100`$\to 3$
  - 0 个 ↑：`000`$\to 1$
- $J_b(I_b)$：**B 段之前的“累积偏移”** 它等于“在 $I_b$ 之前的所有 B 配置，各自能配多少个 A 配置”的总和：
  - `000`（0 个 ↑）：配 3 个 ↑ 的 A，共 $C_3^3 = 1$ 种 $\to$ 偏移 0；
  - `001`（1 个 ↑）：配 2 个 ↑ 的 A，共 $C_3^2 = 3$ 种 $\to$ 偏移 $0 + 1 = 1$；
  - `010`：偏移 $1 + 3 = 4$；`100`：偏移 $4 + 3 = 7$；
  - `011`（2 个 ↑）：配 1 个 ↑ 的 A，共 $C_3^1 = 3$ 种 $\to$ 偏移 $7 + 3 = 10$；
  - `101`：$10 + 3 = 13$；`110`：$13 + 3 = 16$；
  - `111`（3 个 ↑）：配 0 个 ↑ 的 A，共 $C_3^0 = 1$ 种 $\to$ 偏移 $16 + 3 = 19$。

于是

$$\boxed{J = J_a(I_a) + J_b(I_b)}$$

给出 $1 \sim 20$ 的**全局下标**，与四张表里的数字完全吻合。

**为什么这样能“两次读取”?** 因为 $J_a$ 只依赖 A 段（一张 $2^3 = 8$ 项的表），$J_b$ 只依赖 B 段（另一张 8 项的表）。给定一个 6 位整数，先取低 3 位查 $J_a$、再取高 3 位查 $J_b$、相加即可。**表的总大小是** $2 \times 2^{N/2}$，**而不是** $2^N$。

**提示 / 易错**

- $J_a$ **从 1 开始**，$J_b$ **从 0 开始**——这个“不对称”是故意的：$J_b$ 是“前面的块有多大”（偏移），$J_a$ 是“块内第几个”（名次），两者相加才是从 1 开始的名次。**写代码时最容易在这里差 1。**

- 四张表的**行是 A 配置、列是 B 配置**（或反之），每张表只对应一种 $N_{\uparrow}^{A}$ 的取值。**不要试图用一张表覆盖所有情况。**
- 幻灯片上的 `000 → 1 = 1+0` 里，第一个 1 是 $J_a$、0 是 $J_b$，**这正是“两次读取”的字面展示。**
- 本讲义已用程序验证：按上述规则生成的 20 个 $(I_a, I_b, J_a, J_b, J)$ 与论文 Table II（第 20 页）**完全一致。**

---

## 第 20 页 · Lin 表（二）：论文 Table II 与推广问题

# Lin tables

- six sites, three up-spins, three-down spins ⇒ three 0 and three 1

TABLE II. Spin configurations, their representations $I_a$ and $I_b$, their base vectors $J_a(I_a)$ and position vectors $J_b(I_b)$ and their positions in the storage table $J(I)$ by using the new coding technique for unsymmetrized basis (see the text) for a spin-$\frac{1}{2}$ system of size six.

| Configuration $A$ | $I_a$ | $J_a(I_a)$ | Configuration $B$ | $I_b$ | $J_b(I_b)$ | $J=J_a+J_b$ |
|---|---|---|---|---|---|---|
| 1 1 1 | 7 | 1 | 0 0 0 | 0 | 0 | 1 |
| 0 1 1 | 3 | 1 | 0 0 1 | 1 | 1 | 2 |
| 1 0 1 | 5 | 2 | 0 0 1 | 1 | 1 | 3 |
| 1 1 0 | 6 | 3 | 0 0 1 | 1 | 1 | 4 |
| 0 1 1 | 3 | 1 | 0 1 0 | 2 | 4 | 5 |
| 1 0 1 | 5 | 2 | 0 1 0 | 2 | 4 | 6 |
| 1 1 0 | 6 | 3 | 0 1 0 | 2 | 4 | 7 |
| 0 1 1 | 3 | 1 | 1 0 0 | 4 | 7 | 8 |
| 1 0 1 | 5 | 2 | 1 0 0 | 4 | 7 | 9 |
| 1 1 0 | 6 | 3 | 1 0 0 | 4 | 7 | 10 |
| 0 0 1 | 1 | 1 | 0 1 1 | 3 | 10 | 11 |
| 0 1 0 | 2 | 2 | 0 1 1 | 3 | 10 | 12 |
| 1 0 0 | 4 | 3 | 0 1 1 | 3 | 10 | 13 |
| 0 0 1 | 1 | 1 | 1 0 1 | 5 | 13 | 14 |
| 0 1 0 | 2 | 2 | 1 0 1 | 5 | 13 | 15 |
| 1 0 0 | 4 | 3 | 1 0 1 | 5 | 13 | 16 |
| 0 0 1 | 1 | 1 | 1 1 0 | 6 | 16 | 17 |
| 0 1 0 | 2 | 2 | 1 1 0 | 6 | 16 | 18 |
| 1 0 0 | 4 | 3 | 1 1 0 | 6 | 16 | 19 |
| 0 0 0 | 0 | 1 | 1 1 1 | 7 | 19 | 20 |

question: how about higher spins? bosons?

图 20: 幻灯片第 20 页

### 这一页写了什么

标题 *Lin tables*，副标题与上页相同（6 格点、3 上 3 下）。主体是 Lin 论文里的 **TABLE II**：一张 20 行的表，列头是

$$\text{Configuration A} \mid I_a \mid J_a(I_a) \quad \| \quad \text{Configuration B} \mid I_b \mid J_b(I_b) \quad \| \quad J = J_a + J_b.$$

底部一行提问：**question: how about higher spins? bosons?**（高自旋呢？玻色子呢？）

### 图里画了什么

论文 Table II 的扫描图，20 行。其中 $J = 2, 3, 4$ **三行被标成粉色**，$J = 11, 12, 13$ **三行被标成蓝色**——这两组正好是“同一个 B 配置配三个 A 配置”的例子（粉色对应 $I_b = 001$，蓝色对应 $I_b = 100$），用来强调**块结构**。

**要点展开**

把 20 行全部列出来（本讲义按上页规则复算，与论文一致）：

| $J$ | A | $I_a$ | $J_a$ | B | $I_b$ | $J_b$ |
|---|---|---|---|---|---|---|
| 1 | 111 | 7 | 1 | 000 | 0 | 0 |
| 2 | 011 | 3 | 1 | 001 | 1 | 1 |
| 3 | 101 | 5 | 2 | 001 | 1 | 1 |
| 4 | 110 | 6 | 3 | 001 | 1 | 1 |
| 5 | 011 | 3 | 1 | 010 | 2 | 4 |
| 6 | 101 | 5 | 2 | 010 | 2 | 4 |
| 7 | 110 | 6 | 3 | 010 | 2 | 4 |
| 8 | 001 | 1 | 1 | 011 | 3 | 7 |
| 9 | 010 | 2 | 2 | 011 | 3 | 7 |
| 10 | 100 | 4 | 3 | 011 | 3 | 7 |
| 11 | 011 | 3 | 1 | 100 | 4 | 10 |
| 12 | 101 | 5 | 2 | 100 | 4 | 10 |
| 13 | 110 | 6 | 3 | 100 | 4 | 10 |
| 14 | 001 | 1 | 1 | 101 | 5 | 13 |
| 15 | 010 | 2 | 2 | 101 | 5 | 13 |
| 16 | 100 | 4 | 3 | 101 | 5 | 13 |
| 17 | 001 | 1 | 1 | 110 | 6 | 16 |
| 18 | 010 | 2 | 2 | 110 | 6 | 16 |
| 19 | 100 | 4 | 3 | 110 | 6 | 16 |
| 20 | 000 | 0 | 1 | 111 | 7 | 19 |

**结构一目了然**：表的排序是**外层按 $I_b$ 升序、内层按 $I_a$ 升序**；每个 $I_b$ 块的大小 $= C_3^{3-N_\uparrow^B}$（能配上它的 A 配置数）。粉色三行是 $I_b = 001$ 块，蓝色三行是 $I_b = 100$ 块。

**这张表怎么用？** 给定一个 6 位基矢整数 $i$：

```
Ia = PickBit(i, 0, 3)    # 低 3 位 = 配置 A
Ib = PickBit(i, 3, 3)    # 高 3 位 = 配置 B
J  = Ja[Ia] + Jb[Ib]     # 两次内存读取 → 扇区内下标（1..20）
```

**底部那个问题**：“高自旋呢？玻色子呢？”这是留给你的思考题，答案是：

- **高自旋（$S = 1, 3/2, ...$）**：每个格点有 $2S + 1$ 个态，不再是一位比特，需要多个比特或“混合进制”表示；$J_a$、$J_b$ 的构造要推广为“按每段的占据数分布分块”。**思路一样，但表要多几维。**
- **玻色子（Holstein 模型的声子）**：每个格点的声子数原则上无上限，必须**截断**到 $n_{\max}$；“粒子数守恒”仍然是 U(1)，所以分块思路仍然适用，但“每段能配多少种 A 配置”要重新算。

**提示 / 易错**

- **Lin 表只解决“查找”这一件事**。它不改变矩阵的稀疏性，也不改变对角化的复杂度；它省的是**每次矩阵-矢量乘法里查表的时间**。在 $10^9$ 维的问题里，这一项是主要开销。
- 论文用的是 **1-based** 下标（$J = 1, \dots, 20$）；写代码时通常要减 1 变成 0-based。**这个 off-by-one 是 Lin 表实现里最常见的 bug**。
- **A/B 的切分点不一定要在正中间**。最优切分要平衡两张表的大小和查找次数；常见做法是让 $2^{N_A} \approx 2^{N_B}$（即 $N_A \approx N_B$）。

---

# 第四部分：费米子、有限温度与磁化（第 21–24 页）

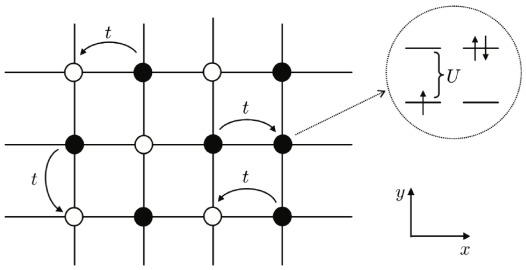
## 第 21 页 · 费米子系统（一）：Hubbard 模型与符号约定

### Fermionic systems

- Hubbard model

$$H = -t \sum_{\langle i,j\rangle,\sigma} C^\dagger_{i,\sigma} C_{j,\sigma} + U \sum_i n_{i\uparrow} n_{i\downarrow},$$

$$n_{i,\sigma} = C^\dagger_{i,\sigma} C_{i,\sigma}$$




- fermionic operators satisfy anti-commutation relations

$$\{C_{i,\sigma}, C^\dagger_{i',\sigma'}\} = \delta_{ii'}\delta_{\sigma\sigma'}, \{C^\dagger_{i,\sigma}, C^\dagger_{i',\sigma'}\} = \{C_{i,\sigma}, C_{i',\sigma'}\} = 0$$

- we fix the convention, all spin-up fermions are put on the left side of spin-down fermions, e.g. $N = 4$,

$$C^\dagger_{0,\uparrow} C^\dagger_{2,\uparrow} C^\dagger_{3,\uparrow} C^\dagger_{1,\downarrow} C^\dagger_{3,\downarrow} |0\rangle$$

spin ↓ 1010 (3rd ⟷ 0th) spin ↑ 1101 (3rd ⟷ 0th) decimal $(10101101)_2 = 173$ (7th ⟷ 0th)

图 21: 幻灯片第 21 页

### 这一页写了什么

标题 *Fermionic systems*。

- **Hubbard model**:

$$H = -t \sum_{\langle i,j\rangle,\sigma} C^\dagger_{i,\sigma} C_{j,\sigma} + U \sum_i n_{i\uparrow} n_{i\downarrow}, \qquad n_{i,\sigma} = C^\dagger_{i,\sigma} C_{i,\sigma}.$$

- **fermionic operators satisfy anti-commutation relations**:

$$\{C_{i,\sigma},\, C^{\dagger}_{i',\sigma'}\} = \delta_{ii'}\delta_{\sigma\sigma'}, \qquad \{C^{\dagger}_{i,\sigma},\, C^{\dagger}_{i',\sigma'}\} = \{C_{i,\sigma},\, C_{i',\sigma'}\} = 0.$$

- **we fix the convention, all spin-up fermions are put on the left side of spin-down fermions, e.g.** $N = 4$:

$$C^{\dagger}_{0,\uparrow} C^{\dagger}_{2,\uparrow} C^{\dagger}_{3,\uparrow} C^{\dagger}_{1,\downarrow} C^{\dagger}_{3,\downarrow} |0\rangle.$$

**图里画了什么**

- **右上角**：一个 $4 \times 3$ 的**正方格子**（12 个格点，空心/实心圆圈表示不同占据），用弯曲箭头标出最近邻跃迁 $t$（跨越上下、左右），虚线引出一个圆形放大图：画的是**在位排斥** $U$——两条能级（单占据 ↑ 与双占据 ↑↓），中间用大括号标 $U$；放大图下方有一个 $x$–$y$ 坐标轴。
- **底部**：一张三列的比特图：

| spin ↓ | spin ↑ | decimal |
|---|---|---|
| `1010` | `1101` | $(10101101)_2 = 173$ |

红色标注：两个 4 位串下面写 **“3rd ←…→ 0th”**，8 位串下面写 **“7th ←…→ 0th”**。

**要点展开**

**Hubbard 模型**（第一讲第 21 页）：$t$ 是跃迁振幅（动能），$U$ 是在位库仑排斥。**注意**：幻灯片这里把跃迁振幅写成 $t$，但在第 23、30 页又用了 $J$ 表示跃迁——**同一个字母在不同页有不同含义，读的时候要看上下文**（这是本讲的一个“命名陷阱”）。

**反对易关系**是费米子与玻色子的分水岭（第一讲第 17 页）：$\{C, C^{\dagger}\} = CC^{\dagger} + C^{\dagger}C = 1$（同格点同自旋），$\{C, C\} = \{C^{\dagger}, C^{\dagger}\} = 0$（自己和自己反对易）。

**为什么要“固定约定”?** 因为反对易关系使得**算符的顺序有物理意义**：交换两个产生算符会多出一个 $-1$。为了在代码里唯一地确定这个符号，必须事先约定**一个标准的算符顺序**。幻灯片选的约定是：

> **所有 ↑ 的产生算符放在 ↓ 的产生算符的左边。**

于是 $N = 4$ 的例子可以写成 $C^{\dagger}_{0\uparrow} C^{\dagger}_{2\uparrow} C^{\dagger}_{3\uparrow} C^{\dagger}_{1\downarrow} C^{\dagger}_{3\downarrow} |0\rangle$：前三个是 ↑（格点 0, 2, 3），后两个是 ↓（格点 1, 3）。

**比特布局**：幻灯片把整个 $2N$ 位整数分成两段——

$$n = n_{\uparrow} + 2^{N}\, n_{\downarrow}, \qquad n_{\sigma} = \sum_{i} \mathrm{occ}_{i,\sigma}\, 2^{i}.$$

即**低 $N$ 位放 ↑、高 $N$ 位放 ↓**。验证一下幻灯片的例子：

- ↑ 在格点 $\{0, 2, 3\}$：$n_{\uparrow} = 2^0 + 2^2 + 2^3 = 1 + 4 + 8 = 13 = 1101_2$ ✓
- ↓ 在格点 $\{1, 3\}$：$n_{\downarrow} = 2^1 + 2^3 = 2 + 8 = 10 = 1010_2$ ✓
- $n = 13 + 16 \times 10 = 13 + 160 = 173$ ✓

**这解释了幻灯片上“↓ 写在左边、↑ 写在右边”的排版**：整数写成二进制时**高位在左**，而高 $N$ 位正是 ↓，所以 ↓ 的比特串排在左边。**这与“算符串里 ↑ 在左”并不矛盾**——一个是“算符顺序”，一个是“整数位布局”。

**提示 / 易错**

- **别把“算符串里 ↑ 在左”和“比特串里 ↓ 在左”搞混**。前者是**符号约定**，后者是**整数编码**。它们恰好相反，是幻灯片刻意展示的。
- **反对易关系里** $\{C_{i\sigma}, C^\dagger_{i'\sigma'}\} = \delta_{ii'}\delta_{\sigma\sigma'}$：不同格点或不同自旋的算符是**反对易**的，不是“互不相干”——这正是第 22 页那些符号因子的来源。
- Hubbard 模型有**两个** U(1) 守恒量：$N_\uparrow$ 与 $N_\downarrow$ 分别守恒（跃迁项 $C^\dagger_{i\sigma}C_{j\sigma}$ 保持 $\sigma$ 不变）。所以扇区数是 $(N+1)^2$，比自旋模型的 $N+1$ 多得多。这也解释了为什么 Hubbard 的 ED 只能做到 20–24 个格点（第 6 页）。

---

## 第 22 页 · 费米子系统（二）：带符号的基矢操作

# Fermionic systems

- useful operations, $N$-site system

$$C^\dagger_{i,\uparrow}\left[\cdots 0_{i,\uparrow}\cdots|0\rangle\right] = (-1)^{\sum_{k=0}^{i-1} n_{k,\uparrow}}\left[\cdots C^\dagger_{i,\uparrow}\cdots|0\rangle\right]$$

$$C_{i,\uparrow}\left[\cdots C^\dagger_{i,\uparrow}\cdots|0\rangle\right] = (-1)^{\sum_{k=0}^{i-1} n_{k,\uparrow}}\left[\cdots 0_{i,\uparrow}\cdots|0\rangle\right]$$

$$C^\dagger_{i,\downarrow}\left[\cdots 0_{i,\downarrow}\cdots|0\rangle\right] = (-1)^{\sum_{k=0}^{i-1} n_{k,\downarrow}+\sum_{k=0}^{N-1} n_{k,\uparrow}}\left[\cdots C^\dagger_{i,\downarrow}\cdots|0\rangle\right]$$

$$C_{i,\downarrow}\left[\cdots C^\dagger_{i,\downarrow}\cdots|0\rangle\right] = (-1)^{\sum_{k=0}^{i-1} n_{k,\downarrow}+\sum_{k=0}^{N-1} n_{k,\uparrow}}\left[\cdots 0_{i,\downarrow}\cdots|0\rangle\right]$$

- $H$ may also introduce an additional sign

$$\left[C^\dagger_{3\downarrow}C_{0\downarrow}\right]\left[C^\dagger_{0\downarrow}C^\dagger_{1\downarrow}|0\rangle\right] = C^\dagger_{3\downarrow}C^\dagger_{1\downarrow}|0\rangle = -C^\dagger_{1\downarrow}C^\dagger_{3\downarrow}|0\rangle$$

图 22: 幻灯片第 22 页

**这一页写了什么**

标题 *Fermionic systems*。

- **useful operations, $N$-site system**（$N$ 格点系统里“有用”的操作）：

$$C^\dagger_{i,\uparrow}[\cdots 0_{i,\uparrow}\cdots|0\rangle] = (-1)^{\sum_{k=0}^{i-1} n_{k,\uparrow}}[\cdots C^\dagger_{i,\uparrow}\cdots|0\rangle],$$

$$C_{i,\uparrow}[\cdots C^\dagger_{i,\uparrow}\cdots|0\rangle] = (-1)^{\sum_{k=0}^{i-1} n_{k,\uparrow}}[\cdots 0_{i,\uparrow}\cdots|0\rangle],$$

$$C^\dagger_{i,\downarrow}[\cdots 0_{i,\downarrow}\cdots|0\rangle] = (-1)^{\sum_{k=0}^{i-1} n_{k,\downarrow}+\sum_{k=0}^{N-1} n_{k,\uparrow}}[\cdots C^\dagger_{i,\downarrow}\cdots|0\rangle],$$

$$C_{i,\downarrow}[\cdots C^\dagger_{i,\downarrow}\cdots|0\rangle] = (-1)^{\sum_{k=0}^{i-1} n_{k,\downarrow}+\sum_{k=0}^{N-1} n_{k,\uparrow}}[\cdots 0_{i,\downarrow}\cdots|0\rangle].$$

  幻灯片用**粉色方框**突出了 $\downarrow$ 情形里多出来的那一项 $\sum_{k=0}^{N-1} n_{k,\uparrow}$。

- **$H$ may also introduce an additional sign**（$H$ 本身也可能带来额外符号）：

$$[C^\dagger_{3\downarrow}C_{0\downarrow}][C^\dagger_{0\downarrow}C^\dagger_{1\downarrow}|0\rangle] = C^\dagger_{3\downarrow}C^\dagger_{1\downarrow}|0\rangle = -C^\dagger_{1\downarrow}C^\dagger_{3\downarrow}|0\rangle.$$

**要点展开**

这是**本讲最容易写错的一页**。核心思想只有一条：

> **在费米子基矢里插入或删除一个算符时，要数一数“它左边有几个占据的算符”，每有一个就乘一个 $-1$。**

这条规则的来源是 Jordan–Wigner 变换（第一讲第 16–17 页）：费米子算符 = 自旋算符 × 一条“符号弦”。

**四条公式逐个看**：

- **$\uparrow$ 的插入/删除**：只数 $\uparrow$ 的占据数，因为按约定所有 $\uparrow$ 都在最左边，插入/删除 $\uparrow$ 时只需要跨过它左边的 $\uparrow$。$C^\dagger_{i\uparrow}$ 插入到第 $i$ 位时，左边有 $\sum_{k<i} n_{k\uparrow}$ 个 $\uparrow$，所以要乘 $(-1)^{\sum_{k<i} n_{k\uparrow}}$。
- **$\downarrow$ 的插入/删除**：除了跨过左边的 $\downarrow$，还要**跨过全部的** $\uparrow$（因为 $\uparrow$ 全部在 $\downarrow$ 的左边），所以要**多乘**一个 $(-1)^{\sum_{k=0}^{N-1} n_{k,\uparrow}}$。**这就是粉色方框强调的那一项。**

用**比特语言**说：

$$\text{符号} = (-1)^{\#\{\text{第}i\text{ 位左边（不含）已占据的同自旋位}\}} \quad (\uparrow),$$

$$\text{符号} = (-1)^{\#\{\text{第}i\text{ 位左边已占据的}\downarrow\} + \#\{\text{全部已占据的}\uparrow\}} \quad (\downarrow).$$

**最后那个例子**是在演示“算符顺序”的后果：

$$[C^\dagger_{3\downarrow}C_{0\downarrow}][C^\dagger_{0\downarrow}C^\dagger_{1\downarrow}|0\rangle] = C^\dagger_{3\downarrow}C^\dagger_{1\downarrow}|0\rangle = -C^\dagger_{1\downarrow}C^\dagger_{3\downarrow}|0\rangle.$$

第一个等号是把 $C_{0\downarrow}$ 作用在 $C_{0\downarrow}^{\dagger}|0\rangle$ 上（抵消，系数 1）；第二个等号是**交换两个产生算符**要出负号——这正是“标准顺序”约定的意义：**只要你把算符重排成标准顺序，符号就自动对了。**

**提示 / 易错**

- **“数左边有几个”里的“左边”是指“格点编号更小”**（在标准顺序下），还是指“比特串里更靠左”？**本讲约定：算符串的顺序是 ↑ 在前、↓ 在后；在同一自旋内按格点编号升序。**因此对 ↑，数的是 $k<i$ 的占据数；对 ↓，数的是 $k<i$ 的 ↓ 占据数，再加上全部 ↑。
- **最容易错的是“忘记 ↓ 的那个额外因子”。**症状是：$H$ 的本征值不对、或者不同扇区之间“看起来”不守恒。建议用一个**已知答案的小例子**验证：比如 2 格点 Hubbard 的 $U=0$ 极限（单粒子能级 $\pm t$）。
- **`PopCntBit` 在这里再次派上用场**：$\sum_{k=0}^{N-1} n_{k,\uparrow}$ 就是“↑ 段整段的 popcount”，用一次 `PopCntBit(n & ((1<<N)-1))` 就得到。
- **符号必须在“每一次矩阵元计算”里都算一遍**，不能只在构造基矢时算一次——因为符号依赖于**具体的算符**（哪个格点、哪个自旋）。

---

## 第 23 页 · 有限温度：期望值、电荷密度与 $\mu$–$n$ 曲线

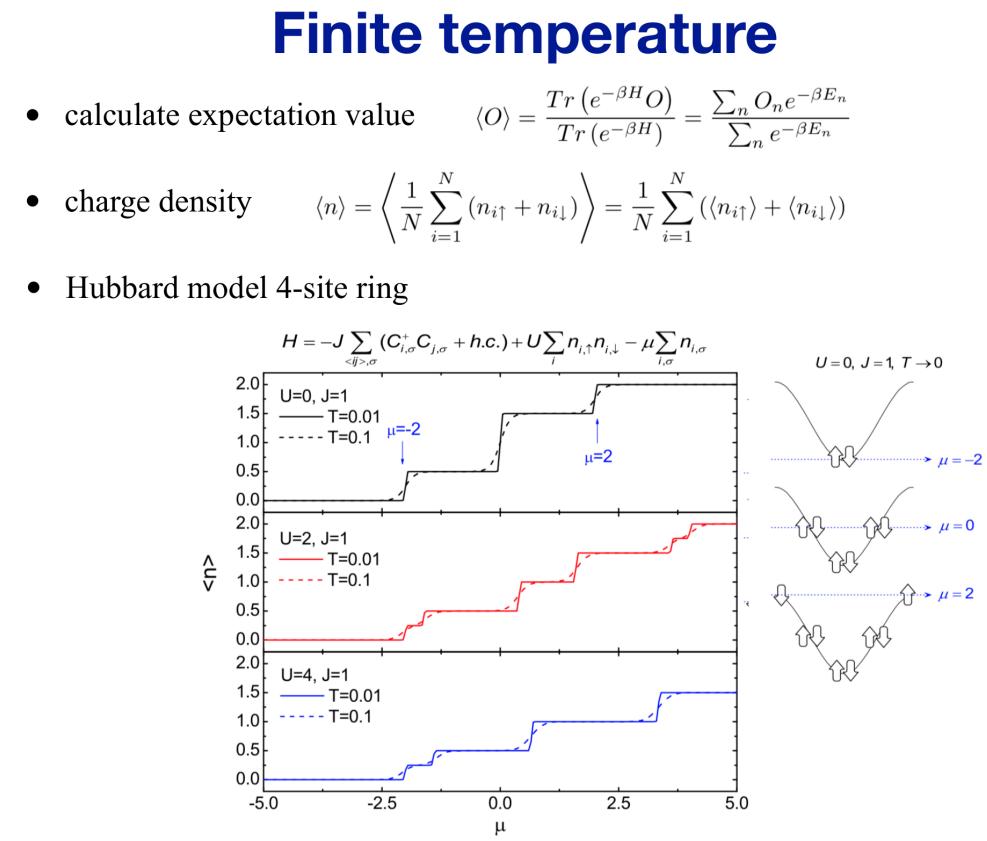



图 23: 幻灯片第 23 页

**这一页写了什么**

标题 *Finite temperature*。

- **calculate expectation value**：

$$\langle O\rangle = \frac{\mathrm{Tr}(e^{-\beta H}O)}{\mathrm{Tr}(e^{-\beta H})} = \frac{\sum_n O_n\, e^{-\beta E_n}}{\sum_n e^{-\beta E_n}}, \qquad O_n = \langle\psi_n|O|\psi_n\rangle.$$

- **charge density**：

$$\langle n\rangle = \Big\langle \frac{1}{N}\sum_{i=1}^{N}(n_{i\uparrow}+n_{i\downarrow})\Big\rangle = \frac{1}{N}\sum_{i=1}^{N}(\langle n_{i\uparrow}\rangle + \langle n_{i\downarrow}\rangle).$$

- **Hubbard model 4-site ring**，哈密顿量：

$$H = -J\sum_{\langle ij\rangle,\sigma}(C^{+}_{i,\sigma}C_{j,\sigma} + \text{h.c.}) + U\sum_i n_{i,\uparrow}n_{i,\downarrow} - \mu\sum_{i,\sigma} n_{i,\sigma}.$$

**图里画了什么**

- **左侧三张堆叠的折线图**（共享横轴 $\mu$，从 $-5$ 到 5；纵轴 $\langle n\rangle$，0 到 2）：
  - **上图（黑色）**：$U=0,\ J=1$。$\langle n\rangle$ 在 $\mu\approx -2$ 处从 0 跳到 0.5，在 $\mu=0$ 附近跳到 1.5，在 $\mu=2$ 处跳到 2——**三级阶梯**。蓝色标注指出 $\mu=-2$ 与 $\mu=2$。
  - **中图（红色）**：$U=2,\ J=1$。阶梯更细更多，出现 $\langle n\rangle\approx 0.5$、1.0、1.5 等平台，到 $\mu\approx 4\sim 5$ 才接近 2。
  - **下图（蓝色）**：$U=4,\ J=1$。台阶被推开，在 $\langle n\rangle = 1.0$ 处有一个**很长的平台**（Mott 平台），到 $\mu\approx 3.5\sim 4$ 才开始上升。
  - 每张图都有两条曲线：实线 $T=0.01$，虚线 $T=0.1$（虚线是实线的“热展宽”版本）。
- **右侧一个能带填充示意图**，标题 $U=0,\ J=1,\ T\to 0$：三条抛物线（能带），每条上有一条水平虚线表示化学势，右侧蓝色标注 $\mu=-2,\ 0,\ 2$；抛物线上的成对 ↑↓ 箭头表示占据的自旋态。

**要点展开**

**有限温度在 ED 里是“白送”的**——因为你已经拿到了**全部本征值与本征矢**（完全对角化时），热平均就是一个加权求和：

$$\langle O\rangle = \frac{\sum_n e^{-\beta E_n}\,\langle\psi_n|O|\psi_n\rangle}{\sum_n e^{-\beta E_n}}.$$

这是 ED 相对于 DMRG/QMC 的一大优势：**DMRG 只能给低能态，QMC 在有限温度要做虚时间演化**，而 ED 只要谱在手，任意温度、任意观测量都是几行代码。

**为什么 $U=0$ 时阶梯出现在 $\mu=-2,0,2$？** 4 格点环的紧束缚能级是

$$\varepsilon_k = -2J\cos\Big(\frac{2\pi k}{4}\Big), \qquad k=0,1,2,3 \implies \varepsilon = -2,\ 0,\ 0,\ +2.$$

每个能级带 2 个自旋自由度，所以：

| $\mu$ 区间 | 填充的电子数 | $\langle n\rangle$ |
|---|---|---|
| $\mu < -2$ | 0 | 0 |
| $-2 < \mu < 0$ | 2（$k = 0$ 能级） | 0.5 |
| $0 < \mu < 2$ | $2 + 4 = 6$（加上简并能级） | 1.5 |
| $\mu > 2$ | 8 | 2 |

**这就是上图的三级阶梯**，跳跃位置正好是能带边 $\pm 2$ 和简并能级 0。右侧的能带示意图画的就是这个图像。

**$U$ 增大后发生了什么？**库仑排斥把能级推开、产生关联能隙：中图（$U = 2$）的台阶变多、变平；下图（$U = 4$）在 $\langle n\rangle = 1$（半满）处出现 **Mott 平台**——这就是第一讲第 22 页讲的 Mott 绝缘体：**半满时体系有电荷能隙，$\mu$ 在能隙里变化时填充不变**。

**“温度”的作用**：虚线（$T = 0.1$）把阶梯“抹圆”了。在 $\beta\Delta E \gg 1$ 时，$T = 0.01$ 的实线几乎就是 $T = 0$ 的阶梯；温度升高后，热激发越过能隙，阶梯变成平滑曲线。**这是区分“能隙”和“无能隙”的一个实验判据。**

**提示 / 易错**

- **这里的 $J$ 是跃迁振幅，不是交换耦合！**第 12–15 页的 $J$ 是 Heisenberg 交换，这里的 $J$ 是 Hubbard 的 hopping（一般文献里写成 $t$）。**同一个符号，两种含义**，看公式里的算符就能区分：有 $C^\dagger C$ 的是跃迁。
- $\langle n\rangle$ **的取值在** $[0, 2]$（每格点最多 2 个电子），不是 $[0, 1]$。如果你算出来超过 2，一定是算符定义错了。
- **完全对角化才“白送”有限温度**。如果只做 Lanczos（只求少数低能态），高温下需要很多态才收敛，这时要改用**有限温度 Lanczos（FTLM）**或**典型态方法（TPQ）**。
- 加化学势 $\mu$ 只改变对角元（$-\mu \sum n$），**不改变扇区结构**，所以可以在每个 $S^z$ 扇区里独立算，然后按 $\mu$ 加权。

## 第 24 页 · 磁化：均匀/交错磁化与磁化率

# Magnetization

- total magnetization of $a$ and $b$ sub-lattices

$$M_{a(b)} = \sum_{i\in a(b)} S_i^z = \sum_{i\in a(b)} \left(n_{i\uparrow} - n_{i\downarrow}\right)/2$$

- uniform magnetization and staggered magnetization

$$m_u = \frac{1}{N}\left(\langle M_a\rangle + \langle M_b\rangle\right) = \frac{1}{2N}\sum_{i=1}^{N}\left(\langle n_{i\uparrow}\rangle - \langle n_{i\downarrow}\rangle\right)$$

$$m_s = \frac{1}{N}\left(\langle M_a\rangle - \langle M_b\rangle\right) = \frac{1}{2N}\left[\sum_{i\in a}\left(\langle n_{i\uparrow}\rangle - \langle n_{i\downarrow}\rangle\right) - \sum_{i\in b}\left(\langle n_{i\uparrow}\rangle - \langle n_{i\downarrow}\rangle\right)\right]$$

- uniform and staggered susceptibility

$$\chi_u = \left.\frac{dm_u(h)}{dh}\right|_{h=0}, \chi_s = \left.\frac{dm_s(h)}{dh}\right|_{h=0}$$

$$\chi_u = \frac{\beta}{N}\left[\left\langle (M_a + M_b)^2\right\rangle - \langle M_a + M_b\rangle^2\right]$$

$$\chi_s = \frac{\beta}{N}\left[\left\langle (M_a - M_b)^2\right\rangle - \langle M_a - M_b\rangle^2\right]$$

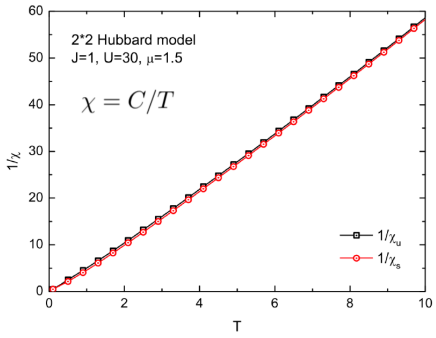



图 24: 幻灯片第 24 页

### 这一页写了什么

标题 *Magnetization*。

- **total magnetization of $a$ and $b$ sub-lattices**（$a$、$b$ 子格的总磁化）：

$$M_{a(b)} = \sum_{i\in a(b)} S_i^z = \sum_{i\in a(b)} (n_{i\uparrow} - n_{i\downarrow})/2.$$

- **uniform magnetization and staggered magnetization**（均匀磁化与交错磁化）：

$$m_u = \frac{1}{N}(\langle M_a\rangle + \langle M_b\rangle) = \frac{1}{2N}\sum_{i=1}^{N}(\langle n_{i\uparrow}\rangle - \langle n_{i\downarrow}\rangle),$$

$$m_s = \frac{1}{N}(\langle M_a\rangle - \langle M_b\rangle) = \frac{1}{2N}\Big[\sum_{i\in a}(\langle n_{i\uparrow}\rangle - \langle n_{i\downarrow}\rangle) - \sum_{i\in b}(\langle n_{i\uparrow}\rangle - \langle n_{i\downarrow}\rangle)\Big].$$

- **uniform and staggered susceptibility**（均匀与交错磁化率）：

$$\chi_u = \frac{dm_u(h)}{dh}\Big|_{h=0}, \qquad \chi_s = \frac{dm_s(h)}{dh}\Big|_{h=0},$$

$$\chi_u = \frac{\beta}{N}\Big[\langle (M_a + M_b)^2\rangle - \langle M_a + M_b\rangle^2\Big],$$

$$\chi_s = \frac{\beta}{N}\Big[\langle (M_a - M_b)^2\rangle - \langle M_a - M_b\rangle^2\Big].$$

**图里画了什么**

右下方一张折线图：

- 横轴 $T$（0 到 10），纵轴 $1/\chi$（0 到 60）。
- 黑色方块 + 实线是 $1/\chi_u$；红色空心圆 + 实线是 $1/\chi_s$。
- 图内文字：“**2*2 Hubbard model**”、“$J = 1,\ U = 30,\ \mu = 1.5$”以及居中的 $\chi = C/T$（居里定律）。
- 两条曲线**几乎重合**，从 $T \approx 0$ 时的 0 附近线性上升到 $T = 10$ 时的约 57。

**要点展开**

**磁化**的物理意义：$M_a$ 是 $a$ 子格（例如方格子的一半）的总自旋。

- **均匀磁化** $m_u$：整个体系的净磁矩——铁磁序的序参量，直接对应外加均匀磁场 $h\sum_i S_i^z$ 的响应。
- **交错磁化** $m_s$：$a$、$b$ 子格的**反号**之和——反铁磁序的序参量，对应**交错磁场** $h\sum_i (-1)^i S_i^z$ 的响应（方格子/二分格子上才有定义）。

**磁化率**：$\chi = dm/dh|_{h=0}$。幻灯片给的是**用涨落表示的公式**（涨落–耗散定理的特例）：

$$\chi = \frac{\beta}{N}\Big[\langle M^2\rangle - \langle M\rangle^2\Big] = \frac{\beta}{N}\,\mathrm{Var}(M).$$

**这个公式极其有用**：它把“对外场的响应”变成了“零场下的涨落”，**不需要真的加磁场**，只要在 $h = 0$ 下算 $M^2$ 的平均值即可。（推导：$Z(h) = \mathrm{Tr}\, e^{-\beta(H - hM)}$，$m = \frac{1}{\beta N}\partial_h \ln Z$，$\chi = \partial_h m|_{h=0} = \frac{\beta}{N}\mathrm{Var}(M)$。）

**图里的物理**：

- 参数是 $2 \times 2$ **Hubbard 模型**（4 个格点的最小方格），$U = 30$ 是**强关联**，$\mu = 1.5$ 使得填充接近半满。强 $U$ + 半满 $\Rightarrow$ 每个格点近似有一个局域磁矩。
- **局域磁矩的标志就是居里定律** $\chi = C/T$，即 $1/\chi \propto T$（直线）。图里两条曲线都是直线，正验证了这一点。
- $1/\chi_u$ **与** $1/\chi_s$ **几乎重合**：说明在这个小簇、这个参数下，均匀与交错磁化率的差别很小（$2 \times 2$ 太小，反铁磁关联还没来得及“长出来”）。在大格子上，$1/\chi_s$ 会外推到**正的截距**（Néel 温度 $T_N$），而 $1/\chi_u$ 外推到**负的截距**（Curie–Weiss 温度 $\Theta_{CW} < 0$）。
- **这就是 ED 算磁化率的套路**：小簇上算 $\langle M^2\rangle$，得到 $1/\chi(T)$，再**外推**（或做有限尺寸标度）到热力学极限。第一讲第 10 页讲的“有限尺度标度”就是干这个的。

**提示 / 易错**

- $1/\chi$ **的直线斜率给出 Curie 常数** $C$，**截距给出 Curie–Weiss 温度**。判断“是否形成长程序”看截距的符号：正截距 = 反铁磁（$\chi_s$），负截距 = 铁磁（$\chi_u$）。
- **公式里的** $\beta = 1/(k_B T)$，所以 $\chi \propto 1/T$ 的居里定律在 $\chi = \beta \mathrm{Var}(M)/N$ 里就是“$\mathrm{Var}(M)$ 不随 $T$ 变化”——这正是“自由局域磁矩”的特征。
- $M_a$、$M_b$ **的定义依赖二分格子**。三角格子、笼目格子不是二分格，“交错磁化”要用更一般的波矢（例如 $\sqrt{3}\times\sqrt{3}$ 序）。
- **注意这里又是** $J = 1$ **表示跃迁**（与第 23 页一致），不是 Heisenberg 的 $J$。

---

# 第五部分：工具链：Python 包（第 25–27 页）

## 第 25 页 · Python 包（一）：完全对角化




图 25: 幻灯片第 25 页

### 这一页写了什么

标题 *Python packages*。整页是一张 **NumPy 官方文档**的截图（`numpy.linalg.eig`，NumPy v1.15，托管在 SciPy.org），右下角用深蓝色加了讲者的批注：

> **complete diagonalization**（完全对角化）

**图里画了什么**

截图内容（逐字）：

> **`numpy.linalg.eig`** ——`numpy.linalg.eig(a)`
>
> Compute the eigenvalues and right eigenvectors of a square array.
>
> **Parameters:** `a : (..., M, M) array` —要计算本征值与本征矢的矩阵
>
> **Returns:** - `w : (..., M) array` —本征值，按重数重复；**不保证有序**；除非虚部为零，否则结果是复数类型。- `v : (..., M, M) array` —归一化（单位长度）的右本征矢，第 `v[:,i]` 列对应本征值 `w[i]`。

右侧边栏显示上一个主题 `numpy.linalg.svd`、下一个主题 `numpy.linalg.eigh`。页面下方有一段代码示例：

```
>>> w, v = LA.eig(np.array([[1, -1], [1, 1]]))
>>> w; v
array([ 1.+1.j,  1.-1.j])
array([[ 0.70710678+0.j        ,  0.70710678+0.j        ],
       [ 0.00000000-0.70710678j,  0.00000000+0.70710678j]])
```

**要点展开**

**完全对角化（complete / full diagonalization）**的含义是：**求出矩阵的全部 $D$ 个本征值与全部本征矢**。

- **算法**：LAPACK 里的 `?syev`/`?heev`（厄米矩阵）或 `?geev`（一般矩阵），典型实现是**三对角化 + QR 迭代**（先 Householder 约化到三对角，再用隐式 QR 求本征值）。
- **代价**：时间 $O(D^3)$，内存 $O(D^2)$。
- **返回值**：全部本征值 + 全部本征矢（一个 $D \times D$ 的矩阵）。

**在 ED 里什么时候用完全对角化?**

| 情形 | 是否用完全对角化 |
|---|---|
| 只需要基态能量 | × 用 `eigsh`（迭代） |
| 需要低能激发谱（前几个态） | × 用 `eigsh(k=...)` |
| 需要**全部**谱（态密度、有限温度、动力学谱） | ✓ |
| 需要**实时演化**（用谱分解） | ✓（但 $D$ 必须很小） |
| $D \lesssim 10^4$ | ✓ 随便用 |
| $D \gtrsim 10^5$ | × 内存/时间都吃不消 |

**例子**：$D = 2^{12} = 4096$ 时，完全对角化要存 $4096^2 \approx 1.7 \times 10^7$ 个复数（约 270 MB，双精度），时间是秒级。$D = 2^{16} = 65536$ 时，本征矢矩阵要 $65536^2 \times 16$ byte $= 68$ GB——**内存就爆了**。所以**“完全”对角化只适用于小系统**，大系统必须用迭代方法（第 26 页）。

**提示 / 易错**

- **`eig` 与 `eigh` 的区别**（第一讲第 28–31 页）：

  - `eig` 处理**一般**方阵，返回**复数**，本征值**不排序**；
  - `eigh` 处理**厄米/实对称**矩阵，返回**实数**（升序），速度快一倍以上、更稳定。**哈密顿量是厄米的，永远优先用 `eigh`！**幻灯片故意展示 `eig`，只是因为它是最通用的接口。
- **本征值不排序**：`eig` 的返回顺序是任意的，要用 `np.argsort` 自己排。`eigh` 返回升序，可以直接取 `w[0]` 当基态能量。
- 幻灯片截图里 `eig` 和 `eigh` 下面的红色波浪线是**浏览器拼写检查**，不是内容。

---

## 第 26 页 · Python 包（二）：迭代对角化

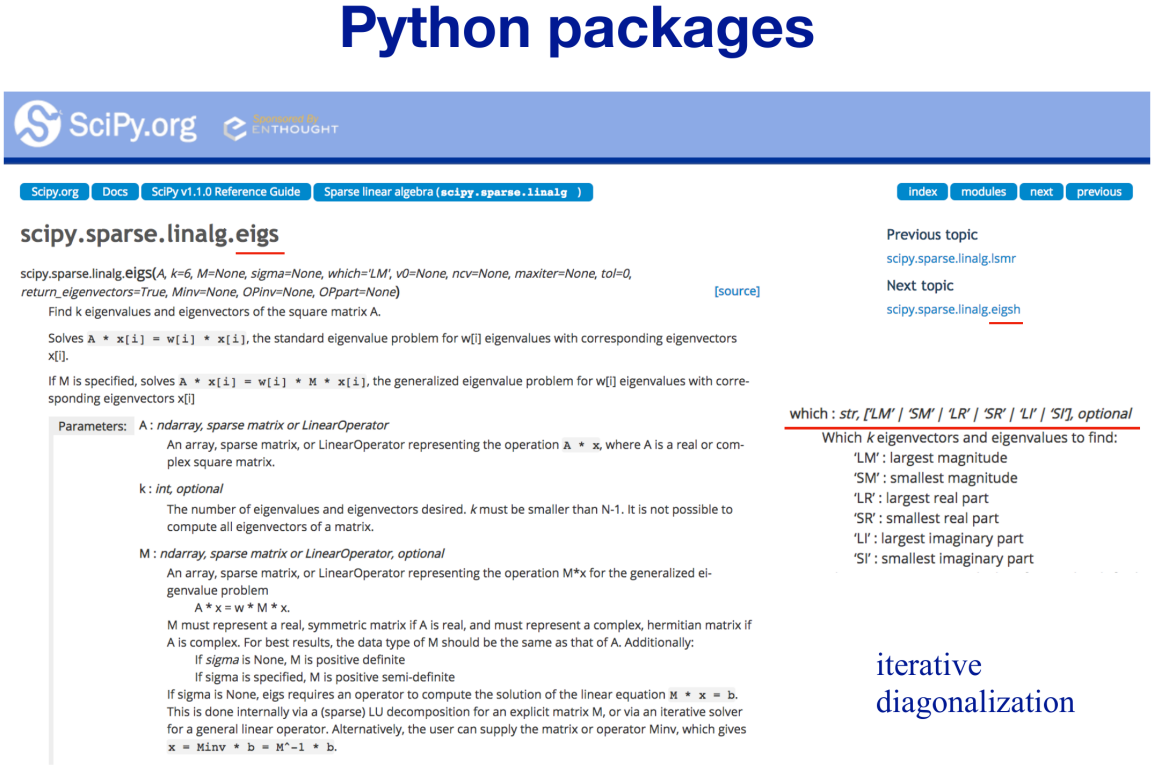



图 26: 幻灯片第 26 页

### 这一页写了什么

标题 *Python packages*。整页是一张 **SciPy 官方文档**的截图（`scipy.sparse.linalg.eigs`，SciPy v1.1.0），右下角讲者批注：

> **iterative diagonalization**（迭代对角化）

### 图里画了什么

截图内容（逐字）：

> **scipy.sparse.linalg.eigs**

`eigs(A, k=6, M=None, sigma=None, which='LM', v0=None, ncv=None, maxiter=None, tol=0, return_eigenvectors=True, Minv=None, OPinv=None, OPpart=None)`

Find k eigenvalues and eigenvectors of the square matrix A. Solves `A * x[i] = w[i] * x[i]` …

**Parameters:** - `A` : ndarray, sparse matrix or LinearOperator —代表 `A * x` 这个操作 - `k` : int —想要的本征值/本征矢个数，**必须小于 N-1**；**不可能求出矩阵的全部本征矢** - `M` : 广义本征问题 $Ax = wMx$ 里的 $M$

右侧边栏：上一个主题 `scipy.sparse.linalg.lsmr`，下一个主题 `scipy.sparse.linalg.eigsh`。

还有一个被红框标出的 `which` 参数说明：`which : str, ['LM' | 'SM' | 'LR' | 'SR' | 'LI' | 'SI']` - `'LM'` : largest magnitude（最大模）- `'SM'` : smallest magnitude（最小模）- `'LR'` : largest real part（最大实部）- `'SR'` : smallest real part（最小实部）- `'LI'` : largest imaginary part（最大虚部）- `'SI'` : smallest imaginary part（最小虚部）

**要点展开**

**迭代对角化**只求**少数几个**本征值，代价远低于完全对角化：

- **算法**：`eigs`/`eigsh` 背后是 **ARPACK**（隐式重启的 Arnoldi / Lanczos 迭代）。核心是**"矩阵-矢量乘法 + 子空间迭代"，不需要矩阵元，只需要 $H$ 对矢量的作用**（这正是第 7 页说的 *needs only operations*）。
- **代价**：只求少数几个极端本征值时，迭代次数通常只有几十到几百，每次矩阵-矢量乘法 $O(\text{nnz}) = O(D)$，所以总时间大致在 $O(D)$ 到 $O(D^2)$ 之间，**远低于完全对角化的 $O(D^3)$**；内存 $O(D \cdot \text{ncv})$。**内存从 $O(D^2)$ 降到 $O(D)$，这是革命性的。**
- **限制**：`k` 必须远小于 $D$；**不可能求出全部本征矢**（文档原文）。

**`which` 参数是 ED 里最关键的开关：**

| 目标 | 写法 |
|---|---|
| 基态能量（代数最小） | `eigsh(H, k=1, which='SA')` （SA = smallest algebraic） |
| 前 $k$ 个低能态 | `eigsh(H, k=20, which='SA')` |
| 最大模本征值（谱半径） | `eigs(H, k=1, which='LM')` |
| 内部本征值（例如某个窗口内） | 用 `sigma` 做**移位反演**（shift-invert） |

**注意**：文档里 `eigs` 的 `which` 列表**没有 `'SA'`/`'LA'`**——那是 `eigsh`（厄米版本）才有的。做 ED 时：**厄米矩阵用 `eigsh`，用 `which='SA'` 找基态；一般矩阵才用 `eigs`。**

**在 ED 里的标准用法：**

```
from scipy.sparse.linalg import eigsh
vals, vecs = eigsh(H, k=6, which='SA')      # H 是 CSC 稀疏矩阵
E0, psi0 = vals[0], vecs[:, 0]
```

**提示 / 易错**

- **`eigsh` 的 `which='SA'` 比 `'SM'` 好**：`'SM'` 对最小模的本征值收敛很慢（甚至可能失败），`'SA'` 专门为厄米矩阵的代数最小值优化。
- **`k` 太大会变慢甚至不收敛**。一般取 `k = 6~20`；如果确实需要很多低能态，可以用 `sigma` 做移位反演（但要做 LU 分解，内存又上去了）。
- **随机初值**：ARPACK 默认用随机初值，所以**每次运行结果会有微小差异**（$10^{-12}$ 量级）。要完全可复现，设 `v0=` 一个固定矢量。
- **`eigs` 返回复数**（即使矩阵是实的）。`eigsh` 返回实数。

---

## 第 27 页 · Python 包（三）：matrix-free 与 `LinearOperator`

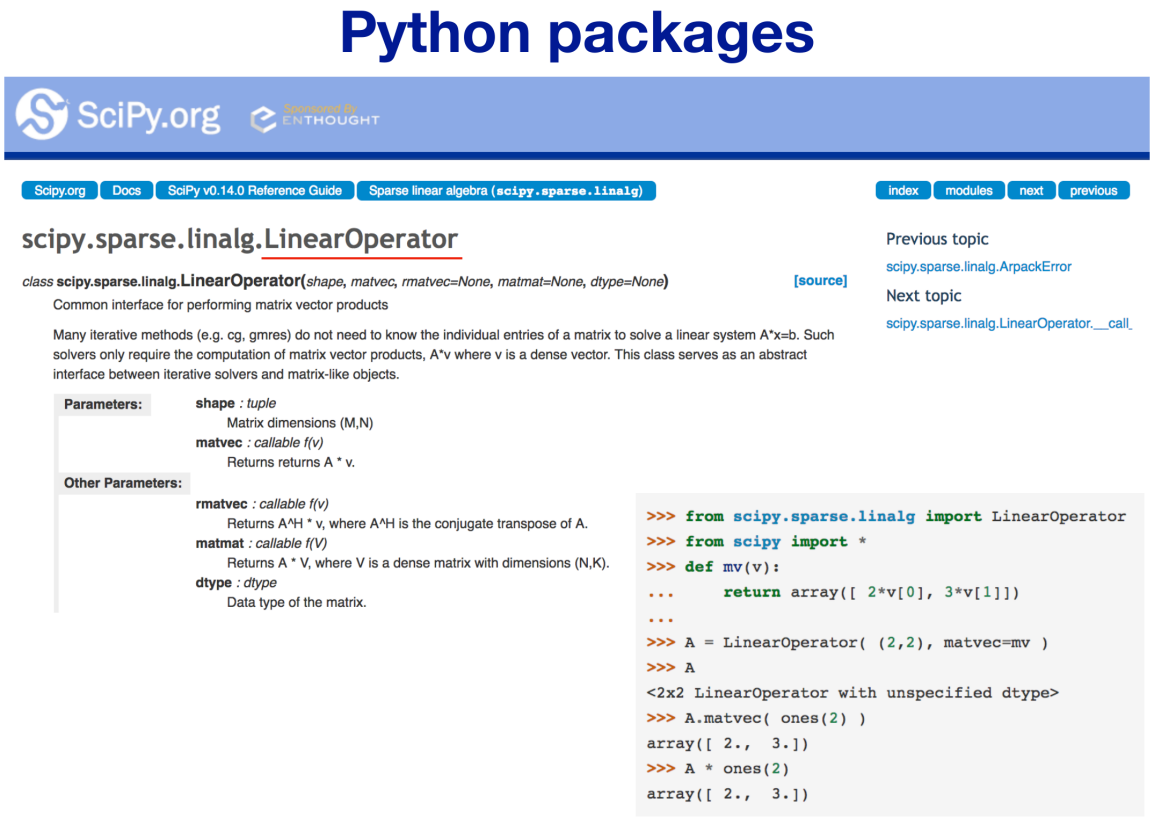



图 27: 幻灯片第 27 页

### 这一页写了什么

标题 *Python packages*。整页是一张 **SciPy 官方文档**的截图（`scipy.sparse.linalg.LinearOperator`，SciPy v0.14.0）。

### 图里画了什么

截图内容（逐字）：

> **`scipy.sparse.linalg.LinearOperator`**

`class scipy.sparse.linalg.LinearOperator(shape, matvec, rmatvec=None, matmat=None, dtype=None)`

**Common interface for performing matrix vector products**（执行矩阵-矢量乘法的通用接口）

Many iterative methods (e.g. cg, gmres) do not need to know the individual entries of a matrix to solve a linear system A$x$=$b$. *Such solvers only require the computation of matrix vector products,* A$v$ where v is a dense vector. This class serves as an abstract interface between iterative solvers and matrix-like objects.

**Parameters:** `shape : tuple`; `matvec : callable f(v)` —Returns A * v

**Other Parameters:** `rmatvec : callable f(v)` —Returns A^H * v; `matmat : callable f(V)` —Returns A * V; `dtype : dtype`

代码示例：

```
>>> from scipy.sparse.linalg import LinearOperator
>>> from scipy import *
>>> def mv(v):
...     return array([ 2*v[0], 3*v[1]])
...
>>> A = LinearOperator( (2,2), matvec=mv )
>>> A
<2x2 LinearOperator with unspecified dtype>
>>> A.matvec( ones(2) )
array([ 2.,  3.])
>>> A * ones(2)
array([ 2.,  3.])
```

**要点展开**

**这一页把第 7 页的 matrix-free 落到了实处。**核心思想：

> **迭代求解器只需要“$H$ 作用在矢量上”这个操作，不需要 $H$ 的元素。**

`LinearOperator` 就是 Python 里表达这个抽象的工具：你只给它一个函数 `matvec(v) -> H@v`，它就能“假装”自己是一个矩阵，喂给 `eigsh`、`cg`、`gmres` 等所有迭代求解器。

**为什么这在 ED 里极其重要?**

- **省内存**：不存 $H$ 的稀疏矩阵（$O(D \cdot N)$ 个非零元），只存几个长度为 $D$ 的矢量（$O(D)$）。对 $D = 10^9$ 这是生死之别。
- **省时间**：可以用**向量化/并行**的方式现场算 $Hv$，甚至可以在 GPU 上算，而不必受稀疏矩阵格式的限制。
- **更灵活**：可以对 $H$ 做**分块作用**（例如按对称性扇区分别处理），或者用**矩阵乘积算符（MPO）**的形式给出 $H$。

**一个 matrix-free 的 Heisenberg 例子**（思路）：

```python
def apply_H(v, Ns, HopList, J=1.0):
    """ 返回 H @ v，不构造矩阵。v 是长度 2**Ns 的复数数组。"""
    out = np.zeros_like(v)
    for i in range(2**Ns):
        # 对角项
        diag = 0.0
        for (p, q) in HopList:
            diag += J * (ReadBit(i,p) - 0.5) * (ReadBit(i,q) - 0.5)
        out[i] += diag * v[i]
        # 非对角项
        for (p, q) in HopList:
            if ReadBit(i,p) != ReadBit(i,q):
                out[FlipBit(FlipBit(i,p),q)] += 0.5 * J * v[i]
    return out

H_op = LinearOperator((Nl, Nl), matvec=lambda v: apply_H(v, Ns, HopList))
E0 = eigsh(H_op, k=1, which='SA', return_eigenvectors=False)[0]
```

**注意**：上面这段是“教学版”，逐基矢的 Python 循环很慢。真正高效的 matrix-free 实现要把整个 `v` 拆成块、用 numpy 位运算**向量化**。

**提示 / 易错**

- **`LinearOperator` 需要你保证“它真的线性、且厄米”**（如果矩阵是厄米的，要设 `rmatvec=matvec` 或传 `dtype`），否则 `eigsh` 会给出**看起来正常但实际错误**的结果。这是 matrix-free 最大的风险：**没有矩阵可以检查厄米性。**
- **`LinearOperator` 与“稀疏矩阵”不是二选一**：小系统直接用 `scipy.sparse` 更省事（可以随时 `.toarray()` 检查），大系统才上 `LinearOperator`。
- 幻灯片截图里的 `eigs/eigh/LinearOperator` 下的红波浪线同样是**拼写检查**。
- **文档版本是 2014 年的 v0.14.0**，但 `LinearOperator` 的接口至今没变。

---

# 第六部分：实时演化与非厄米（第 28–32 页）

## 第 28 页 · 实时演化：形式解与 Lanczos 加速

# Real time evolution

- the time evolution of a quantum system is governed by the time-dependent Schroedinger equation

$$i\hbar\frac{\partial}{\partial t}|\psi(t)\rangle = H|\psi(t)\rangle$$

- for a given initial state $|\psi(0)\rangle$

  using exact diagonalization, we obtain $E_n$ and $|\psi_n\rangle$

  the final state is

$$|\psi(t)\rangle = e^{-iHt/\hbar}|\psi(0)\rangle = \sum_n e^{-iE_nt/\hbar}|\psi_n\rangle\langle\psi_n|\psi(0)\rangle = \sum_n C_n e^{-iE_nt/\hbar}|\psi_n\rangle$$

  where $C_n = \langle\psi_n|\psi(0)\rangle$

- using the Lanczos vectors, only a very small Krylov space is needed

  $n \leq 20$ is sufficient

  time evolution can be computed efficiently

图 28: 幻灯片第 28 页

### 这一页写了什么

标题 *Real time evolution*。

- **the time evolution of a quantum system is governed by the time-dependent Schroedinger equation**（量子系统的演化由含时薛定谔方程支配）：

$$\mathrm{i}\hbar\frac{\partial}{\partial t}|\psi(t)\rangle = H|\psi(t)\rangle.$$

- **for a given initial state $|\psi(0)\rangle$: using exact diagonalization, we obtain $E_n$ and $|\psi_n\rangle$; the final state is**（“final state”用蓝色标出）：

$$|\psi(t)\rangle = e^{-\mathrm{i}Ht/\hbar}|\psi(0)\rangle = \sum_n e^{-\mathrm{i}E_nt/\hbar}|\psi_n\rangle\langle\psi_n|\psi(0)\rangle = \sum_n C_n\, e^{-\mathrm{i}E_nt/\hbar}|\psi_n\rangle,$$

  **where** $C_n = \langle\psi_n|\psi(0)\rangle$。

- **using the Lanczos vectors, only a very small Krylov space is needed**（用 Lanczos 矢量，只需要很小的 Krylov 空间）
  - $n \leq 20$ **is sufficient**（$n \leq 20$ 就够）
  - **time evolution can be computed efficiently**（时间演化可以高效计算）

**要点展开**

**形式解**是第一步：把含时薛定谔方程"积分"一次（$H$ 不显含时间时），

$$|\psi(t)\rangle = e^{-\mathrm{i}Ht/\hbar}|\psi(0)\rangle.$$

**关键观察**：指数算符 $e^{-\mathrm{i}Ht/\hbar}$ 在**能量本征基**里是对角的！把初态按本征态展开 $|\psi(0)\rangle = \sum_n C_n|\psi_n\rangle$，每个分量只是**乘上一个相位** $e^{-\mathrm{i}E_n t/\hbar}$：

$$\boxed{|\psi(t)\rangle = \sum_n C_n\, e^{-\mathrm{i}E_n t/\hbar}|\psi_n\rangle}$$

**这就是"ED 做实时演化"的全部原理**：一旦有谱，演化就是"给每个分量转相位"。（这正对应第一讲第 15 页"诚实版本"里的 `expm`；那里的 `expm(-1j*H*t) @ psi0` 在数学上就是这里的求和。）

**但为什么还要 Lanczos?** 因为对 $D = 10^9$ 的系统，既不能完全对角化，也不能直接把 $e^{-\mathrm{i}Ht}$ 的矩阵作用上去。这时用 **Krylov 方法**：

1. 以 $|\psi(0)\rangle$ 为起点，构造 **Krylov 空间** $\mathcal{K}_m = \mathrm{span}\{|\psi_0\rangle, H|\psi_0\rangle, H^2|\psi_0\rangle, \ldots, H^{m-1}|\psi_0\rangle\}$；
2. 在这个 $m$ 维小空间里把 $H$ **三对角化**（Lanczos），得到一个小矩阵 $T_m$；
3. 在小空间里做 $e^{-\mathrm{i}T_m t}$（$m \times m$ 矩阵，$m \leq 20$ 就够）；
4. 投影回去得到 $|\psi(t)\rangle$。

**为什么 $m \leq 20$ 就够?** 因为**短时间演化只需要低能谱的信息**，而 Krylov 空间天然地"捕获"了与初态耦合最强的那些本征分量。时间越长需要的 $m$ 越大；但即使如此，$m \sim 10^2$ 也远小于 $D$。

**提示 / 易错**

- 幻灯片把 Schrödinger 拼成 **Schroedinger**（用 `oe` 代替 `ö`），这是排版惯例，不是笔误。
- $e^{-\mathrm{i}Ht/\hbar}$ **是幺正的**（$H$ 厄米时），所以**总概率守恒**：$\||\psi(t)\rangle\| = 1$。如果你的数值结果范数漂移，说明算法不稳。
- **注意** $C_n = \langle\psi_n|\psi(0)\rangle$ **的共轭顺序**：是"本征态在左、初态在右"。写反了相当于算 $\langle\psi(0)|\psi_n\rangle = C_n^*$，相位方向反了。
- **时间步长**：用谱分解时 $t$ 可以任意大（相位是精确的）；用时间步进（如 Crank–Nicolson、Runge–Kutta）时，步长受 $\Delta t \lesssim 1/\|H\|$ 限制。

---

## 第 29 页 · 实时演化实例（一）：高斯波包与色散

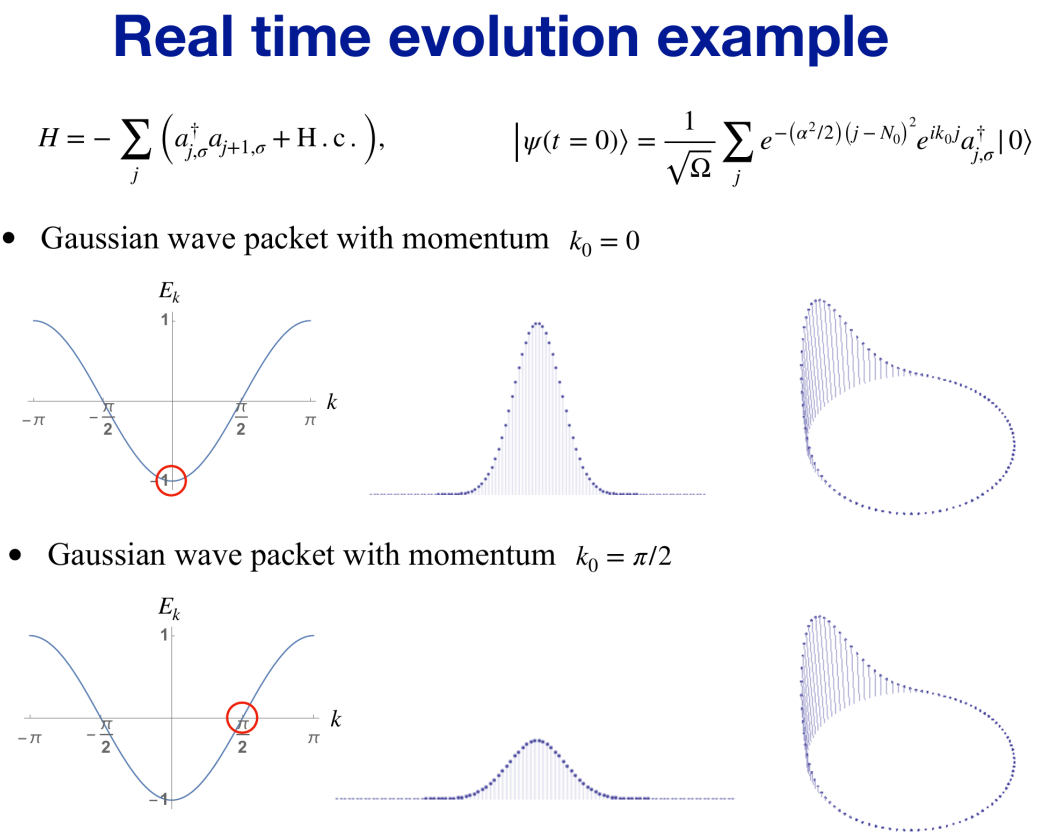



图 29: 幻灯片第 29 页

### 这一页写了什么

标题 *Real time evolution example*，两个公式并排：

$$H = -\sum_j (a^\dagger_{j,\sigma} a_{j+1,\sigma} + \mathrm{H.c.}),$$

$$|\psi(t=0)\rangle = \frac{1}{\sqrt{\Omega}} \sum_j e^{-(\alpha^2/2)(j-N_0)^2}\, e^{\mathrm{i}k_0 j}\, a^\dagger_{j,\sigma}|0\rangle.$$

两组内容：

- **Gaussian wave packet with momentum** $k_0 = 0$（动量为 0 的高斯波包）
- **Gaussian wave packet with momentum** $k_0 = \pi/2$

### 图里画了什么

两组各三张图：

- **左：色散关系** $E_k$–$k$。横轴从 $-\pi$ 到 $\pi$（标了 $-\pi, -\pi/2, \pi/2, \pi$），纵轴标 1 和 $-1$。曲线是**反余弦型**：在 $k = 0$ 取极小、在 $k = \pm\pi$ 取极大。

  - 第一组：**红色圆圈**标在 $(k, E_k) = (0, -1)$——**带底**。
  - 第二组：**红色圆圈**标在 $(k, E_k) = (\pi/2, 0)$——**拐点**。
- **中：动量空间的“棒棒糖”图**（无坐标轴，只有一条虚线基线和一排点）。
  - 第一组：一个又高又窄的高斯峰，中心在 $k_0 = 0$。
  - 第二组：一个**更宽更矮**的高斯峰，中心在 $k_0 = \pi/2$ 附近。
- **右：环上的实空间波包**（一个斜看的圆环，环上竖起一排“栅栏”表示 $|\psi|^2$）。
  - 两组都是**局域在环的一小段**上的高斯包。（两张右图看起来几乎一样——因为初态在实空间都是高斯包，区别在**动量**，而动量体现在中间那张图的宽度上。）

**要点展开**

**紧束缚色散**：把 $H = -\sum_j (a_j^\dagger a_{j+1} + \text{h.c.})$ 做傅里叶变换，

$$\varepsilon_k = -2J\cos k, \qquad k = \frac{2\pi m}{N},$$

**能带宽度** $W = 4J$（从 $-2J$ 到 $+2J$）。（幻灯片图上纵轴只画到 $\pm 1$，是示意/归一化的画法；重要的是**形状**：反余弦。）

**群速度** $v_g(k) = \dfrac{d\varepsilon_k}{dk} = 2J\sin k$。这解释了红圈的位置：

| 初态 | $k_0$ | $v_g = 2J\sin k_0$ | 物理图像 |
|---|---|---|---|
| 第一组 | 0（带底） | 0 | **波包不动**，只**扩散**（色散） |
| 第二组 | $\pi/2$（拐点） | $2J$（最大） | **波包以最大速度弹道传播** |

**“高斯波包”的构造**：

$$|\psi(0)\rangle = \frac{1}{\sqrt{\Omega}} \sum_j \underbrace{e^{-(\alpha^2/2)(j-N_0)^2}}_{\text{实空间包络}} \underbrace{e^{\mathrm{i}k_0 j}}_{\text{动量}} a_{j,\sigma}^\dagger |0\rangle.$$

- $N_0$ 是波包中心，$\alpha$ 控制宽度（$\alpha$ 小 = 包宽；$\alpha$ 大 = 包窄），$\Omega$ 是归一化常数（第 30 页给出）。
- **实空间越窄 $\Rightarrow$ 动量空间越宽**（傅里叶变换的不确定性关系），这正是中间两张“棒棒糖”图宽度不同的原因。

**物理意义**：这一页在演示“**色散 + 群速度**”这两个最基本的固体物理概念如何通过实时演化“看得见”。它是下一面“非扩散波包”的铺垫。

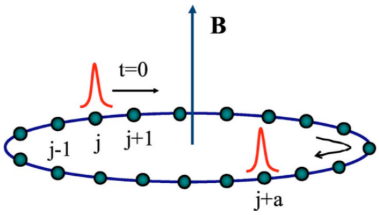
**提示 / 易错**

- $E_k$ **的纵轴刻度**：图上标的是 $\pm 1$，而 $H = -\sum(a^\dagger a + \text{h.c.})$ 给出的色散是 $\varepsilon_k = -2J\cos k$。也就是说，**图里画的是** $\varepsilon_k/2J = -\cos k$ （纵轴以带宽的一半为单位；$J = 1$ 时最小值 $-1$ 在 $k = 0$，最大值 $+1$ 在 $k = \pm\pi$）。关键是“最小值在 $k = 0$、最大值在 $k = \pm\pi$、拐点在 $k = \pm\pi/2$”这个形状。
- $k_0 = 0$ **的波包不动，但会扩散**（因为不同 $k$ 分量速度不同）；$k_0 = \pi/2$ **的波包会跑，而且跑得最快**。这是理解第 30–31 页的关键。
- $\alpha$ **与包宽的关系**：包宽 $\sim 1/\alpha$。第 30 页的图用的是 $\alpha = 0.1$，对应一个很宽的包（约 10 个格点）。

## 第 30 页 · 实时演化实例（二）：环上磁通与非扩散波包

# Real time evolution example

- a ring lattice with $N$ sites threaded by a magnetic field




$$H[\phi] = -J\sum_{j,\sigma}(e^{i2\pi\phi/N}a^{\dagger}_{j,\sigma}a_{j+1,\sigma} + \mathrm{H.c.})$$

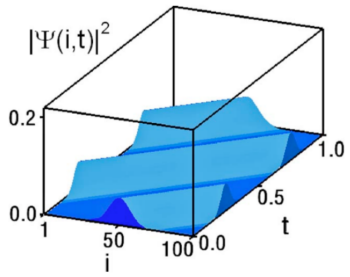



FIG. 2. (Color online) Numerical simulation of the time evolution of a zero-momentum Gaussian wave packet (GWP) with $\alpha$ =0.1 in position-space $i$ of the 100-site ring with $\phi$=$N$/4. The time $t$ is in the unit of 100/$J$.

- time evolution of a Gaussian wave packet with momentum $k_0$

$$|\psi(t=0)\rangle = \frac{1}{\sqrt{\Omega}}\sum_j e^{-(\alpha^2/2)(j-N_0)^2}e^{ik_0 j}a^{\dagger}_{j,\sigma}|0\rangle$$

$$\Omega = \sum_j \exp[-\alpha^2(j-N_0)^2]$$

- non-spreading wave packet evolution
- solid-state flying qubit

S. Yang, Z. Song, and C. P. Sun, PRA 73, 022317 (2006)

图 30: 幻灯片第 30 页

### 这一页写了什么

标题 *Real time evolution example*。

- **a ring lattice with $N$ sites threaded by a magnetic field** （被磁场穿过的 $N$ 格点环）
- **time evolution of a Gaussian wave packet with momentum**：

$$|\psi(t=0)\rangle = \frac{1}{\sqrt{\Omega}}\sum_j e^{-(\alpha^2/2)(j-N_0)^2}e^{ik_0 j}a^{\dagger}_{j,\sigma}|0\rangle,$$

$$\Omega = \sum_j \exp[-\alpha^2(j-N_0)^2].$$

- **non-spreading wave packet evolution**（非扩散波包演化）
- **solid-state flying qubit**（固态飞行量子比特）
- 引用：**S. Yang, Z. Song, and C. P. Sun, PRA 73, 022317 (2006)**。

### 图里画了什么

- **左上：环格点示意图**。一个斜看的椭圆环（深蓝），上面有约 16 个青绿色格点；上弧有三个相邻格点标着 $j-1$, $j$, $j+1$；在格点 $j$ 处有一个红色高斯峰，右边有一个黑色箭头标 $t=0$；环中心有一根竖直蓝色箭头标 $B$（穿过环的磁通）；右下方另一个红色高斯峰标 $j+a$，表示后来的位置；环右侧有一个小的弧形双向箭头表示环流方向。环下方是哈密顿量：

$$H[\phi] = -J\sum_{j,\sigma}\Big(e^{\mathrm{i}2\pi\phi/N}a^{\dagger}_{j,\sigma}a_{j+1,\sigma} + \mathrm{H.c.}\Big).$$

- **右上：3D 曲面图** $|\Psi(i,t)|^2$。纵轴 $|\Psi|^2$（标 0.0 与 0.2），一个底面轴是格点 $i$（标 1、50、100），另一个底面轴是时间 $t$（标 0.0、0.5、1.0）。曲面是一条**沿对角线的窄脊**：波包从 $i\approx 1$ 出发，随时间**整体平移**到 $i\approx 100$，**脊的宽度几乎不变**——这就是"非扩散"。图下方是论文原图注：

> **FIG. 2.** (Color online) Numerical simulation of the time evolution of a zero-momentum Gaussian wave packet (GWP) with $\alpha = 0.1$ in position-space $i$ of the 100-site ring with $\phi = N/4$. The time $t$ is in the unit of $100/J$.

**要点展开**

**Peierls 替换 / 磁通**：环上穿一个磁通 $\phi$（以磁通量子为单位），跃迁振幅要乘一个相位 $e^{\mathrm{i}2\pi\phi/N}$（沿环走一圈相位累积 $2\pi\phi$）。**这个相位的效果等价于给波包一个动量**：

$$k_0 = \frac{2\pi\phi}{N}.$$

（论文里就写成 $k_0 = 2\pi\phi/N$，见第 31 页的图注。）

**"非扩散波包"（non-spreading wave packet）**是这一页的核心物理。一般的高斯波包会**扩散**（因为不同 $k$ 分量的群速度不同，包会越变越宽）。但在这条**余弦能带**上，如果初态是"动量空间的高斯包"并且选择**合适**的 $k_0$，波包在传播过程中**几乎不扩散**——原因是：

- 在拐点附近（$k_0\approx\pi/2$），色散**近似线性**：在 $k=\pi/2$ 处展开 $\varepsilon_k = -2J\cos k \approx 2J\,(k-\pi/2)$，线性色散意味着**所有分量速度相同**（$v_g = 2J$）⇒ 不扩散；
- 更准确地说，这是一个"**自相似**"的相干态，其形状在演化中保持不变。

**"固态飞行量子比特"（solid-state flying qubit）**是把这件事推向应用的设想：用一个**不扩散的波包**（例如自旋向上/向下两个分量）在固体里传输量子信息——它天然是"飞行量子比特"的候选载体。这是作者（杨硕老师本人）2006 年的工作。

**提示 / 易错**

- **图注写的是"zero-momentum GWP"（$k_0 = 0$），但同一图注又给了 $\phi = N/4$**，而正文 $k_0 = 2\pi\phi/N$ 会把 $\phi = N/4$ 换算成 $k_0 = \pi/2$。**这两者不自洽**，很可能是在引用论文图注时把另一幅图（$k_0 = 0$ 的那幅）的文字一起抄了过来。读图时**以参数 $\phi = N/4$、$k_0 = \pi/2$ 为准**（图里那个波包明显在跑，不是静止的）。
- $t$ **的单位是** $100/J$，也就是图上 $t = 1.0$ 对应真实时间 $100/J$。做个量级估计：群速度 $v_g = \mathrm{d}\varepsilon_k/\mathrm{d}k = 2J\sin k$，在 $k_0 = \pi/2$ 处 $v_g = 2J$（格点/单位时间），绕 100 格点的环跑一圈需要 $100/(2J) = 50/J$，对应图上 $t = 0.5$。所以 $t\in[0,1]$ 大致是"绕环两圈"的演化，波包会与自身相遇、干涉。
- **磁通的作用不是"只改一个相位"**：Peierls 替换把跃迁相位 $e^{\mathrm{i}2\pi\phi/N}$ 引进哈密顿量，等价于把能带整体在动量空间平移，于是波包的中心动量变成 $k_0 = 2\pi\phi/N$，**群速度** $v_g = 2J\sin k_0$ **随之改变**。第

29 页“$k_0 = 0$ 不动、$k_0 = \pi/2$ 跑得最快”就是同一个机制。（单粒子概率密度确实对规范变换不变，但那是“换规范”，不是“换磁通”——换磁通是换物理。）

---

## 第 31 页 · 实时演化实例（三）：开边界、自干涉与量子复兴

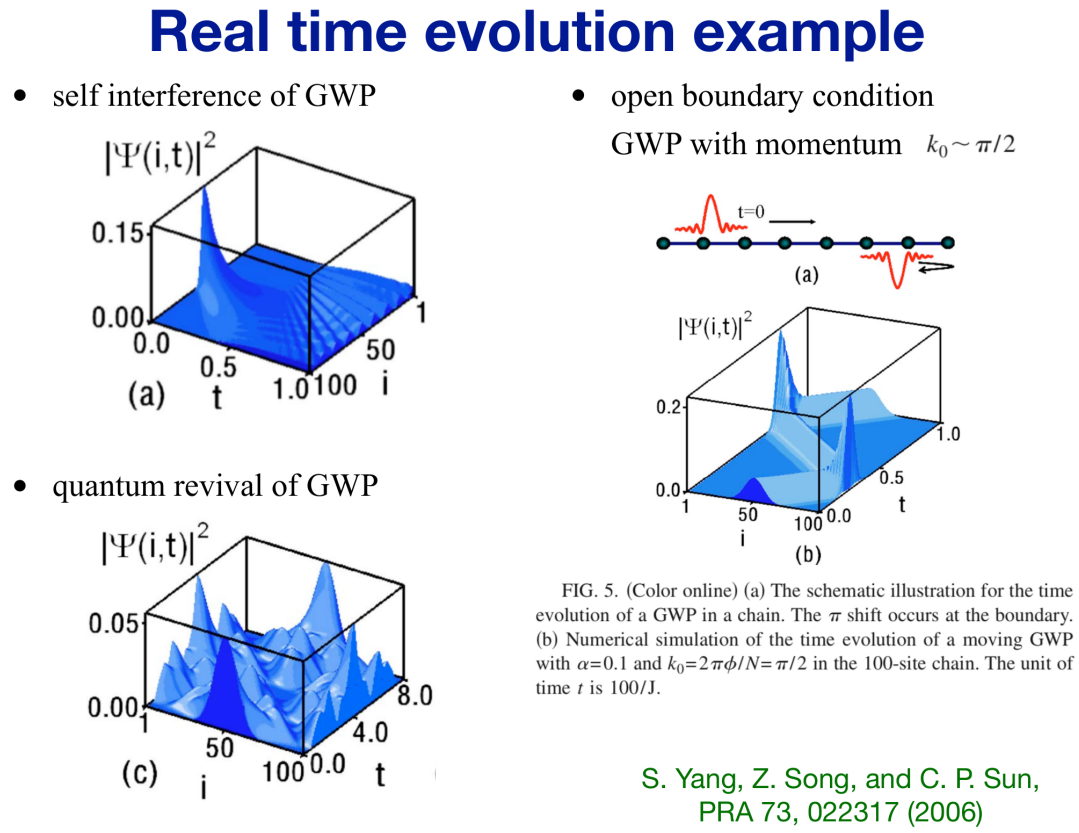



图 31: 幻灯片第 31 页

### 这一页写了什么

标题 *Real time evolution example*。

- **self interference of GWP**（高斯波包的自干涉）
- **quantum revival of GWP**（高斯波包的量子复兴）
- **open boundary condition**（开边界条件）
- **GWP with momentum** $k_0 \sim \pi/2$

图注（论文 FIG. 5，逐字）：

> **FIG. 5.** (Color online) (a) The schematic illustration for the time evolution of a GWP in a chain. The $\pi$ shift occurs at the boundary. (b) Numerical simulation of the time evolution of a moving GWP with $\alpha = 0.1$ and $k_0 = 2\pi\phi/N = \pi/2$ in the 100-site chain. The unit of time $t$ is $100/J$.

引用：**S. Yang, Z. Song, and C. P. Sun, PRA 73, 022317 (2006)**。

**图里画了什么**

- **右上：示意图 (a)**。一条水平的一维链（蓝线 + 深青色格点），左端有一个红色振荡的高斯波包标 $t = 0$，一个向右的箭头表示它向右运动；在链的右端，第二个红色波包被画成**倒置的（在链下方）**，配一个向左的箭头——表示**从开边界反射，并带一个 $\pi$ 相移**。
- **左上：3D 曲面 (a)（自干涉）**。纵轴 $|\Psi(i,t)|^2$（0.00 与 0.15），底面轴 $t$（0.0、0.5、1.0）与 $i$（1、50、100）。曲面从早期一个高而窄的峰开始，随时间**铺展开**、变矮，并出现**振荡的干涉条纹**——这就是“自干涉”。
- **右中：3D 曲面 (b)（运动的波包，开边界）**。纵轴 $|\Psi|^2$（0.0 与 0.2），底面轴 $i$（1、50、100）与 $t$（0.0、0.5、1.0）。一条**紧凑的移动高斯包**沿链传播，到达 $i = 100$ 的开边界后出现**反射的脊**。
- **左下：3D 曲面 (c)（量子复兴）**。纵轴 $|\Psi|^2$（0.00 与 0.05），底面轴 $i$（1、50、100）与 $t$（0.0、4.0、8.0，注意时间范围大得多）。曲面高度振荡：初始局域的包**碎裂成多个极大**，然后在 $t \approx 8$ 附近**重新聚拢**——这就是**量子复兴**。

**要点展开**

这一页讲三个现象，都是“波包在有限系统里演化”的必然结果：

1. **开边界的反射与 $\pi$ 相移**。在开边界处，波包被反射。对这条链（$\varepsilon_k = -2J\cos k$，$k_0 = \pi/2$），反射会带来一个 $\pi$ **的相位跳变**（示意图 (a) 中倒置的波包就是它）。这个 $\pi$ 相移是“硬壁边界条件”的普遍特征（和无限深势阱里波函数在壁上为零同理）。

2. **自干涉（self interference）**。因为系统是**有限大小**的，波包绕一圈（或反射回来）后会与“自己”相遇并干涉，于是 $|\Psi(i,t)|^2$ 出现**振荡条纹**——图 (a) 里那些细密的波纹。

3. **量子复兴（quantum revival）**。有限系统的能谱是**离散**的，所以演化是**准周期**的：经过一段时间后，各个相位 $e^{-\mathrm{i}E_n t}$ 重新对齐，波包**恢复到初始形状**。复兴时间由能谱的（近似）等间距决定：

$$T_{\text{revival}} \sim \frac{2\pi}{\Delta E} \quad (\Delta E = \text{相邻能级间距}).$$

   图 (c) 的时间范围（0 到 8，单位 $100/J$）比 (a)(b) 大一个量级，正是为了看到复兴。

**这三个现象在物理上的意义**：

- 它们是**有限尺寸效应**的直接可视化，也是“为什么小系统不能用热力学极限的语言描述”的最好例子。
- **量子复兴**在冷原子、超导量子比特、离子阱里都已经被实验观测到，是“量子相干性”的漂亮演示。
- **反射的 $\pi$ 相移 + 自干涉**合起来说明：**在有限链上，“飞行量子比特”必须考虑边界的相干反射**——这是把第 30 页的设想变成器件时要解决的问题。

**提示 / 易错**

- 三个 3D 图的**纵轴刻度不同**：(a) 到 0.15、(b) 到 0.2、(c) 到 0.05。**不要直接比较峰高**，要看“包是否保持形状”。
- **面板编号有重复**：左上的图被标为 (a)，但论文里 (a) 是示意图、(b) 是演化图；左下被标为 (c)。**这是幻灯片排版造成的，读图时看纵轴和横轴范围来区分。**

- $\pi$ **相移在 $|\Psi|^2$ 上看不出来**（相位不影响模方），它只体现在**波函数本身**（以及干涉条纹的位置）上。所以示意图 (a) 用“倒置的波包”来表示相位的反号，而不是模方的变化。

---

### 第 32 页 · 非厄米哈密顿量：双正交基与时间演化

# Non-Hermitian Hamiltonian

- non-Hermitian Hamiltonian

$$H \neq H^\dagger$$

- bi-orthogonal basis

$$H\left|F_n\right\rangle = \omega_n \left|F_n\right\rangle$$
$$\left\langle G_n\right| H = \left\langle G_n\right| \omega_n \qquad \text{or} \quad H^\dagger \left|G_n\right\rangle = \omega_n^* \left|G_n\right\rangle$$

- identity operator

$$\mathbb{I} = \sum_n \frac{\left|F_n\right\rangle \left\langle G_n\right|}{\left\langle G_n | F_n\right\rangle}$$

- time-evolution operator

$$e^{-iHt} = \sum_n \frac{\left|F_n\right\rangle e^{-i\omega_n t} \left\langle G_n\right|}{\left\langle G_n | F_n\right\rangle}$$

this in principle solves all the dynamics

图 32: 幻灯片第 32 页

#### 这一页写了什么

标题 *Non-Hermitian Hamiltonian*。四条：

- **non-Hermitian Hamiltonian**：

$$H \neq H^\dagger.$$

- **bi-orthogonal basis**（双正交基）：

$$H|F_n\rangle = \omega_n |F_n\rangle,$$

$$\langle G_n|H = \langle G_n|\omega_n \qquad \text{or} \qquad H^\dagger|G_n\rangle = \omega_n^*|G_n\rangle.$$

- **identity operator**（单位算符）：

$$\mathbb{1} = \sum_n \frac{|F_n\rangle\langle G_n|}{\langle G_n|F_n\rangle}.$$

- **time-evolution operator**（时间演化算符）：

$$e^{-\mathrm{i}Ht} = \sum_n \frac{|F_n\rangle\, e^{-\mathrm{i}\omega_n t}\,\langle G_n|}{\langle G_n|F_n\rangle}.$$

最后一行：**this in principle solves all the dynamics**（这在原则上解决了所有的动力学问题）。

**要点展开**

**厄米哈密顿量**（$H = H^\dagger$）的本征态是**正交归一**的：$\langle\psi_m|\psi_n\rangle = \delta_{mn}$，单位算符是 $\sum_n |\psi_n\rangle\langle\psi_n|$。**非厄米哈密顿量**（$H \neq H^\dagger$）破坏了这个性质：

- **左右本征矢不同**：右本征矢 $|F_n\rangle$ 满足 $H|F_n\rangle = \omega_n|F_n\rangle$，左本征矢 $\langle G_n|$ 满足 $\langle G_n|H = \omega_n\langle G_n|$（等价地 $H^\dagger|G_n\rangle = \omega_n^*|G_n\rangle$）。
- **本征值可以是复数**：$\omega_n \in \mathbb{C}$。虚部有明确的物理意义：$\mathrm{Im}\,\omega_n < 0$ 表示**衰减**（耗散），$\mathrm{Im}\,\omega_n > 0$ 表示**增益**。
- **本征矢不正交**：$\langle F_m|F_n\rangle \neq 0$（$m \neq n$），而且可能出现**例外点（exceptional point）**——本征值与本征矢同时简并，本征矢塌缩成一个（Jordan 块），这是非厄米系统特有的奇异现象。

**“双正交基”的解决办法**：把左右本征矢**配对**使用。定义**双正交归一化** $\langle G_m|F_n\rangle = \delta_{mn}\langle G_n|F_n\rangle$，就得到

$$\mathbb{1} = \sum_n \frac{|F_n\rangle\langle G_n|}{\langle G_n|F_n\rangle}, \qquad e^{-\mathrm{i}Ht} = \sum_n \frac{|F_n\rangle e^{-\mathrm{i}\omega_n t}\langle G_n|}{\langle G_n|F_n\rangle}.$$

**第二个公式就是非厄米版的“谱分解演化”**与第 28 页的厄米公式逐项对应：

| 厄米 | 非厄米 |
|---|---|
| $H\|\psi_n\rangle = E_n\|\psi_n\rangle$ | $H\|F_n\rangle = \omega_n\|F_n\rangle$，$\langle G_n\|H = \langle G_n\|\omega_n$ |
| $E_n$ 实 | $\omega_n$ 复 |
| $\langle\psi_m\|\psi_n\rangle = \delta_{mn}$ | $\langle G_m\|F_n\rangle = \delta_{mn}\langle G_n\|F_n\rangle$ |
| $\mathbb{1} = \sum_n \|\psi_n\rangle\langle\psi_n\|$ | $\mathbb{1} = \sum_n \|F_n\rangle\langle G_n\|/\langle G_n\|F_n\rangle$ |
| $e^{-\mathrm{i}Ht} = \sum_n \|\psi_n\rangle e^{-\mathrm{i}E_n t}\langle\psi_n\|$ | $e^{-\mathrm{i}Ht} = \sum_n \|F_n\rangle e^{-\mathrm{i}\omega_n t}\langle G_n\|/\langle G_n\|F_n\rangle$ |

**为什么课程要讲这个？**因为非厄米哈密顿量在今天非常活跃：

- **耗散系统**：开放量子系统、冷原子中的粒子损失、光子晶体、波导 QED；
- **$\mathcal{PT}$ 对称系统**：本征值可以是实的（$\mathcal{PT}$ 对称破缺相变），以及**例外点**附近的奇异响应（传感器、单向传输）；
- **非厄米趋肤效应**：开边界下的本征态全部局域在边界上（体-边对应被破坏）；
- **物理实现**：光学/声学超材料、电路、冷原子。

**提示 / 易错**

- $\omega_n^*$ **的共轭**：$H^\dagger|G_n\rangle = \omega_n^*|G_n\rangle$ 里的星号来自 $(\langle G_n|H)^\dagger = H^\dagger|G_n\rangle = (\omega_n\langle G_n|)^\dagger = \omega_n^*|G_n\rangle$。**别漏掉星号。**
- $\langle G_n|F_n\rangle \neq 0$ **是必需的**（非简并时通常成立），否则分母为零、双正交基失效（这正是例外点附近的困难）。
- **本征值是复数时，“能量”不再是可观测量**，$e^{-\mathrm{i}\omega_n t}$ 会衰减/增长。这在**开放系统**里有意义，但如果你是在做一个封闭系统的 ED 而得到了复数本征值，**多半是** $H$ **写错了（不厄米）**——例如费米子符号漏了、或者矩阵元写反了。这是一个很好的**自动检查**。
- **厄米情形是特例**：当 $H = H^\dagger$ 时 $|G_n\rangle = |F_n\rangle$，分母变成 1，一切回到第 28 页的公式。

---

# 第七部分：作业（第 33–34 页）

## 第 33 页 · 作业规则

**Homework rules**

please submit through http://learn.tsinghua.edu.cn

- please submit the source code, detailed notes explaining your code and results, and your log with AI
- deadline is Sep. 27, 23:00
- if you submit today, you will have more points
- if you submit after the deadline, you will have less points
- if you are an expert in exact diagonalization, this homework may be waived
  please contact me in person and submit something you have done before

图 33: 幻灯片第 33 页

### 这一页写了什么

标题 *Homework rules*。标题下面**紧跟一行缩进的提交网址**（蓝色下划线）：

> **please submit through http://learn.tsinghua.edu.cn**（通过清华网络学堂提交）

然后是五条要点：

- **please submit the source code, detailed notes explaining your code and results, and your log with AI**（请提交**源代码**、解释代码与结果的**详细说明**，以及你与 **AI 的对话记录**；其中 *source code* 与 *detailed notes* 在幻灯片上是**蓝色**的）
- **deadline is Sep. 27, 23:00**（截止时间 9 月 27 日 23:00；幻灯片上 *deadline* 是**红色**、日期是**蓝色**，是全页最醒目的一条）
- **if you submit today, you will have more points**（今天交有加分）
- **if you submit after the deadline, you will have less points**（迟交扣分）
- **if you are an expert in exact diagonalization, this homework may be waived**
  - **please contact me in person and submit something you have done before**（如果你是 ED 专家，可以免做，但请当面联系我并提交你以前做过的东西）

**要点展开**

三条要求，值得逐条理解：

1. **提交渠道**：通过清华网络学堂 http://learn.tsinghua.edu.cn 提交。网址被单独提到标题下面、并用蓝色下划线标出，是这一页第一眼看到的信息——交作业的地方和作业本身一样重要。具体题目见作业文档。
2. **提交三样东西**：源码 + 详细说明 + **AI 对话记录**。注意第三项——这门课**明确鼓励用 AI 写代码**，但要求你**诚实地记录**你问了什么、AI 给了什么、你怎么判断对错。这与第一讲第 36 页“良好的编程习惯”里强调的**写注释、用版本控制、如实记录每一步是怎么做的**一脉相承（那一页的第 4 条与第 7 条）。**AI 记录也是评分项（第 34 页 [+20]）**。
3. **时间分**：今天交加分、迟交每天扣分（第 34 页给出具体数字）。**这是一个“鼓励早做”的机制**，因为这门课的作业是“越早开始越容易”。

**“免做”条款**的意义：课程假设学生的背景差异很大。如果你已经在科研里做过 ED，不必重复劳动，但要**当面说明**并提交**以前的工作**——这样老师能确认你真的掌握了，也避免你浪费时间。

**提示 / 易错**

- **“detailed notes”是硬性要求**，不是可选的。第 34 页的评分表里，说明文档占 **40 分**（10 + 10 + 10 + 10），与源码的 40 分**一样重**。
- **AI log 要真实**：把与 AI 的对话（prompt 与回答的关键部分）保存下来。**不要**只贴“最终代码”。
- **截止日是 9 月 27 日 23:00**，也就是本讲（9 月 21 日）之后第 6 天；幻灯片上“submit today”的“today”指的是**讲课当天**。别把“还有一周”当成“还有很久”。

---

### 第 34 页 · 本次作业评分标准

# Grading of this homework

- the source code can be executed [+10] and can provide correct results [+20]
  the source code has high readability [+10]
- there is a detailed note file to explain (1) basic principle [+10], (2) source code [+10], and (3) results [+10]
  the note file is well-written and easy to understand [+10]
- your log with AI [+20]
- did not use $U(1)$ symmetry [-20]
- time-dependent score: today: [+5], after DDL: [-1 per day]
- self-motivation score: new ideas or extra results [+10]

图 34: 幻灯片第 34 页

#### 这一页写了什么

标题 *Grading of this homework*。

- **the source code can be executed [+10] and can provide correct results [+20]**
  - **the source code has high readability [+10]**
- **there is a detailed note file to explain (1) basic principle [+10], (2) source code [+10], and (3) results [+10]**
  - **the note file is well-written and easy to understand [+10]**
- **your log with AI [+20]**
- **did not use U(1) symmetry [-20]**
- **time-dependent score: today: [+5], after DDL: [-1 per day]**
- **self-motivation score: new ideas or extra results [+10]**

#### 要点展开

把评分表整理成一张清单：

| 项目 | 分值 |
|---|---|
| 源码可执行 | +10 |

| 项目 | 分值 |
|---|---|
| 源码给出正确结果 | +20 |
| 源码可读性高 | +10 |
| 说明文档：基本原理 | +10 |
| 说明文档：源码解释 | +10 |
| 说明文档：结果分析 | +10 |
| 说明文档写得清楚易懂 | +10 |
| 与 AI 的对话记录 | +20 |
| **未使用 U(1) 对称性** | **−20** |
| 时间分：今天交 / 迟交 | +5 / 每天 −1 |
| 自驱分：新想法或额外结果 | +10 |

**几个观察：**

1. **总分基数 100**（$10+20+10+10+10+10+10+20=100$），再加上时间分（+5）与自驱分（+10）作为加分，减去 U(1) 的 −20。
2. **“正确结果 +20”与“AI 记录 +20”并列最大单项**——这门课最看重的还是**算对**。而“算对”的标准就是第 15 页那张验证清单（已知极限、对称性、求和规则、两种方法对照）。
3. **说明文档总共 40 分**（基本原理 10 + 源码解释 10 + 结果分析 10 + 写得清楚 10），与源码的 40 分（可执行 10 + 正确结果 20 + 可读性 10）**恰好相等**。这传达的信息很清楚：**“能跑”只完成了一半，“讲清楚”才算完成**。这与第一讲第 36 页的八条编程习惯完全一致。
4. **U(1) 是硬性要求（−20）**。第 17 页讲了怎么实现：按 $N_\uparrow$ 分桶、桶内重编号、只对角化目标扇区。**不做 U(1) 的代价是维数大 $\sqrt{\pi N/2}$ 倍**（$N=40$ 时约 8 倍），若用完整对角化，时间还要再乘 $(\pi N/2)^{3/2}$（约 500 倍），在大系统上根本跑不动——所以扣分很重。
5. **自驱分 [+10]**：鼓励你多做一点——例如多算一个物理量、做一次有限尺寸标度、加一个对称性、和文献对比。这是“从完成作业到做研究”的那一步。

**提示 / 易错**

- “**did not use U(1) symmetry [-20]**”是**扣分**不是“不加分”。也就是说，即使你的代码完全正确，只要没有用 U(1)，最高只能拿 80 分。
- **时间分**：今天交 +5，之后每过一天 −1。9 月 27 日 23:00 是截止时间。
- “**self-motivation score**”需要你在说明文档里**明确写出**你额外做了什么，否则老师可能看不到。

---

# 附录

## 附录 A · 中英术语对照

| 中文 | English | 幻灯片页 |
|---|---|---|
| 精确对角化 | exact diagonalization (ED) | 2 |

| 中文 | English | 幻灯片页 |
|---|---|---|
| 哈密顿量的矩阵表示 | matrix representation of Hamiltonian | 2、7 |
| 稀疏矩阵 | sparse matrix | 7、14 |
| 无矩阵（矩阵实时重算） | matrix-free / matrix recalculation on the fly | 7、27 |
| 完全对角化 | complete (full) diagonalization | 7、25 |
| 迭代对角化 | iterative diagonalization | 7、26 |
| 希尔伯特空间 | Hilbert space | 4、7 |
| 基矢 / 基矢表示 | basis state / basis representation | 7、8 |
| 基矢查找 | basis lookup | 7、18 |
| 比特运算 | bit operations | 9、10、11 |
| 置位 / 清零 / 翻转 / 读位 | set / clear / flip / read bit | 10 |
| 数位（1 的个数） | popcount | 11 |
| 取位 | pick bits | 11 |
| 循环移位 | circular bit shift | 11 |
| 海森堡模型 | Heisenberg model | 12 |
| 反铁磁 | antiferromagnet (AF) | 12 |
| 周期性边界条件 | periodic boundary condition (PBC) | 12 |
| 对角元 / 非对角元 | diagonal / off-diagonal terms | 12、13 |
| 升降算符 | raising / lowering operator ($S^{\pm}$) | 13 |
| 三元组（行、列、值） | (row, column, value) triplets / COO | 14 |
| 对称性 / 块对角化 | symmetry / block diagonalization | 16 |
| 平移对称性 | translation symmetry | 16 |
| 宇称 / 反射对称性 | parity / reflection symmetry | 16 |
| 自旋反演对称性 | spin-inversion symmetry | 16 |
| 点群 | point group (PG) | 16 |
| 二十-十二面体 | icosidodecahedron | 16 |
| 子空间（扇区） | subspace / sector | 17 |
| 组合数 | binomial coefficient | 17 |
| Lin 表 | Lin table | 18、19、20 |
| 两次直接内存读取 | two direct memory reads | 18 |
| 费米子系统 | fermionic systems | 21、22 |
| Hubbard 模型 | Hubbard model | 21 |
| 反对易关系 | anticommutation relation | 21 |
| 算符顺序约定 | operator ordering convention (↑ left of ↓) | 21 |
| 有限温度 | finite temperature | 23 |

| 中文 | English | 幻灯片页 |
|---|---|---|
| 配分函数 / 热平均 | partition function / thermal average | 23 |
| 电荷密度 | charge density | 23 |
| 化学势 | chemical potential ($\mu$) | 23 |
| Mott 平台 / 关联能隙 | Mott plateau / correlation gap | 23 |
| 磁化 | magnetization | 24 |
| 均匀 / 交错磁化 | uniform / staggered magnetization | 24 |
| 磁化率 | susceptibility ($\chi$) | 24 |
| 居里定律 | Curie's law ($\chi = C/T$) | 24 |
| 涨落–耗散定理 | fluctuation–dissipation theorem | 24 |
| 实时演化 | real time evolution | 28、29、30、31 |
| 含时薛定谔方程 | time-dependent Schrödinger equation | 28 |
| Krylov 空间 | Krylov space | 28 |
| Lanczos 矢量 | Lanczos vectors | 28 |
| 高斯波包 | Gaussian wave packet (GWP) | 29、30、31 |
| 色散关系 | dispersion relation | 29 |
| 群速度 | group velocity | 29 |
| 非扩散波包 | non-spreading wave packet | 30 |
| 固态飞行量子比特 | solid-state flying qubit | 30 |
| 磁通（Peierls 替换） | magnetic flux (Peierls substitution) | 30 |
| 自干涉 | self interference | 31 |
| 量子复兴 | quantum revival | 31 |
| 开边界条件 | open boundary condition (OBC) | 31 |
| 非厄米哈密顿量 | non-Hermitian Hamiltonian | 32 |
| 双正交基 | bi-orthogonal basis | 32 |
| 左右本征矢 | left / right eigenvector | 32 |
| 例外点 | exceptional point | 32 |
| 单位算符 | identity operator | 32 |
| 时间演化算符 | time-evolution operator | 32 |

---

## 附录 B ·常见错误清单（写 ED 代码前先读一遍）

1. **格点编号方向搞反**。本讲：格点 $i$ = 第 $i$ 位，格点 0 在最右边。若与第一讲的 `np.kron` 路线混用，格点顺序会反，所有局域量都错。
2. **`Nl = 2**Ns` 写成 `2*Ns`**（应为 `2**Ns`，即 $2^N$）。
3. **`ReadBit` 忘了 `>> n`**，返回 0 或 $2^n$，导致 $S_i^z = \text{ReadBit} - 0.5$ 变成 $-0.5$ 或 $2^n - 0.5$。

4. **对角元被错误地放进 `if` 里**，只有"自旋相反"的基矢才有对角元，漏掉 $+\frac{1}{4}$ 的贡献。
5. **环上漏掉最后一条键** $(N-1,0)$（或反过来，在开链上多加了一条）。
6. **非对角元的行、列写反**。实厄米 $H$ 时矩阵不变（$H^{\mathsf{T}}=H$），谱不会错；但复厄米 $H$ 时写反等于用了 $H^{*}$，**关联函数与时间演化都会错**。
7. **`eig` 与 `eigh` 混用**：哈密顿量是厄米的，用 `eigh`/`eigsh`；`eig` 返回复数且不排序。
8. **`which='SM'` 找基态**：应该用 `'SA'`（对 `eigsh`）。
9. **费米子符号**：↓ 的插入/删除**多一个** $\sum_k n_{k\uparrow}$ **因子**（第 22 页粉色框）；忘了它，本征值会错。
10. **Lin 表的 off-by-one**：$J_a$ 从 1 开始、$J_b$ 从 0 开始；论文用 1-based，代码用 0-based。
11. **用普通移位代替循环移位**做平移，破坏周期性边界。
12. **bitwise 优先级**：`x & 1 == 1` 会被解析成 `x & (1 == 1)`，永远加括号。
13. **不检查厄米性**：matrix-free 时没有矩阵可查，建议在构造完后用稠密小系统验证 `(H - H.T).nnz == 0`。
14. **不用 U(1) 对称性**：最大扇区维数只有 $2^N/\sqrt{\pi N/2}$，也就是**用 U(1) 能省** $\sqrt{\pi N/2}$ **倍**（$N=40$ 时约 8 倍）——这正是作业里"不用 U(1) 扣 20 分"的原因。
15. **不写验证**：只算出一个数就交作业。**必须做**：已知极限（$J=0$、$U=0$、$N=2$）、对称性、求和规则（$\mathrm{Tr}\,H=0$）、两种方法对照（稠密 vs 稀疏、ED vs 严格解）。

---

## 附录 C · 延伸阅读

### 精确对角化的方法综述

- H. Q. Lin, *Exact diagonalization of quantum-spin models*, Phys. Rev. B **42**, 6561 (1990). ——**Lin 表**的原始论文（第 18–20 页）。
- J. Schnack, *Exact diagonalization techniques*, in *Computational Many-Particle Physics*, Lecture Notes in Physics **739**, Springer (2008). ——综述章节。
- A. Weiße, H. Fehske, *Exact diagonalization techniques*, Rev. Comput. Chem. **23**, 529 (2008). ——另一种综述，含大量实现细节。
- N. Laflorencie, D. Poilblanc, *Algorithms for spin systems*, 见 *Strongly Correlated Systems: Numerical Methods*, Springer (2013).

### Krylov / Lanczos

- C. Lanczos, *An iteration method for the solution of the eigenvalue problem of linear differential and integral operators*, J. Res. Natl. Bur. Stand. **45**, 255 (1950).
- R. B. Lehoucq, D. C. Sorensen, C. Yang, *ARPACK Users' Guide*, SIAM (1998). ——`eigs`/`eigsh` 的底层。
- T. J. Park, J. C. Light, *Unitary quantum time evolution by iterative Lanczos reduction*, J. Chem. Phys. **85**, 5870 (1986). ——实时演化的 Lanczos 方法。

### 费米子符号与 Hubbard 模型

- J. Hubbard, *Electron correlations in narrow energy bands*, Proc. R. Soc. Lond. A **276**, 238 (1963).
- E. H. Lieb, F. Y. Wu, Phys. Rev. Lett. **20**, 1445 (1968).

- J. E. Hirsch, *Two-dimensional Hubbard model: Numerical simulation study*, Phys. Rev. B **31**, 4403 (1985). ——早期 ED + QMC。


**非厄米物理**


- C. M. Bender, S. Boettcher, *Real spectra in non-Hermitian Hamiltonians having $\mathcal{PT}$ symmetry*, Phys. Rev. Lett. **80**, 5243 (1998). ——$\mathcal{PT}$ 对称的开创性工作。
- Y. Ashida, Z. Gong, M. Ueda, *Non-Hermitian physics*, Adv. Phys. **69**, 249 (2020). ——综述。
- E. J. Bergholtz, J. C. Budich, F. K. Kunst, *Exceptional topology of non-Hermitian systems*, Rev. Mod. Phys. **93**, 015005 (2021).


**实时演化 / 波包**


- S. Yang, Z. Song, C. P. Sun, *Non-spreading wave packet and solid-state flying qubit*, Phys. Rev. A **73**, 022317 (2006). ——第 29–31 页的原始工作。
- A. Weichselbaum, S. Capponi, P. Lecheminant, A. M. Tsvelik, A. Delft, *Unified approach to the time evolution of one-dimensional quantum systems* (综述).


**参考书**


- H. Fehske, R. Schneider, A. Weiße (eds.), *Computational Many-Particle Physics*, Lecture Notes in Physics **739**, Springer (2008).
- A. Avella, F. Mancini (eds.), *Strongly Correlated Systems: Numerical Methods*, Springer Series in Solid-State Sciences **176**, Springer (2013).


---

## 结语

第二讲的内容看起来很长（34 页、四个模块），但它的主线只有一条：

> **把一个 $2^N$ 维的量子多体问题，变成一个稀疏矩阵的本征值问题。**

这条主线可以拆成四步，每一步都对应讲义里的一块：

1. **表示**：基矢 ↔ 整数 ↔ 比特（第 8–11 页）。
2. **构造**：$H$ 的每一项 → “行、列、值”三元组（第 12–15、21–22 页）。
3. **压缩**：用对称性把大矩阵切成小块，用 Lin 表高效查找（第 16–20 页）。
4. **求解与测量**：完全/迭代对角化、有限温度、实时演化、非厄米（第 23–32 页）。

如果只带走三句话：

1. **ED 的全部技巧，都藏在“比特”里。**基矢是整数，算符是比特翻转，对称性是比特的置换，费米子符号是比特的计数。**把比特运算写对，ED 就写对了一半。**
2. **对称性是免费的午餐。** U(1) 是最容易用、收益最大的一种（第 17 页）；不做 U(1) 要扣 20 分，做对了最大扇区维数能降 $\sqrt{\pi N/2}$ 倍（$N = 40$ 时约 8 倍）。
3. **算出来只是开始，验证才算完成。**第 15 页那张验证清单（$N = 2$、$N = 4$ 环、迹为零、厄米性、非零元个数）请务必全部跑一遍——这是你以后所有 ED 工作的“地基检测”。

第三讲（ED II）会接着讲：如何用 U(1)/平移对称性把维数再压下去、如何做动力学关联函数与谱函数、如何用有限尺寸标度把结果外推到热力学极限。

祝你算得开心。